Afeka Academic College of Engineering in Tel-Aviv

School of Electrical Engineering

Project Name: Smart Thermal System for Patient Safety Monitoring

Engineering Report

Students: Guy Chen 206379620

Yaniv Blau 207106121

Roy Lieberman 316416502

Project Supervisor: Or Zilberberg

Project Advisor: Dr. Oshrit Hoffer

In collaboration with the Center for Mental Health in Be'er Sheva

Supervisor's approval of the document review and submission authorization:

_____

Date of Submission:

_____

Acronyms and Abbreviations

Acronyms and Abbreviations used in this report are listed below.

Acronym	Meaning	Acronym	Meaning
AI	Artificial Intelligence	mAP	Mean Average Precision
ANN	Artificial Neural Network	MCU	Micro-Controller Unit
ATD	Acoustic Threat Detector	ML	Machine Learning
CNN	Convolutional Neural Network	MLT	Multi-Level Transmit
CPU	Central Processing Unit	MLX	Short for Melexis (semiconductor and sensor company)
CRC	Cyclic Redundancy Check	MTU	Maximum Transmission Unit
CV	Computer Vision	PAM	Pulse Amplitude Modulation
FB	Fixed Bullet	PCB	Printed Circuit Board
FCS	Frame Check Sequence	POE	Power Over Ethernet
FH	Fixed Housing / Fixed Hybrid	PTZ	Pan-Tilt-Zoom
GPU	Graphics Processing Unit	RAM	Random Access Memory
GUI	Graphic User Interface	R-CNN	Regions with Convolutional Neural Network
HD	High-Definition	ReLU	Rectified Linear Unit
ID	Intrusion Detection	RT	Real-Time
IEEE	Institute of Electrical and Electronics Engineers	SCB	Single Board Computer
IIC / I2C / I^2C	Inter-Integrated Circuit	SCL	Serial Clock Line
ILSVRC	ImageNet Large Scale Visual Recognition Challenge	SDL	Serial Data Line
IoU	Intersection over Union	SFD	Start Frame Delimiter
IP	Internet Protocol	SGD	Stochastic Gradient Descent
IR	Infra-Red	SIM	Subscriber Identity Module
ISO	International Organization for Standardization	SOW	Statement of Work
LAN	Local Area Network	UI / UX	User Inteface / User Experience
LTE	Long Term Evolution	YOLO	You-Only-Look-Once
MAC	Media Access Control		

Supervisor's Approval

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Mr. Or Zilberberg, for his professional guidance and mentorship. His dedication was invaluable, particularly his encouragement to explore different solutions during the research phase. His belief in our abilities has inspired us to pursue these areas further in our future careers.

We also thank Dr. Oshrit Hoffer for her essential advice and motivation, which was instrumental in the success of this project.

We are grateful to the staff at the Mental Health Center in Be'er Sheva, especially Ms. Shahar Cohen and Prof. Doron Toder, for their assistance and personal care for both our team's success and wellbeing. We would also like to thank them for providing the necessary tools and resources, and for enabling a smooth research process.

We extend our appreciation to the Electrical Engineering department at Afeka College. Over the past four years, their commitment to academic excellence has provided an environment that fosters learning and innovation, leaving a lasting impression on us.

Lastly, we acknowledge the Projects Department for their consistent support. Their responsiveness and problem-solving mindset were vital in navigating the stages of this work, ensuring a productive outcome.

This project would not have been possible without the combined efforts of all those mentioned above, and we deeply thank them for their contributions.

Table of Contents

1 Abstract

1.1 תקציר (עברית)

פרויקט זה מציע תכן, פיתוח והערכה של מערכת ניטור שומרת-פרטיות (Privacy-preserving) המיועדת למחלקות פסיכיאטריות סגורות. המערכת, שפותחה בשיתוף עם המרכז לבריאות הנפש בבאר שבע, נועדה לפתור את המתח התפעולי בין הצורך בניטור קפדני לשמירה על בטיחות המטופלים, לבין החובה האתית לשמור על פרטיותם. בשונה מפתרונות קיימים המסתמכים על מצלמות אבטחה פולשניות או פיקוח אנושי, מערכת זו עושה שימוש בחיישנים תרמיים ברזולוציה נמוכה על מנת לזהות התנהגויות חריגות מבלי ללכוד מידע חזותי המאפשר זיהוי אישי.

הפתרון המוצע הוא מערכת מחשוב קצה (Edge-computing) המבוססת על בקר משובץ מרכזי המקושר למספר מערכים של חיישנים תרמיים:

- **שכבת החישה**: המערכת משתמשת בחיישנים תרמיים תוך בחינת חיישנים שונים ללכידת חתימות חום במקום תמונות אופטיות.
- **שכבת העיבוד:** המערכת נוקטת בגישה אלגוריתמית כפולה, הבוחנת הן עיבוד תמונה קלאסי והן מודלים של למידה עמוקה (Deep Learning) לסיווג תבניות תרמיות בזמן אמת. בהקשר לפרטיות ואבטחת מידע, אנו מציגים מספר אפשרויות לפתרון, כאשר המערכת בכל מקרה כוללת אבטחת מידע במבנה הבסיסי שלה (כיוון שלא ניתן לזהות פני אדם ברזולוציית החיישנים המוצגים בפרויקט).
- **תקשורת:** מידע באמצעות Ethernet ליחידת בקרה מרכזית, הכוללת ממשק משתמש גרפי (GUI) מינימליסטי המציג חיווי סטטוס (כגון "שריפה" או "פעילות חריגה") ללא הצגת וידאו חי.

בהתבסס על דרישות קליניות שהוגדרו על ידי הצוות הרפואי, המערכת תוכננה לזהות ארבעה תרחישים חריגים ספציפיים בתוך דקה מרגע התרחשותם:

1. **מעשי אלימות:** עימותים פיזיים בין מטופלים.
2. **התנהגות מינית:** אינטימיות פיזית אסורה בין מטופלים.
3. **הצתת אש:** זיהוי עליות טמפרטורה מהירות הנגרמות על ידי מצתים או סיגריות.
4. **כניסה לאזור מוגבל:** כניסה לא מבוקרת לאזורים אסורים.

הזיהוי של מעשי אלימות והתנהגות מינית מתבצע יחד בסיווג של מגע בין פציינטים, כיוון שמגבלות המערכת אינן מאפשרות עיבוד וידאו לזיהוי התנהגויות מדויק. בנוסף, כיוון שמגע בין פציינטים הוא אסור בין כותלי המרכז לבריאות הנפש, אנו מתייחסים לזיהוי זה כתנאי מספיק להתראה.

הפרויקט מקיף הן יישום הנדסי מעשי והן מחקר תיאורטי. מטרת מחקר מרכזית היא קביעת הרזולוציה התרמית המינימלית הנדרשת לזיהוי אמין של התרחישים, וזיהוי האיזון האופטימלי בין סיבוכיות אלגוריתמית (קלאסית מול אלגוריתמים לומדים) לבין אילוצי החומרה. תוכנית העבודה בנויה באופן מדורג:

- **שלב 1:** אינטגרציית חומרה ותכנון מערך חיישנים.
- **שלב 2:** איסוף מאגר נתונים (Dataset) תרמי ייעודי המדמה את ארבעת תרחישי המטרה.
- **שלב 3:** אימון ותיקוף אלגוריתמים (קלאסיים ולמידת מכונה) עם יעד דיוק זיהוי של לפחות 80% על סט הבדיקה.
- **שלב 4:** אינטגרציית מערכת ובדיקות שטח בזמן אמת.

הטמעה מוצלחת של אב-טיפוס זה תספק חלופה סקלבילית (ניתנת להרחבה) ומשתלמת כלכלית (כ-$250 ליחידה) למערכות רדאר או מערכות אופטיות יקרות. המערכת מציעה פתרון של "פרטיות כעיקרון תכנוני" (Privacy-by-design) המבטיח עמידה ברגולציה (חוק טיפול בחולי נפש התשנ"א-1991), תוך הפחתה משמעותית של זמני התגובה לאירועים מסכני חיים בסביבות טיפול רגישות.

1.2 Abstract

This project proposes the design, development, and evaluation of a privacy-preserving monitoring system intended for closed psychiatric wards. Developed in collaboration with the Center for Mental Health in Be'er Sheva, the system aims to resolve the operational conflict between the need for rigorous patient safety monitoring and the ethical mandate to preserve patient privacy. Unlike existing solutions that rely on intrusive video surveillance or human supervision, this system utilizes low-resolution thermal sensors to detect abnormal behaviors without capturing personally identifiable visual information (PII). The proposed solution is an edge-computing system based on a central embedded controller (Raspberry Pi 5) interfaced with multiple thermal sensor arrays:

Sensing Layer: The system utilizes infrared thermal sensors (specifically evaluating the AMG8833 8x8 array and MLX90640 32x24 array) to capture heat signatures rather than optical images.

Processing Layer: The system employs a dual-algorithmic approach, testing both classical image processing and Deep Learning models to classify thermal patterns in real-time.

Communication: Alerts are transmitted via Ethernet to a central control unit, featuring a minimalist Graphical User Interface (GUI) that provides status indicators (e.g., "Fire," "Abnormal Activity") without displaying live video feeds.

Based on clinical requirements defined by the medical staff, the system is designed to detect four specific anomaly scenarios within one minute of occurrence:

1. **Violent Acts:** Physical scuffles between patients.
2. **Sexual Behavior:** Unauthorized physical intimacy between patients.
3. **Fire Ignition:** Detection of rapid temperature spikes caused by lighters or cigarettes.
4. **Restricted Area Breach:** Unsupervised entry into prohibited zones.

The detection of violent acts and sexual behavior is done together with the detection of touch between patients, since the system's limitations do not allow for more complex video analysis models. Since touch between patients is forbidden within the medical center, we treat touch as a sufficient requirement to alert the staff.

The project encompasses both practical engineering implementation and theoretical research. A key research goal is to determine the minimal thermal resolution required for reliable scenario recognition and to identify the optimal balance between algorithmic complexity (Classical vs. AI) and hardware constraints. The work plan follows a phased structure:

- **Phase 1:** Hardware integration and design for sensor breakouts.
- **Phase 2:** Collection of a custom thermal dataset simulating the four target scenarios.
- **Phase 3:** Training and validation of algorithms (classical and ML) with a target detection accuracy of at least 80% on the test set.
- **Phase 4:** System integration and real-time field testing.

The successful deployment of this prototype will provide a scalable, cost-effective (approx. $250/unit) alternative to expensive radar or optical systems. It offers a "privacy-by-design" solution that ensures regulatory compliance (Israeli Mental Health Treatment Act) while significantly reducing response times to life-threatening incidents in sensitive care environments.

2 Introduction

2.1 Opening

In closed wards and psychiatric institutions today, ensuring the safety of both patients and medical staff is a critical and ongoing challenge. Research done by Dr. Ashleigh Anderson from the University Hospitals/Case Medical Center, Cleveland, Ohio, shows that psychiatrists and mental healthcare professionals are three times more likely to be exposed to violent patients than other medical staff.[1]

Existing solutions rely primarily on video surveillance and human monitoring—methods that often infringe on patient privacy and fail to adequately address real-time emergency situations. There is a growing demand for an innovative technological solution that provides a high level of safety without compromising privacy.

Following an interview with Prof. Doron Todder, head of the Mental Health Center in Be'er Sheva, the main risks that hospital surveillance teams aim to prevent include:

1. **Fire hazards** – The most significant risk is fire outbreaks in or near the center. Fires pose a serious threat for several reasons:
    - Patients may be under the influence of medications that impair their ability to evacuate. Some patients may not comprehend the danger or may be unaware of their surroundings. Evacuation procedures are complex and require staff to not only evacuate but also prevent panic attacks and escape attempts.
    - Personality-related psychiatric disorders (e.g., schizophrenia) are associated with a higher risk of arson.
    - Smoking is prohibited in the center unless medically approved and under supervision. Many patients struggle with this restriction and attempt to find secluded areas to smoke secretly. The use of lighters, therefore, presents a significant fire risk.
2. **Violent acts** – The mental health center includes several wards, some of which are closed, meaning patients are not permitted to leave. These wards typically house patients with more severe mental health conditions. Additionally, patients share rooms with two or three occupants, increasing the potential for altercations and abuse. These incidents occur particularly (though not exclusively) in the closed wards.
3. **Sexual relations** – Although patients interact daily, sexual activity is prohibited as part of the therapeutic protocol. Despite this, incidents of sexual relations between patients do occur and are a concern for staff.
4. **Patient access to restricted areas** – Patient movement and residence zones are strictly defined, and they are not allowed to be present outside of permitted areas.

[1] A. Anderson, S.G West, Violence Against Mental Health Professionals: When the Treater Becomes the Victim

In addition to these risks, legal requirements must also be considered. The Israeli **Mental Health Treatment Act (1991)**[2] mandates that treatment institutions uphold the dignity and privacy of their patients. This has been interpreted to mean that photographing or recording patients without their explicit consent may violate their rights. As a result, psychiatric hospitals and mental health institutions operate under strict internal regulations aimed at protecting patient privacy, including restrictions on photography without prior authorization.

To address all of the aforementioned risks and legal requirements, the proposed solution is a **smart thermal monitoring system** to be developed by the students as part of their final project. This system offers a unique approach: it will rely solely on infrared temperature sensors strategically placed in closed-room environments. The system will process thermal patterns in real time and utilize dedicated detection algorithms to spot any abnormal behavior as mentioned before and alert the medical team. This will be done by analyzing movement patterns, heat sources, and sudden temperature changes.

In the following images, one can see the immediate differences between an optical (regular) image (on the right) and a thermal image (on the left). Notice the human that walks in the back of the image: in the optical image, he is barely noticeable, while on the thermal image, he shines brightly (due to the temperature differences between the human body and the environment). Another thing to note is the fact that in the optical image, we can see more details from the environment: notice the tree and how you can separate between the leaves, while it is much harder to do so in the thermal image, since the image is blurred.

Figure 1: Thermal Image (left) vs. Optical Image (right)[3]

The low-resolution thermal sensing will enable the system to identify dangerous situations such as smoking, physical altercations, or entry into restricted areas—without recording patients' identities or using any form of visual recognition.

[2] The Mental Health Treatment Act, 1991, regulates the rights of individuals with mental health conditions, as well as the measures that may be taken concerning them.
[3] Kintronics, "Thermal versus optical IP cameras"

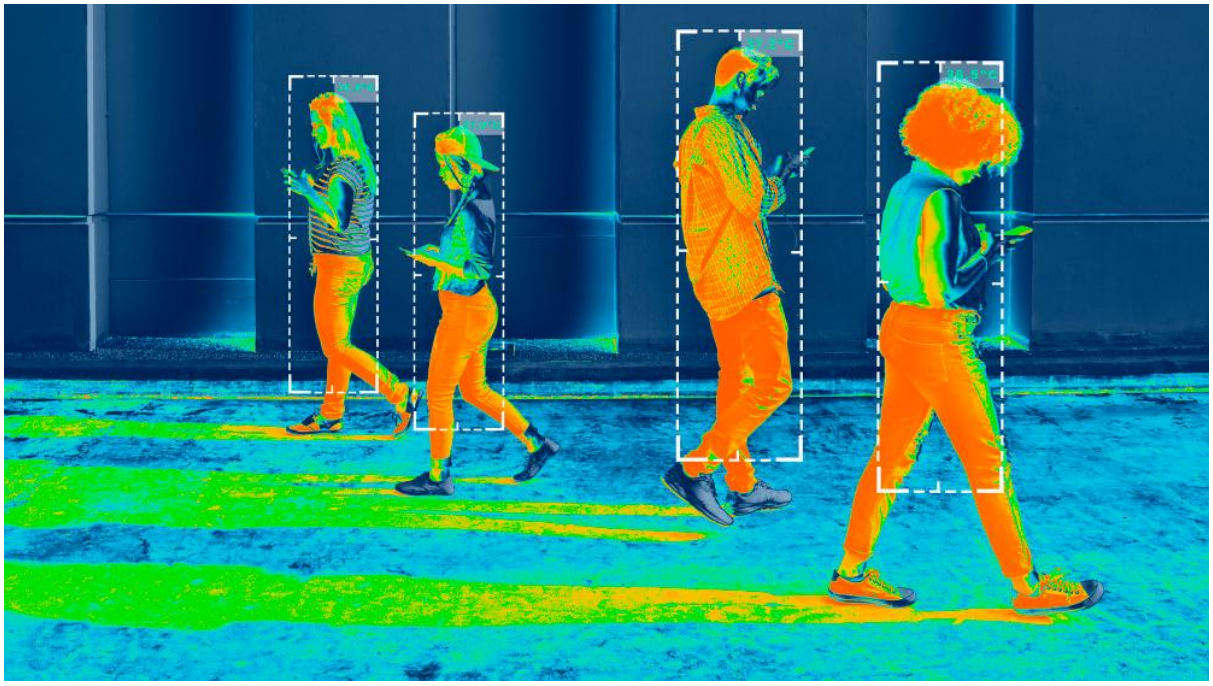


Figure 2: Human Detection in thermal images[4]

While this project is meant to be a proof of concept, at the end of the day, the goal is to implement this system throughout the mental health center. Therefore, we've designed the system to rely on relatively simple and low-cost hardware, to make sure that if the system were to be scaled and implemented in the entire health center, it would be at an affordable price. In addition to the costs, it is also important for us to alert in real time and minimize the response time to the abnormal events in these sensitive circumstances.

This project is a combination of a theoretical and research endeavor with a practical engineering solution. As part of our work, we tried different approaches to optimize the delicate balance between patients' privacy and our goal to detect dangerous behaviors. This process included iterative testing, alternative comparison, and performance evaluation. We explored classical approaches vs. learning algorithms, evaluated different models and compared different evaluation methods.

[4] Visionify.ai, "Infrared thermal imaging for people detection"

2.2 Goals, Objectives and Metrics

2.2.1 Project Objective

The project objectives are divided into the following sub-objectives:

1. Acquisition of knowledge and skills in image processing and computer vision algorithms, including convolutional neural networks (CNNs) and advanced classical image processing techniques.
2. To design and implement a system based on thermal sensors and a central control computer.
3. Real-time video analysis for detecting abnormal behaviors (violence, sexual contact, fire ignition, or presence in restricted areas), while fully preserving privacy—specifically avoiding facial recognition or any identifying visual information.

2.2.2 Project Goals

Several fundamental goals have been set for the project:

1. Hardware design and selection – Research and selection of appropriate components. Emphasizing reliability and fault-tolerant communication capabilities.
2. Data Collection – The dataset will contain low-resolution thermal images from multiple low-resolution thermal sensors.
3. 3D Mapping - Using spatial analysis with multiple thermal sensors concurrently to improve spatial detection.
4. Tool Evaluation – validation testing both the software and the entire system.
5. Strict privacy preservation – the system will not store or record video but will operate solely through real-time processing and will not transmit any data.

2.2.3 Project Metrics

The following metrics will serve as indicators of the project's success:

1. **Latency:** The system shall detect and alert on abnormal events with a maximum system latency of 60 seconds. Latency is defined as the time elapsed from the event occurrence to the visualization of the text-based alert on the central operator screen. This includes sensor data acquisition, local processing time on the Raspberry Pi, and network transmission of the alert payload.
2. **Accuracy:** The system shall achieve a Minimum Mean Accuracy of 80% across all event classes (Violence, Fire, Restricted Area, Sexual Contact) on the validation dataset. To ensure safety, the classification threshold shall be tuned to minimize False Negatives for critical life-safety events (e.g., Fire), ensuring high Recall even at the cost of slightly lower Precision.
3. **System:** The system shall demonstrate a Distributed Edge-Computing Architecture where all video processing occurs locally on the Raspberry Pi to ensure privacy and bandwidth efficiency.
    - **Scalability:** The communication protocol must be structured to support unique addressing for up to 200 future units, though the PoC will validate this using 1 unit with 3 concurrent sensors.
    - **Health Telemetry:** The system must perform a 'Watchdog' function, verifying sensor connectivity and signal integrity every 60 seconds. In the event of sensor failure or video signal loss (e.g., occlusion/noise), a specific 'System Fault' alert must be sent to the central control.
4. The Central Control Unit shall feature a 'Zero-Video' Graphical User Interface designed specifically for non-technical medical staff.
    - **Alert Presentation:** Active alerts must be displayed as prominent, high-contrast text notifications indicating only the Event Type and Room Location (e.g., 'ALERT: Fire detected in Room 11') without displaying thermal imagery.
    - **Event Logging:** The GUI must automatically record a persistent history log of events, storing the timestamp, location, and event classification for retrospective review.

2.2.4 Responsibility Allocation & Module Ownership

To ensure the successful completion of the project metrics defined above, the development and integration tasks are divided among the team members. Ownership is assigned based on the specific algorithmic and system sub-components required to meet the accuracy and latency constraints.

Metric Reference	Sub-System / Module	Primary Responsibility	Description of Task
Metric 1 (Latency)	System Software & Algorithms	Guy Chen	Optimizing software architecture to ensure fast reaction speeds across each part of the system.
Metric 2 (Accuracy)	Fire Detection Algorithm	Roy Lieberman	Developing and tuning the thermal thresholding and dynamic analysis to detect rapid temperature spikes (fire).
Metric 2 (Accuracy)	Human Detection	Yaniv Blau	Implementing the computer vision pipeline to identify human presence and within the frame (Privacy-Preserving).
Metric 2 (Accuracy)	Contact Detection	Guy Chen	Developing the logic to detect when two "human" objects intersect (sexual contact/violence).
Metric 3 (System)	Edge-Computing & Hardware Integration	Yaniv Blau	Integration of the 3 thermal sensors with the Raspberry Pi, ensuring efficient multiprocessing and "Watchdog" health monitoring.
Metric 3 (System)	Communication Protocol	Roy Lieberman	Establishing the secure, text-only messaging between the RPi and the Central Unit to ensure <60s latency.
Metric 4 (GUI)	Central Control Unit (GUI)	Roy Lieberman	Designing the "Zero-Video" interface for medical staff, including the alert pop-up logic and historical event logging.

2.3 Requirements

This section outlines the requirements set by the Mental Health Center for the project and its operation.

2.3.1 Functional Requirements

The following are essential requirements for the system's proper operation and for fulfilling its mission profile.

1. 24/7 operation with continuous power supply (with an emphasis on nighttime hours)

The Mental Health Center operates continuously, with certain mental wards housing patients around the clock. The system must remain fully operational at all times, particularly during nighttime hours - ensuring uninterrupted monitoring and safeguarding patient well-being.

2. Detection and alert within one minute from the moment of the event

To enable timely intervention and minimize potential harm, the system must detect critical events and issue an alert within no more than one minute from the moment the event occurs. This prompt response time is essential for ensuring the safety of patients and enabling staff to act quickly in emergency situations.

3. Detection accuracy and prevention of false alerts

The system must accurately detect real incidents while minimizing false positives. Maintaining high detection precision is essential to ensure that staff can trust the alerts and respond appropriately. Excessive false alerts may lead to desensitization or unnecessary interventions, reducing overall system effectiveness and potentially compromising patient safety.

2.3.2 Non-Functional Requirements

These are general system requirements that are important to the Mental Health Center but do not have a direct impact on the system's mission profile.

1. Risk Reduction of Abnormal and Hazardous Incidents

The system must actively contribute to reducing the risk of abnormal or dangerous incidents, including physical violence, sexual activity between patients, fire ignition, and unauthorized entry into restricted areas. Early detection and real-time alerts are critical for preventing harm and maintaining a safe and controlled environment within the facility.

2. Privacy Protection and Data Security

While ensuring patient safety, the system must strictly protect patient privacy, including their personal identity and medical information. All data collection, processing, and storage must comply with relevant privacy regulations and ethical standards, ensuring that security measures do not compromise patient dignity or confidentiality in accordance with patient-privacy laws (see footnotes)[5] [6] [7] [8].

3. Seamless Integration and Operational Compatibility

The system must be designed for smooth implementation within the Mental Health Center, ensuring compatibility with existing workflows. It should support effective collaboration with clinical staff and center management, including intuitive interfaces, minimal training requirements, and alignment with operational protocols to avoid disruptions in daily care routines.

4. Enhanced Staff Efficiency and Sense of Security

The system should support the care team's work by streamlining routine tasks, reducing response times, and minimizing manual monitoring efforts. By providing reliable alerts and situational awareness, the system must also contribute to a heightened sense of security among staff, enabling them to perform their duties with greater confidence and focus.

[5] The Treatment of Mentally Ill Act, 1991
[6] The Protection of Privacy Act, 1981
[7] The Patient Rights Act, 1996
[8] Standards for Filming in Trauma and Intensive Care Rooms in the Emergency Department ,09/2024

2.4 Project Status

As the project is currently in the prototype and research phase, full compliance with the aforementioned requirements has not yet been achieved. The following section details the current gap between the requirements and the existing prototype status.

2.4.1 Functional Gaps

1. Alert System Integration (Communication Subsystem)

Requirement: Detection and alert transmission within one minute.

Current Status: The core detection algorithms have been implemented and validated locally. However, the system is currently a standalone prototype. The network communication module required to transmit alerts to a remote-control unit or staff station has not yet been integrated. Consequently, while the system can detect an event effectively, it cannot yet alert remote personnel.

2. Algorithmic Precision & False Positives

Requirement: High detection accuracy with minimal false alerts.

Current Status: The current prototype utilizes classical computer vision techniques applied to thermal imagery. While these algorithms have successfully demonstrated proof-of-concept capabilities in detecting major motion and interaction (see Chapter 7), they have not yet reached the statistical reliability required for a marketable product. The current False Positive Rate remains higher than the operational requirement, necessitating further optimization and the transition to Deep Learning methodologies, which were not yet implemented during this phase of the project.

2.4.2 Non-Functional Gaps

1. Clinical Environment Integration

Requirement: Seamless integration and staff interaction.

Current Status: The system has been tested in controlled environments but has not yet been deployed physically within the Mental Health Center's wards. As such, validation of the "Staff Efficiency" and "Sense of Security" requirements through user feedback has not been realized.

3 Literature and Market Review

3.1 Literature Review

This subsection reviews books and academic/professional articles that we believe will support our learning and development process throughout the project.

3.1.1 Books

- **Szeliski, R.**, *Computer Vision: Algorithms and Applications*, Springer, 2022[9]

  This book provides a solid theoretical foundation and introduction to the field of computer vision. It includes mathematical derivations and explains how developed formulas apply to images of all types. Using this book, we aim to acquire the necessary knowledge to develop an advanced project in image processing and computer vision.

- **Pajankar, A.**, *Raspberry Pi Computer Vision Programming*, Packt, 2019[10]

  This book provides a practical introduction to using the OpenCV library with the assumption that programming is being done on a Raspberry Pi 4. It includes practical examples and tutorials in areas such as real-time image processing, object detection, camera interfaces, and more.

- **Zhang, A. et. al,** *Dive into Deep Learning,* Cambridge University Press, 2023[11]

This book serves as a comprehensive educational resource for deep learning, integrating theoretical frameworks with practical application. It functions as a complete curriculum, supplemented by extensive coding exercises and a robust online repository of executable examples.

3.1.2 Articles and Research Papers

- **Dubagunta, N.** (2022), *Using Machine Learning for Real-Time Detection of Violence in Video Footage*[12]

  This article discusses the detection of violent behavior in images and video using various machine learning methods, and the effective selection of models for this task. It helps us understand the challenges in detecting extreme behavior from visual input and informs us of the adaptation of our processing methods accordingly.

- **Vijeikis, R.** (2022), *Efficient Violence Detection in Surveillance*, *Sensors*[13]

  This paper presents a deep learning model for real-time violence detection in video. The proposed model is computationally lightweight, making it suitable for execution on

[9] R. Szeliski, *Computer Vision: Algorithms and Applications*
[10] Pajankar, A., *Raspberry Pi Computer Vision Programming*
[11] Zhang, A. et. al, *Dive Into Deep Learning*
[12] Dubagunta, N. (2022), *Using Machine Learning for Real-Time Detection of Violence in Video Footage*
[13] Vijeikis, R. (2022), *Efficient Violence Detection in Surveillance*, *Sensors*

microprocessors and edge devices. We believe a similar model can be developed to meet the goals of our project.

- **Tateno, S.** (2020), Privacy-preserved fall detection method with three-dimensional convolutional neural network using low-resolution infrared array sensor, Sensors[14]

This paper presents a similar problem to ours, and a novel way of dealing with fall detection using the MLX90640. We took inspiration from this paper, using its pre-processing methods as a starting point to allow better use of our data for detection.

[14] Tateno, S. (2020), Privacy-preserved fall detection method with three-dimensional convolutional neural network using low-resolution infrared array sensor, Sensors

3.2 Market Review

3.2.1 Systemic Alternatives

This section analyzes existing market solutions that address the problem of continuous patient monitoring. While many of these systems focus on elderly care and fall prevention, they represent the current state-of-the-art in privacy-preserving monitoring and serve as the systemic benchmark for the proposed project.

1. PreSAGE (CoNEX Healthcare)

Overview: PreSAGE is a continuous patient monitoring platform developed in Singapore, primarily designed for fall prevention in hospitals and nursing homes.

Technology: It utilizes a ceiling-mounted thermal sensor combined with Artificial Intelligence (AI) to predict bed exits before they happen.

- **Relevance:** Like the proposed project, PreSAGE relies on thermal imaging to maintain patient privacy (non-intrusive).
- **Gap Analysis:** PreSAGE is highly specialized for **predictive fall prevention** (e.g., detecting when a patient sits up in bed). It is not optimized for detecting complex social interactions between multiple patients, such as physical violence or sexual activity, which are critical safety requirements for a psychiatric ward.

2. ThingX

Overview: ThingX offers IoT-based health monitoring solutions, often focusing on wearable integration for vital signs tracking.

- **Technology:** Unlike the passive sensors discussed in this report, ThingX solutions typically rely on wearable wristbands equipped with digital temperature and pulse rate sensors to transmit data to a central dashboard.
- **Relevance:** It addresses the "health monitoring" aspect of the mission profile but utilizes a contact-based modality.
- **Gap Analysis:** In a psychiatric environment, wearable devices are often **clinically non-viable**. Patients in acute distress may remove the device, damage it, or use it as a tool for self-harm. A passive, contactless system is superior for high-risk wards where patient cooperation cannot be guaranteed.

3. Intercall (Safeguard Sensor)

Overview: Intercall is a major provider of nurse call systems. Their "Safeguard Sensor" is an add-on thermal solution.

- **Technology:** A ceiling-mounted thermal sensor that integrates directly into the standard nurse call system. It monitors bed occupancy and movement.
- **Relevance:** It demonstrates a successful integration of thermal sensors into existing hospital infrastructure (a key non-functional requirement).
- **Gap Analysis:** The system is **binary in nature**—it detects presence or absence (occupancy). It lacks the advanced computer vision capabilities required to classify specific behavioral hazards (e.g., aggression) or track multiple individuals simultaneously in a shared common room.

4. Butlr / Butlr Care

Overview: Butlr allows for people-sensing using low-resolution thermal arrays, marketed heavily for both smart buildings and "Butlr Care" for senior living.

- **Technology:** They use sensors very similar to the MLX90640 (used in this project) to capture heat signatures. Their API interprets these signatures to detect occupancy, traffic flow, and falls.
- **Relevance:** Butlr proves that low-resolution thermal sensors are commercially viable for detecting human posture and motion.
- **Gap Analysis:** Butlr's algorithms are optimized for *occupancy analytics* (counting people) and *fall detection*. They do not currently market an active surveillance solution capable of identifying aggressive behavior in real-time, which requires higher-level temporal analysis.

5. ITRI (Industrial Technology Research Institute)

Overview: ITRI has developed a "High-Privacy AI Digital Caregiver."

- **Technology:** This system combines thermal imaging with millimeter-wave (mmWave) radar to detect vital signs (heart rate, breathing) and activities without cameras.
- **Relevance:** It represents a high-end "sensor fusion" approach.
- **Gap Analysis:** While technologically superior in detecting vital signs, the complexity and cost of sensor fusion (Thermal + Radar) make it expensive to deploy at scale. Furthermore, it is designed for physiological monitoring rather than behavioral safety monitoring (violence prevention).

6. Vayyar Care (Radar Imaging)

Overview: Vayyar is an Israeli company that provides 4D imaging radar solutions, primarily targeting the elderly care market for fall detection and intruder alerts.

- **Technology:** Uses Radio Frequency (RF) waves to create a 3D point cloud of the environment. It can "see" through darkness, steam, and privacy curtains without using any optical lenses.
- **Relevance:** It represents the "Gold Standard" for privacy compliance, as the sensor physically cannot capture a recognizable image of a face or body features.
- **Gap Analysis:** While excellent for posture detection (standing vs. falling), radar point-clouds generally lack the spatial resolution provided by thermal imaging. This makes it difficult to reliably distinguish between complex social interactions without a high rate of false positives.

7. Oxehealth (Oxevision)

Overview: Oxehealth provides a vision-based patient monitoring platform that is currently deployed in mental health hospitals and police custody suites, mainly in the UK and Europe.

- **Technology:** It utilizes standard optical sensors combined with Infrared (IR) illumination. It uses software algorithms to track activity and even measure pulse and breathing rates remotely (using slight pixel color changes).
- **Relevance:** It is a direct competitor in the specific market of psychiatric ward monitoring.
- **Gap Analysis:** Unlike the proposed project, Oxehealth relies on actual cameras. Even though they use software to mask privacy, the presence of an optical lens creates a "perceived" privacy intrusion that patients may distrust. Additionally, it requires active IR illumination at night, whereas the proposed thermal system is completely passive and undetectable.

Enterprise / Product	Primary Technology	Target Market	Privacy Level	Key Gap Analysis
PreSAGE (CoNEX)	Thermal + AI	Fall Prevention	High	Optimized for predicting bed exits; lacks algorithms for detecting complex social interactions or violence.
ThingX	IoT Wearables	Vitals Monitoring	High	Contact-based solution is clinically non-viable in acute psychiatric wards due to risks of removal or self-harm.
Intercall (Safeguard)	Thermal	Nurse Call Integration	High	Limited to binary detection (presence/absence); lacks computer vision capabilities to classify specific behaviors.
Butlr	Low-Res Thermal	Smart Building / Elderly	High	Focused on spatial analytics (counting) and simple falls; not designed for real-time aggression or safety monitoring.
ITRI	Thermal + Radar	Physiological Vitals	High	High cost and complexity due to sensor fusion; focuses on health metrics (heart rate) rather than behavioral safety.
Vayyar Care	4D RF Radar	Elderly Care / Falls	Very High	Radar point-clouds lack the spatial resolution required to reliably distinguish between complex interactions.
Oxehealth (Oxevision)	Optical + Active IR	Psych Wards / Custody	Medium	Relies on optical cameras (lens), creating a "perceived" privacy intrusion; requires active IR illumination at night.

Table 1 – System Analysis Summary

4 Methods

4.1 Component Selection

4.1.1 Controllers

Reviewing the initial options for controller selection before getting started, the controllers that were considered were as follows:

- **Raspberry Pi 4**
  Released on June 19th, 2019, this device is a compact, low-cost single board computer (SCB), roughly the size of a credit card. Serving as a central unit that manages inputs and outputs from various sensors, processes the data received, and performs tasks such as logging data, detection, and communication between devices. This means that SCBs of this kind (not only the Raspberry Pi 4) can act as a dedicated device for reading and cross-referencing thermal data, detecting abnormal behaviors, and transmitting alerts. Raspberry Pi models come pre-installed with Python 3- which is ideal for our project, with support for other programming languages.

The Raspberry Pi 4 is equipped with a Broadcom BCM2711 processor, a quad-core processor with a clock speed of 1.8GHz. It has variants of 1,2,4 and 8 GB of RAM, and supports up to 6 I2C buses. It has two ports for both USB 2.0 and 3.0, supports both Wi-Fi and Ethernet networking, and offers storage via microSD. This unit requires a 5V/3A power supply (via USB-C), and while an external heatsink is suggested for heat management, it operates with passive cooling.[15] It is priced at 200-400 NIS for the unit by itself (excluding various attachments and accessories) at the official Israeli distributer, Piitel (depending on RAM). [16]

- **Raspberry Pi 5**
  The next generation of the Raspberry Pi SCB family, it is an improved version of the above model. While it does not differ from the Pi 4 in functionality, it boasts better specifications for improved performance, and better real-time processing capabilities.

The main difference between this model and the previous one is the processor, which is a quad-core Broadcom BCM2712[17], with a clock speed of 2.4 GHz, and RAM variants of 2, 4, 8 and 16 GB. It has the same I2C support, similar networking protocols, and while it offers more storage options, it is irrelevant for our purposes, as privacy constraints prevent us from storing any data long term.

Pricing varies from 250 to 600 NIS at Piitel, with the same regard for additional accessories that might be required.[18]

While the Raspberry Pi 5 holds a significantly more powerful processor that will indeed serve our goal of performing real-time object detection and behavioral, it is both more expensive and might require additional hardware for heat management (a separate active cooler).

[15] Raspberry Pi Foundation, *Raspberry Pi 4 Tech Specs*
[16] Piitel, *Rasbperry Pi 4 MODEL B*
[17] Raspberry Pi Foundation, *Raspberry Pi 5 Tech Specs*
[18] Piitel, *The Next Generation - Raspberry Pi 5*

- **ESP32**
  The Espressif ESP32 is a microcontroller unit (MCU), which excels in low power, lightweight applications. While it is not a full computer like the SCBs, it is widely used for operating sensors and real time data transfers.

The ESP32 has a Dual-core Xtensa LX6 processor with a clock speed of 240MHz, around 520 KB of RAM, Wi-Fi and Bluetooth connectivity (but no Ethernet), and supports up to 2 I2C buses.[19]

While initially the ESP32 seemed like a viable option due to its small size, strong community support, low power consumption and cheap price (as low as 40 NIS on websites such as AliExpress), it is ultimately a bad fit for this project, since its insufficient processing power cannot allow us to run heavier libraries like OpenCV (even with its C++ version).

- **TI TCA9548A**
  Texas Instruments' TCA9548A is an 8-channel, bidirectional I2C multiplexer[20]. It is designed to let one I2C master bus share communications between multiple downstream buses, when this sometimes proves to not be doable.

For our project, while the Raspberry Pi models support several I2C ports, there may be issues stemming from external components (such as thermal sensors) having a default I2C address set, at times with no option to change said address. While there are means to redirect said bus addresses via software, it might prove to be simpler to use this switcher to rapidly switch between sensors for data collection. It will not truly by simultaneously transmitting data from all sensors but will allow fast enough switching to enable the SCB to process the data within our time constraints.

After considering our needs for this project, we have selected the Raspberry Pi 5 as our controller and chosen to work with the TI TCA9548A multiplexer. The ESP32 was disqualified due to its limitations in processing power, since our goal is to perform all data processing locally to preserve patient privacy. The Raspberry Pi 4 seems like a decent contender that might be able to handle the computations necessary for the local data processing, but hardware benchmarks prove that the Raspberry Pi 5, while still having a quad-core CPU, is both more capable of providing the necessary power and is faster at doing so, with a 33% increase in clock speed. The Pi 5 does run a little hotter under heavy load, but as the official active cooler is readily available and not very expensive, and the fact that the price difference between the Pi 4 and 5 is not very large, the Pi 5 looks to be the best candidate for the job. For the I2C multiplexer, there is a definite need for either hardware or a software-based solution for connecting several sensors of the same type via I2C, since the internal bus address of each sensor is not something that can be changed. After researching possible workarounds, it seems that a hardware solution is the best, if not only, way to circumvent this issue. So far, the TCA9548A multiplexer has proved to be an excellent choice, with its extremely high switching speed that far surpasses our working rates, the data collection essentially becomes simultaneous.

[19] Espressif, *ESP32 Series Datasheet Version 4.9*
[20] Texas Instuments, *TCA9548A*

4.1.2 Thermal Sensors

The following are the initial thermal sensors candidates that were considered prior to beginning the work.

- **ADAFRUIT AMG8833**
  The Adafruit AMG8833 is a compact, cost-effective thermal imaging sensor built around Panasonic's Grid-EYE technology. It features an 8x8 array of infrared (IR) sensors, providing a total of 64 individual temperature readings per frame. The sensor operates over an I2C interface, making it compatible with a wide range of microcontrollers and single-board computers, including the Raspberry Pi. [21]

  - **Resolution:** 8x8 (64 pixels)
  - **Temperature Range:** 0°C to 80°C
  - **Accuracy:** ±2.5°C
  - **Detection Distance:** Up to 7 meters for human presence
  - **Frame Rate:** Up to 10Hz
  - **Operating Voltage:** 3.3V or 5V
  - **Special Features:** Configurable interrupt pin for threshold-based alerts, STEMMA QT connectors for solderless integration
  - **Community & Support:** Extensive software libraries and guides are available for Arduino, CircuitPython, and Python, simplifying integration and development.
  - **Use Case:** Suitable for basic presence detection, low-resolution thermal mapping, and DIY thermal camera projects.

- **101020557 (Grove - AMG8833 8x8 Infrared Temperature Sensor Array)**
  This sensor module utilizes the same Panasonic AMG8833 sensor as the Adafruit version but is presented in the Grove ecosystem for rapid prototyping.[22]

  - **Resolution:** 8x8 (64 pixels)
  - **Temperature Range:** 0°C to 80°C
  - **Accuracy:** ±2.5°C
  - **Detection Distance:** Up to 7 meters
  - **Interface:** I2C (default address 0x68, optional 0x69)
  - **Operating Voltage:** 3.3V / 5V
  - **Features:** Plug-and-play with Grove connectors, compatible with both Arduino and Raspberry Pi platforms
  - **Use Case:** Ideal for educational, hobbyist, and rapid prototyping applications where ease of use and modularity are prioritized.

[21] Adafruit, AMG8833 IR Thermal Camera Breakout – product page
[22] Seeed Studio, Grove - AMG8833 8x8 Infrared Thermal Temperature Sensor Array - product page

- **MLX90641**
  The Melexis MLX90641 is a higher-resolution thermal sensor array, providing a 16x12 grid (192 pixels) for more detailed thermal imaging.[23]
    - **Resolution:** 16x12 (192 pixels)
    - **Temperature Range:** -40°C to +300°C (object), -40°C to +125°C (operating)
    - **Accuracy:** ±1°C (typical)
    - **Field of View:** Available in 55°x35° (standard) and 110°x75° (wide angle)
    - **Refresh Rate:** Programmable from 0.5Hz to 64Hz
    - **Interface:** I2C
    - **Operating Voltage:** 3.3V
    - **Features:** Factory calibrated, robust operation in harsh environments, compact TO39 package
    - **Use Case:** Well-suited for industrial, automotive, and smart building applications requiring moderate resolution and high thermal stability.
- **PIM366 / PIM365 (Thermal Camera Breakout, MLX90640)**
  These modules are based on the Melexis MLX90640 sensor, offering a 32x24 pixel grid (768 pixels) for significantly improved thermal detail.[24]
    - **Resolution:** 32x24 (768 pixels)
    - **Temperature Range:** -40°C to +300°C
    - **Accuracy:** Approx. ±1°C
    - **Field of View:** 55°x35° (standard) or 110°x75° (wide angle)
    - **Refresh Rate:** Programable up to 64Hz
    - **Interface:** I2C (address 0x33)
    - **Operating Voltage:** 3.3V or 5V
    - **Special Notes:** Due to the high data rate and processing requirements, these sensors are best paired with more capable microcontrollers or single-board computers, such as the Raspberry Pi.
    - **Community & Support:** Python libraries and documentation are available for integration with Raspberry Pi and Arduino platforms.
- **MLX90640BAA/B (Melexis MLX90640 Series)**
  The MLX90640BAA/B sensors are part of the MLX90640 family, sharing the same 32x24 resolution and core specifications as the above modules.[25]
    - **Resolution:** 32x24 (768 pixels)
    - **Temperature Range:** -40°C to +300°C
    - **Accuracy:** ±1°C
    - **Field of View:** 55°x35° (BAA) or 110°x75° (BAB)
    - **Refresh Rate:** Programmable from 0.5Hz to 64Hz
    - **Interface:** I2C
    - **Operating Voltage:** 3.3V
    - **Features:** Factory calibrated, compact TO39 package, no need for frequent recalibration

[23] Melexis, Far infrared thermal sensor array (16x12 RES) - product page
[24] Pimoroni, MLX90640 Thermal Camera Breakout - Product Page
[25] Melexis, Far infrared thermal sensor array (32x24 RES) – Product page

- **Use Case:** Suitable for applications demanding higher spatial resolution in thermal imaging, such as fire prevention, surveillance, HVAC monitoring, and presence detection.

As the Grove 101020557 is essentially an AMG8833 module adapted to the Grove ecosystem-which is not used in this project-we selected the Adafruit AMG8833 as our lowest-resolution option. At this tier, once the 8×8 sensor was adopted, an intermediate 16×12 resolution did not offer sufficient added value: it increases complexity without providing the level of spatial detail achieved by higher-resolution sensors, while still maintaining similar privacy constraints. Because of this, the 16×12 module was omitted and the design moved directly to the 32×24 resolution class. Within this tier, the candidate modules were functionally similar, so the MLX90640BAA was chosen as it provides a favorable trade-off between field of view, spatial resolution, and practical integration aspects such as breakout-board availability and calibration effort.

This concludes the section regarding previously considered sensors. We have recently added one more sensor to this list that is still being considered:

- **Long-wave IR Thermal USB Camera (Waveshare 26984, 80×62 pixels, 90° FOV)**

The Waveshare Thermal-90 USB Camera (SKU 26984) is a long-wave infrared (LWIR) imaging module designed for continuous thermal video streaming over USB-C, featuring an 80×62-pixel microbolometer/thermopile array and a wide 90° horizontal field of view.[26]

- **Resolution:** 80×62 (4,960 pixels)
- **Temperature Range:** −20 °C to +400 °C
- **Accuracy:** ±2°C (Ambient 10-70 °C)
- **Field of View:** 90°(H) × 68°(V) (wide-angle variant)
- **Refresh Rate:** Up to 25 FPS thermal video stream
- **Interface:** USB Type-C
- **Operating Voltage:** 5V
- **Features:** Shutterless design for continuous operation, compact USB form factor, 8–14 µm. LWIR band coverage, suitable for integration with embedded hosts.
- **Use Case:** Well suited for privacy-preserving monitoring tasks such as abnormal presence/occupancy analysis, activity pattern monitoring and fire detection.

Currently, the dataset is still being built and the viability of the chosen resolution tiers is being evaluated-that is, how suitable the selected sensors are for the task.
At this moment, we have collected a data set built with the MLX90640BAA, but not one built with the lower resolution AMG8833. While we are currently able to achieve decent accuracy with the 32x24 resolution sensor, we can see that it is quite difficult to determine any recognizable features at this resolution. Thus, we have begun to consider an even higher resolution sensor, as it seems that we might be able to maintain the privacy restrictions at ever higher resolutions. This will allow for greater accuracy and better results of the object detection and analysis process and easier implementation.

[26] Waveshare, Long-wave IR Thermal Imaging Camera Module - Product Page

In addition to this, we are researching whether it would be acceptable to use a resolution that is indeed high enough to provide recognizable features, but immediately induce blurring on any physical attributes, in accordance with the restrictions.
The newest addition to the 'under consideration' list, the Waveshare 26984, offers a resolution that is approximately five times higher than our current sensors. At this point, it is undetermined if this resolution will be above the threshold for identifiable features. The Waveshare sensor, aside from its much-improved resolution and field of vision, connects to the Pi 5 via USB-C connectivity. This simplifies the physical set-up by eliminating the need for an I2C multiplexer as there is no issue with addresses using several USB sensors at once, and the integration is immediate. An even bigger potential advantage is the robustness of USB cables as the length increases. Currently, we are operating with short, roughly 30 cm I2C wiring, which is already prone to reliability issues, and this instability is expected to worsen with increased cable length. A solution to this issue is of high importance. It is, of course, worth mentioning that the price point of the Waveshare sensor is nearly four times that of the MLX, so it is also a consideration. To conclude, both AMG8833 and the Waveshare 26984 are still under consideration. A data set will be built for the AMG, and further research into the restrictions on identifiable features and data storage will be made before reaching a decision regarding the Waveshare sensor.

4.1.3 Communication Methods (Controller ↔ Sensors)

- **I2C (Inter-Integrated Circuit) Communications**

As a reminder, I2C is a popular two-wire (plus common ground) address-based serial protocol that allows multiple low-speed peripherals to share the same bus. This makes the protocol well suited for integrating several identical modules (thermal sensors, in our case) in a single room while keeping the wiring simple and inexpensive.

In our system, I2C provides sufficient bandwidth for periodic frame acquisition and straightforward operation of several sensors. Because the currently selected sensor modules (MLX90640) have their I2C address hard-coded, it is not possible to assign unique addresses per device. To overcome this, we use an off-the-shelf I2C multiplexer (TCA9548A), which allows the controller to select which sensor is currently connected to the main bus. Given the current frame acquisition rate of about 8 Hz, the multiplexer's switching time is negligible, so the system effectively achieves real-time acquisition.

A separate issue that may arise in the next stage of the project is signal integrity of the I2C communication. As stated earlier, our goal requires several sensors to be connected to the same controller in order to gather sufficient spatial information. This is crucial for accurately determining whether subjects are truly touching/standing in overly-close proximity or simply standing one behind another. To obtain the required spatial data, we must position the sensors at opposite corners of the room, which in turn demands relatively large distance between the sensors and the controller.

Standard I2C wiring has already proven noisy at the lengths currently in use (roughly 30 cm long), and we expect the integrity to degrade with longer runs. A possible solution to this is to use I2C repeaters (also referred to as buffers or extenders). These are ICs that are placed in series with the bus, that increase the drive capability and reduce the apparent bus capacitance over longer cables, improving signal integrity.

Since operation over such distances has not yet been tested, it is not clear whether repeaters will be strictly required, but either way we have looked into possible repeaters. Some options are:

- **NXP P82B715**[27]
- **NXP PCA9600DPZ**[28]

These devices are designed to drive I2C signals over significantly longer cabling (on the order of tens of meters when correctly deployed with twisted-pair wiring and terminations).[29]

- **USB Type-C**

As mentioned in the component selection section (4.1.2), we have begun considering the usage of Waveshare 26984 as the thermal sensor in this system. Since this sensor utilizes USB-C connectivity instead of I2C, we will give a brief overview of the interface.

[27] NXP, P82B715 I2C-bus extender, datasheet.
[28] NXP, PCA9600 Dual bidirectional bus buffer
[29] NXP, AN10710 Features and applications of the P82B715 I2C-bus extender

USB Type-C is a modern 24-pin, reversible USB connector that can carry both data and power over a single cable and is now widely used in embedded and consumer devices. In this project, USB-C is relevant because the aforementioned Waveshare sensor presents itself as a USB peripheral and uses the USB-C link to stream thermal frames while simultaneously drawing its supply from the Raspberry Pi 5.[30] [31]

Compared with the low-level I2C bus used by the MLX90640 modules, a USB-connected sensor behaves as a higher-level device: the controller communicates with it through the standard USB host stack rather than by directly accessing sensor registers on a shared bus. This has several practical implications. First, USB cabling is mechanically robust and specified for longer distances than typical I^2C wiring, which simplifies sensor placement in larger rooms and reduces sensitivity to noise and bus capacitance on the link between the controller and the sensor. Second, multiple identical USB-C sensors can coexist on the same system as independent devices (via the Pi's USB ports or a powered hub), eliminating I2C-address conflicts and removing the need for an I2C multiplexer or bus repeaters. On the other hand, this approach introduces a dependency on the vendor's USB protocol or software library and offers less fine-grained, deterministic timing control than the current I2C-based design.[32] This means that it would, theoretically, be more difficult (or impossible) to accurately define the exact timing of the frame acquisition. However, given that the system requires only a modest frame rate (on the order of a few frames per second per sensor), we estimate that the reduced timing determinism of a USB-based interface will not pose a significant problem for this application.

[30] All About Circuits, Guide to USB-C Pinout and Features.
[31] USB.org, USB Type-C.
[32] Rocktech, I2C vs USB: Choosing the Right Interface for PCAP Touch Screens

4.1.4 Communication Methods (Controller ↔ Host Computer)

In the SOW, several options were reviewed for communication between each room controller and the hospital control room, including wired Ethernet (with optional Power over Ethernet, PoE), Wi-Fi over the existing wireless infrastructure, and cellular links such as LTE/5G. In the current design phase, we selected Ethernet as the primary communication standard for this system. The decision was driven by the following factors:

- Reliability & Latency: In a critical application such as safety and hazard-monitoring, the connection must be impervious to the interference, signal dropouts and dead zones common with wireless signals in dense hospital environments. Ethernet provides the consistent, low-latency link required for immediate alert delivery.
- Infrastructure Alignment: Hospitals rely on robust wired networks for operations. Choosing Ethernet ensures immediate compatibility with existing IT security protocols and infrastructure without requiring new wireless spectrum allocation.
- System Stability: Native support for Ethernet on the Raspberry Pi reduces architectural complexity, allowing for the development effort to focus on the core sensing and local processing tasks rather than troubleshooting wireless drivers' instability.

While Ethernet was chosen as the baseline standard, the system's modular architecture allows for the future integration of Wi-Fi or cellular networks if the specific site conditions demand it.

4.2 System Architecture

Our proposed architecture utilizes a Raspberry Pi as the central embedded controller, interfaced with thermal imaging sensors serving as data acquisition nodes. Due to Raspberry Pi's single available I2C port and the address conflicts inherent to MLX sensors (which share identical default I2C addresses), the system incorporates an I2C multiplexer. This component segments the bus, enabling individual addressing and sequential data acquisition from all three sensors.

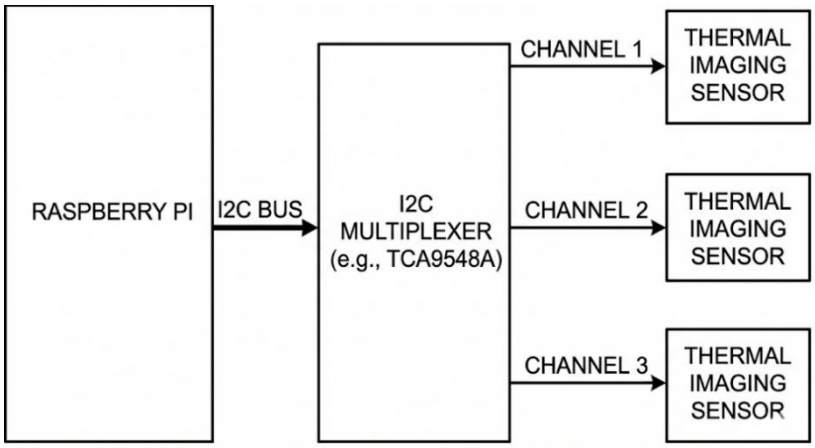


Figure 1 – System architecture with I2C sensors

To achieve necessary spatial resolution via triangulation, the sensors must be positioned at distinct corners of the room. This geometric requirement necessitates cabling lengths that exceed standard I2C specifications. The I2C protocol is not natively robust for long-distance transmission; bus lengths exceeding 1 meter are highly susceptible to signal degradation and data loss caused by parasitic capacitance.

Mitigation Strategies To address these signal integrity issues, the following solutions are currently under evaluation:

- Integration of I2C bus buffers or repeaters to drive higher capacitance loads. These would be connected between the MUX and the MLX sensors (3 would be required- one for each channel).
- Utilization of low-capacitance, shielded cabling to minimize signal attenuation. (again, one for each channel)
- Migration to a communication protocol designed for long-distance differential signaling (e.g., RS-485), which would necessitate selecting alternative sensor hardware.

As detailed in the component selection analysis, the **Waveshare Long Wave Thermal Camera USB module** presents a viable resolution to the signal integrity challenges. Unlike the current I2C implementation, this module utilizes the Universal Serial Bus (USB) protocol for data transmission. USB is inherently more robust over extended cable lengths, effectively mitigating the degradation issues caused by parasitic capacitance. Furthermore, this architecture simplifies the hardware topology by eliminating the I2C multiplexer, as the three sensors can be interfaced directly via the Raspberry Pi's onboard USB ports.

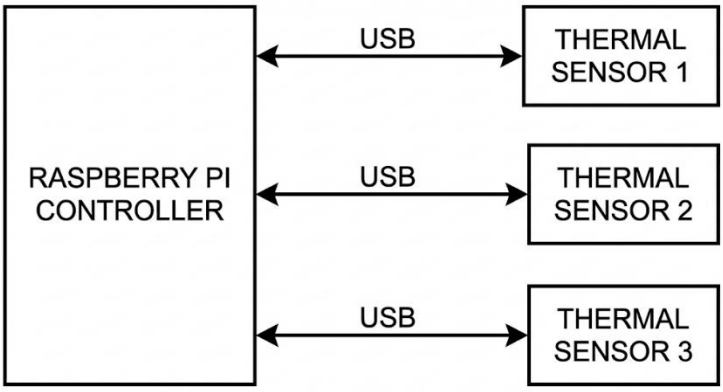


Figure 4 – System architecture with USB sensors

Mechanical Packaging and System Security Complementing the electronic hardware, the system will utilize custom-fabricated housing produced via additive manufacturing (3D printing). The mechanical design includes a centralized chassis to accommodate the Raspberry Pi controller and the Multiplexer unit, along with three individual enclosures for the distributed sensor nodes. These rigid casings are critical for ensuring system integrity in a clinical setting; they provide strain relief for cabling and serve as tamper-resistant barriers to prevent accidental disconnection or intentional interference by patients.

4.2.1 Flow diagram

As stated before, the system needs to handle 3 main challenges:

- Presence detection:
  In case a patient enters a restricted area or in order to confirm presence of patients in rooms for more complex detection scenarios.
- Contact detection:
  Detect inappropriate contact between patients
- Fire detection:
  Detect fire in early stages.

The algorithmic flow in High-level is as shown in the following diagram:

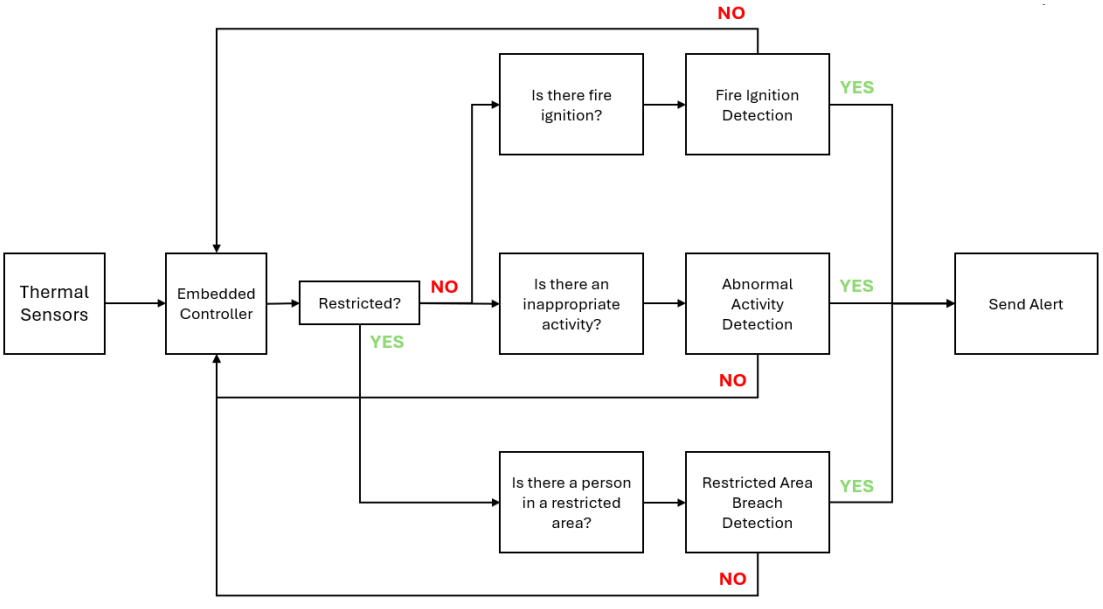


Figure 5 – High level Algorithmic flow

4.3 Data Collection and Labelling

4.3.1 Multi-label vector labeling

The initial subset of the dataset we have gathered thus far, was created to facilitate the process of exploratory data analysis, and the evaluation of classical algorithms. This pilot set consists of recorded videos annotated for three specific binary classification tasks: the presence of fire, the presence of humans, and the occurrence of physical contact. Consequently, each data point is associated with a label vector $y = [Human, Fire, Touch]^T$. These labels represent existence rather than enumeration (i.e., we classify presence or absence, not the number of occurrences).

This initial subset of the dataset (which comprises 459 thermal labeled images) is what we have managed to gather and label thus far, and in later stages of the project, more data will be collected to comprise a more fitting dataset for training machine learning models.

Representative examples are provided below:

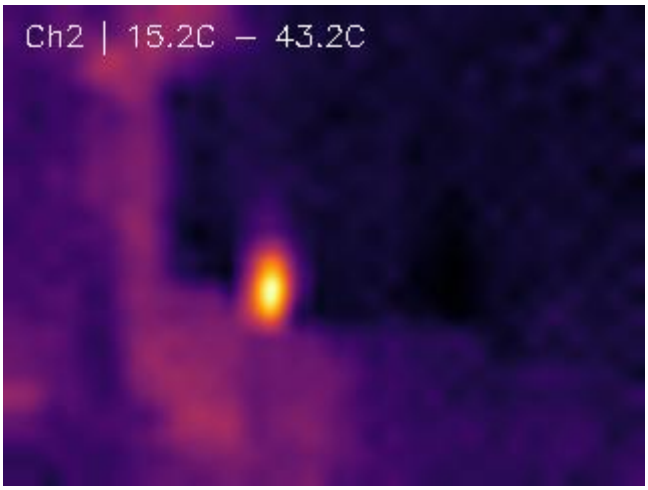


Figure 6 - Ignited lighter captured at 1 meter.

This image depicts an ignited lighter recorded at a distance of 1 meter. It is classified as $[0,1,0]^T$ (Fire only).

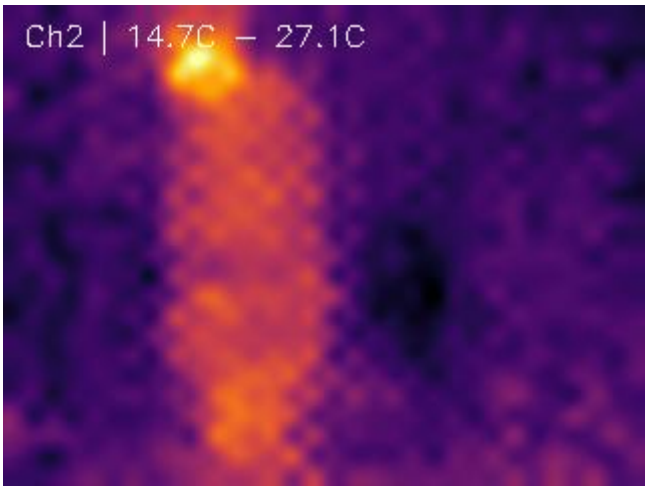


Figure 7 - Person standing 2 meters away wearing a coat

This image captures a human subject standing approximately 2 meters from the sensor. The subject is wearing a coat, which insulates the body heat, creating a distinct thermal contrast between the head and the torso. The classification vector is $[1,0,0]^T$.

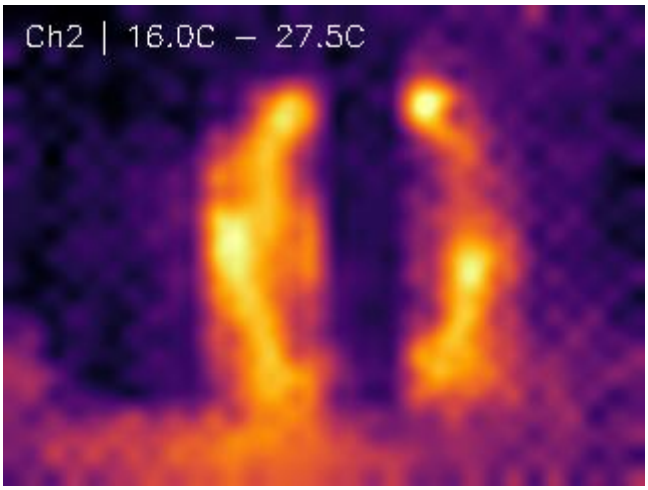


Figure 8 - Two people facing each other (1–2m distance) without contact

Two subjects are shown facing each other at a distance of 1–2 meters. The image is classified as $[1,0,0]^T$, indicating the presence of humans but the absence of fire or physical contact.

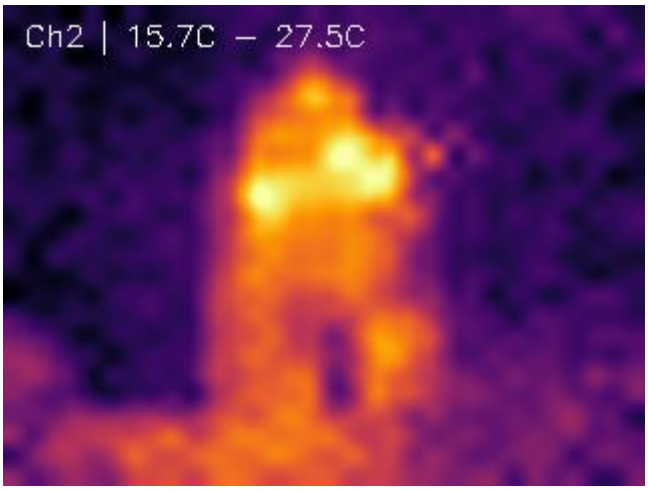


Figure 9 - Two people hugging (1–2m distance)

Two subjects are shown embracing (hugging) at a distance of 1–2 meters. The classification vector is $[1,0,1]^T$, denoting the presence of humans and physical contact, with no fire present.

4.3.2 Bounding box labeling

While this project doesn't use object detection algorithms, we labeled each of the frames that had either fire or a human in it to have a better "ground truth" understanding of the data.

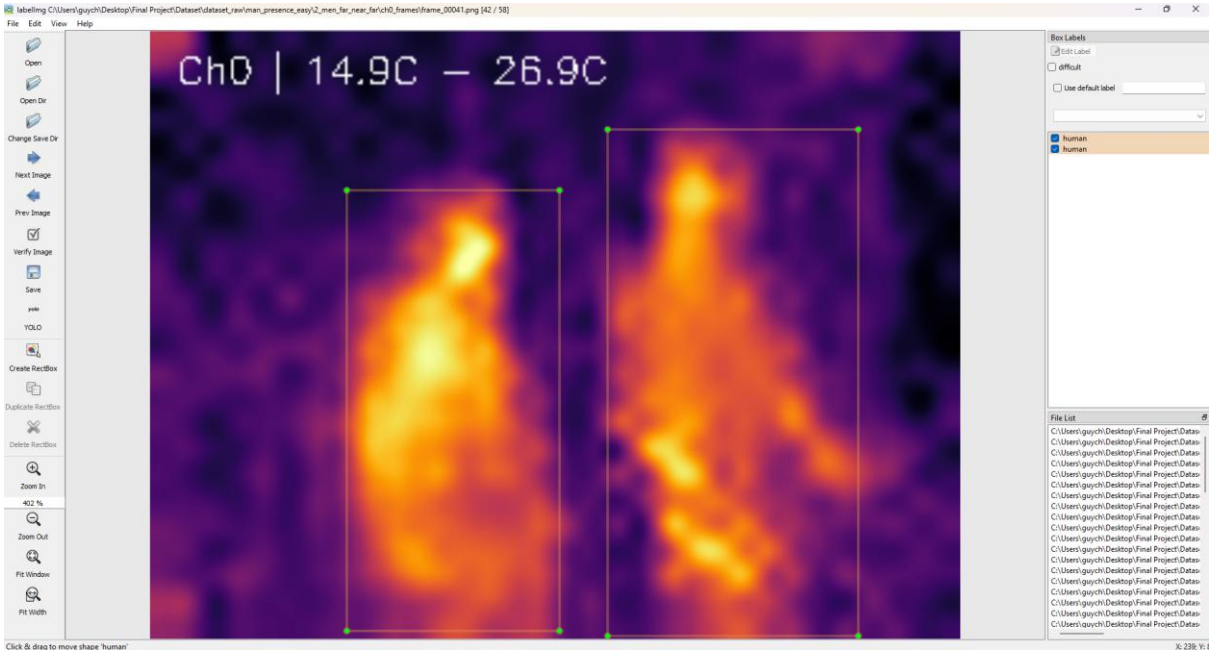


Figure 10 – Labeling Software

We used the imgLabel python package to label each of the labeled images with bounding boxes labeled as fire or human. We used the YOLO format, saving a .txt file that includes the location of the box in the image, normalized to image size (so we can always display the bounding box, even if the image is resized, or even if it's height-width ratio changes).

Labeling the images in this way manually is tedious, but it allows us to reach better conclusions in the exploratory data analysis phase, so it was deemed necessary.

4.3.3 Dataset Summary

The examples shown above span all of the labels that were used for this small dataset that comprised of a total of 459 frames. Most of the videos were recorded by two sensors with different perspectives and are called channels (0/2). The following table shows the structure of the dataset that was recorded:

Category	Video name	Channel	Number of Frames	Description
Fire	lighter_on_3_sec	0	19	A lighter was ignited
		2	19	
	lighter_on_and_off_3_sec	2	15	A lighter was ignited and extinguished
	lighter_on_off_on_off_5_sec	0	26	A lighter was ignited and extinguished twice
		2	26	
Human	1_man_jump_in_place_4_sec	0	14	A man jumps in place
		2	14	
	1_man_out_in_out_7_sec	0	33	A man walks into the image, then walks out, wearing a coat
		2	33	
	1_man_out_in_out_ no_coat_7_sec	0	27	A man walks into the image, then walks out, without wearing a coat
		2	27	
	1_sec_static_1_man	0	5	A man standing in place
		2	5	
	2_men_far_near_far	0	58	Two men, starting a far from each other, coming into a hug, then walking away from each other
		2	58	
	5_sec_1_man_movement	0	21	A man walks back and forth
		2	21	
	No_man_3_sec	0	19	Empty image, no man or fire (control group)
		2	19	

Table 2 - Dataset Structure

4.3.4 Augmentations

Due to the logistical and ethical constraints of recording in a sensitive mental health facility, acquiring a large-scale annotated dataset is challenging. Deep learning models, however, typically require extensive data to minimize overfitting. To address this scarcity, we employed **Data Augmentation** techniques to synthetically expand the training corpus.

We applied the following transformations to the raw thermal images:

- **Geometric Transformations:** To enforce **rotational and reflectional invariance**, images were randomly rotated by $\theta \in [-15^O, 15^O]$ and horizontally flipped. This ensures the model learns to recognize subjects regardless of their orientation relative to the camera.
- **Thermal Noise Injection:** To simulate sensor instability and improve robustness against low-fidelity inputs, Gaussian noise $(\mu = 0, \sigma = 2^O C)$ was injected into the thermal maps.

This process expanded the effective dataset size by a factor of 5. Beyond merely increasing the sample count, these augmentations prevent the model from memorizing specific spatial patterns (overfitting), thereby compelling it to learn robust, generalized feature representations.

4.4 Algorithm Description

In this section we will discuss the various solutions standing in front of us regarding the 3 main challenges we intend to solve.

4.4.1 Pre-processing Pipeline

In order to better understand the data and have each of the following algorithms explained in this chapter perform better, we propose a novel pre-processing pipeline, based on S. Tateno's paper.

The pipeline is built as follows:

Figure 11: Pre-processing Pipeline Model

1. **Spatial Denoising** is the process of removing noise from an image by exploiting the high spatial correlation between neighboring pixels, operating on the theory that noise often appears as high-frequency variation while the underlying image structure (like surfaces and objects) changes more gradually. More specifically, **Gaussian smoothing** is a linear spatial denoising technique that acts as a low-pass filter, reducing high-frequency noise by replacing each pixel's value with a weighted average of its neighbors. Unlike a simple box filter that weighs all neighbors equally, Gaussian smoothing assigns weights according to a Gaussian distribution (bell curve), giving significantly higher importance to the central pixel and immediate neighbors while diminishing the influence of distant pixels. This approach effectively mimics the blur produced by an out-of-focus optical system and provides a smoother, more natural reduction of noise with fewer edge artifacts than a uniform average.

According to the OpenCV documentation[33], the output image is calculated by convolving the input image with a 2D Gaussian kernel. The coefficients of this kernel, $G(x, y)$, are determined by the following formula:

$$G(x,y) = A \cdot e^{-\left(\frac{(x-\mu_x)^2}{2\sigma_x^2} - \frac{(y-\mu_y)^2}{2\sigma_y^2}\right)}$$

Where:

- x and y are the distances from the origin (the center of the kernel)
- μ_x and μ_y are the means (the peak positions), typically located at the center of the kernel.

[33] openCV's gaussianBlur function's documentation

- σ_x and σ_y are the standard deviations in the x and y directions, respectively, which control the spread (amount of blurring) of the Gaussian function.
- A is a normalization constant ensuring that the sum of the kernel weights equals 1.

2. **Background subtraction** is a fundamental computer vision technique that isolates moving or significant objects (foreground) from a static scene by treating the visual input as a superposition of a time-invariant background layer and a time-variant foreground layer. Theoretically, this acts as a temporal high-pass filter, where signals that remain constant over time (DC components) are suppressed, while changing signals (AC components) are preserved. In our case, **Environmental Filtering** refers to the semantic goal of this operation: mathematically removing the energy signature of the physical environment—such as the thermal emissions of walls — so that the only remaining signal energy corresponds to anomalies (humans or fire).

The theoretical calculation involves two stages: modeling the reference background $B(x, y)$ and computing the difference map $D(x, y)$.

a. Background Modeling (Mean Field Approximation): We estimate the static environment by averaging K calibration frames where no targets are present:

$$B(x, y) = \frac{1}{K} \sum_{k=1}^{K} I_{calib}(x, y, k)$$

b. Difference Calculation (Environmental Filtering): For a new frame $I(x, y, t)$, the environmental signal is removed by subtraction:

$$D(x, y, t) = I(x, y, t) - B(x, y)$$

3. **Residual Rectification** is the process of converting the signed difference signal (the residual) into a strictly positive magnitude, functioning theoretically as an energy extraction step. In signal processing terms, this is analogous to "full-wave rectification" where an alternating current (AC) signal is converted to direct current (DC) to measure its total power regardless of polarity. In the context of anomaly detection for thermal or radar data, a "negative" residual occurs when a target is colder or less reflective than the background environment (e.g., a human blocking a warm radiator). Theoretically, this step ensures that the system detects the *presence* of a disturbance rather than its directionality, preventing the loss of information that would occur if negative values were simply clipped to zero.

$$Output(x, y, t) = |D(x, y, t)| = |I(x, y, t) - B(x, y)|$$

The mathematical operation calculates the L1 norm (Manhattan distance) of the residual at each pixel.

4.4.2 Human Detection

To reliably detect patient presence using a single thermal sensor, the system must distinguish the human thermal signature from environmental noise and static heat sources (e.g., radiators, electronics). We will review 3 different approaches:

1. **Adaptive Gaussian Thresholding (spatial filtering):**
   This classical approach relies on the premise that a human target introduces a local high-frequency thermal anomaly relative to their immediate surroundings. Unlike global thresholding, which uses a single temperature value for the entire room, this method adapts to local thermal gradients, making it robust against uneven room heating.
   Mathematical Formulation:
   The algorithm calculates a unique threshold $T(x,y)$ for every pixel based on a weighted average of its neighborhood. The binary segmentation mask $D(x,y)$ is derived as:

$$D(x,y) = \begin{cases} 1, & I(x,y) > \left(\sum_{i=-k}^{k}\sum_{j=-k}^{k} G(i,j)\cdot I(x+i,y+j)\right) - C \\ 0, & else \end{cases}$$

   Where G is a Gaussian kernel of size $(2k+1)\times(2k+1)$ and C is a constant offset to filter noise.

   Algorithm flow
   a. Adaptive thresholding – compute the local mean for each pixel, and segment regions that exceeded that mean by constant C.
   b. Morphological Closing – At the segmented regions, perform dilation followed by erosion to fill thermal gaps within the detected heat blob (caused by clothing insulation, for example).
   c. Geometric Filtering - Validate the detected blob using solidity and aspect ratio constraints (rejecting perfect squares or blobs that are too small, far from the human temperature range, etc.).

2. **Histogram of Oriented Gradients (HOG) + SVM (Feature-Based ML)**
   This approach shifts focus from "temperature intensity" to "shape analysis."
   It is effective for distinguishing a person from a radiator, as a radiator has rigid vertical edges while a human has complex curves.

   Mechanism
   HOG (Histogram of Oriented Gradients): In this part, we first calculate the gradient magnitude and orientation for every pixel in the image. We then divide the image into a grid of cells (typically 8×8 pixels). Within each cell, we construct a histogram of gradient orientations, binning the angles from $0°$ to $180°$ and weighing the votes by the pixel's gradient magnitude. To ensure robustness against local contrast variations (thermal intensity shifts), these cells are grouped into larger blocks (e.g., 2×2 cells) for contrast normalization. Finally, these normalized histograms are concatenated into a single feature vector that describes the object's structural shape.

   SVM (Support Vector Machine): A classifier that operates within a supervised learning framework, necessitating a labeled dataset of HOG feature vectors for calibration. During the training phase, the SVM solves a convex optimization problem to determine the optimal weight vector W and bias term b. This process constructs a hyperplane decision that

maximizes the geometric margin between positive (human) and negative (background) samples by minimizing the associated **Hinge Loss**.
Mathematically, the system minimizes the following objective function, which balances margin maximization with error penalization:

$$\min_{W,b}\left(\frac{1}{2}\|W\|^2 + C\sum_{i=1}^{N} max\big(0, 1 - y_i(W \cdot x_i + b)\big)\right)$$

where $\boldsymbol{y_i}$ is the class label $(\pm\mathbf{1})$ and $\boldsymbol{C}$ is the regularization parameter.
During the inference phase, the system does not 'compare' images loosely; it projects the feature vector X of a new frame onto this learned weight vector. The presence of a human is determined by the sign of the decision function:

$$f(X) = sign(W \cdot X + b)$$

A positive result indicates that the feature vector lies on the 'human' side of the decision boundary, while a negative result classifies the region as background.

Algorithm Flow

a. Gradient Computation: Calculate the horizontal (G_x) and vertical (G_y) gradients for every cell:

$$Magnitude = \sqrt{G_x^2 + G_y^2}, \qquad Angle = \arctan\left(\frac{G_y}{G_x}\right)$$

b. Cell Histogramming: Vote gradient magnitudes into angular bins (e.g., 9 bins for 0-180 degrees) for each cell.
c. Block Normalization: Normalize histograms across larger blocks to account for global contrast changes.
d. SVM Classification: Feed the feature vector into a pre-trained Linear SVM. The SVM outputs a positive score if the features match the "Human" hyperplane and negative otherwise.

3. Convolutional Neural Network (MobileNet-SSD)

This approach utilizes Deep Learning to perform object detection. Unlike the previous methods which rely on manual feature engineering, a Convolutional Neural Network (CNN) automatically learns hierarchical features - from edges to body parts, to identify the patient.
MobileNet-SSD is a hybrid deep learning architecture designed specifically for resource-constrained environments. It combines MobileNet, a feature extractor built on efficient Depthwise Separable Convolutions, with the Single Shot MultiBox Detector (SSD), which performs localization and classification in a single forward pass. This combination allows it to bypass the heavy computational load of standard Convolutional Neural Networks (CNNs), making it potentially viable for the Raspberry Pi.
Mechanism:
The network performs "Single Shot Detection" (SSD). It discretizes the output space of bounding boxes into a set of default boxes over different ratios and scales per feature map location.
Algorithm Flow:

a. Input Normalization: Resize the thermal frame to 300×300 pixels and normalize pixel values to $[-1, 1]$.

b. Feature Extraction: The image passes through the MobileNet backbone (depth-wise separable convolutions) to extract feature maps.
c. Bounding Box Prediction: The SSD head predicts offsets for bounding boxes and class probabilities (e.g., "Person: 98%") for each box.
d. Non-Maximum Suppression (NMS): If multiple overlapping boxes detect the same person, NMS suppresses all but the one with the highest confidence score.

4.4.3 Contact detection

While detecting human presence is standard, detecting interaction presents a significant challenge. Given the strict no-contact policy at the Mental Health Center, the system aims to identify prohibited behaviors such as violence or sexual relations. To achieve this, the system requirements were defined to alert on any physical contact between patients—a simplified but still complex objective.

This section evaluates three distinct methodological approaches to contact detection: classical computer vision, traditional machine learning, and deep learning. Beyond analyzing the theoretical underpinnings of each algorithm, we propose an integration strategy that synthesizes these methods to ensure a comprehensive and robust system architecture.

1. Classical Computer Vision – Geometric Fusion Pipeline

The first algorithmic approach utilizes classical multi-view geometry. We employ Planar Homography to fuse the three camera views onto a common coordinate system (the floor plane). This method is deterministic, explainable, and computationally lightweight, making it an ideal baseline.

Mathematical Formulation: Homography and Projection

1. The Homography Matrix[34]

The relationship between a point $x = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$ in the camera image plane and a point $X = \begin{bmatrix} X_w \\ Y_w \\ w \end{bmatrix}$ on the room's floor ($Z = 0$) is described by a projective transformation matrix H (homography). The relation is defined as follows:

$$X \sim Hx$$

$$\begin{bmatrix} wX_w \\ wY_w \\ w \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \cdot \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

Where w is some scalar, given by the calculation and allows the calculation of X_w, Y_w (homogenous coordinates).

The matrix H has 8 degrees of freedom (since it is scale invariant, usually $h_{33} = 1$).

2. Calibration via Thermal Markers

To resolve the homography matrix H, point correspondence between the image plane and the ground plane are required. Standard optical checkerboards lack thermal contrast and are indistinguishable to infrared sensors. Consequently, we utilized Heated Calibration Targets (e.g., heated fluid containers) positioned at known floor coordinates.

For a set of $N \geq 4$ point pairs, we formulate a system of linear equations $Ah = 0$, which is solved using SVD (singular value decomposition).

The mapping from image coordinates (u, v) to world coordinates (X_w, Y_w) is given by:

[34] Introduction to Computer Vision (UC Berkeley) – Image analysis: homography: planar homography

$$X_w = \frac{h_{11}u + h_{12}v + h_{13}}{h_{31}u + h_{32}v + h_{33}}, \qquad Y_w = \frac{h_{21}u + h_{22}v + h_{23}}{h_{31}u + h_{32}v + h_{33}}$$

By clearing the denominator and rearranging terms, we derive the following homogeneous linear equations for a single point correspondence:

$$-u \cdot h_{11} - v \cdot h_{12} - h_{13} + uX_w \cdot h_{31} + vX_w \cdot h_{32} + X_w \cdot h_{33} = 0$$

$$-u \cdot h_{21} - v \cdot h_{22} - h_{23} + uY_w \cdot h_{31} + vY_w \cdot h_{32} + Y_w \cdot h_{33} = 0$$

This is an example of one point pair, and the more points you have, the more accurate the estimation of h_{nm} parameters will be.

Algorithm Flow: The Geometric Pipeline

For each camera view $k\epsilon\{1,2,3\}$:

1. Segmentation and Foot Point Extraction:

The pipeline begins by ingesting the bounding box data from the human detection module. To project a subject's location onto the 2D floor plan, we rely on the planar assumption that the subject is standing on the ground. Therefore, the contact point is approximated as the bottom-center of the detected bounding box.

Given a bounding box defined by top-left coordinates (x, y) and dimensions (w, h), the foot point P_{foot} is calculated as:

$$P_{foot} = \left(x + \frac{w}{2}, y + h\right)$$

Where the point P_{foot} serves as $\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$ for the homography transformation derived in Mathematical Formulation section.

2. Ground Plane Projection

Apply the homography H_k to every detected P_{foot} in camera k.

$$P_{ground} = H_k \cdot P_{foot}$$

This transforms pixel coordinates into real-world meters. We now have a cloud of points on the room map representing the locations of people.

3. Data Association and Clustering

Ideally, a single subject is detected by all three cameras, yielding a set of three projected floor points. In practice, the number of detections N (where $N \leq 3$) may vary due to:

a) **Field of View (FOV) Constraints:** The subject occupies a blind spot in one or more cameras.

b) **Occlusion:** The line-of-sight is obstructed by another patient or an obstacle.

To resolve the true location, we deploy a validation and estimation algorithm on the set of projected points $\{P_n\}_{n=1}^{N}$:

1. Validation:

i. If $N = 1$, Single-source detections lack cross-validation and are discarded to prevent false positives.

ii. If $N \geq 2$, we compute the Euclidean distance $D_{ij} = \|P_i - P_j\|$ between all available points.

1. Points are considered valid only if $D_{ij} < \epsilon$, where ϵ is a pre-defined error threshold accounting for homography inaccuracies.

2. **Outlier Removal ($N = 3$):** If only one pair satisfies the threshold (e.g., $D_{12} < \epsilon$, but $D_{13} > \epsilon$ and $D_{23} > \epsilon$), the point common to the failing pairs (P_3) is identified as an outlier and is discarded. The system proceeds using only the consistent pair (P_1, P_2).

2. Position Estimation:

The final subject location $(\hat{P}_x, \hat{P}_y)$ is computed as the centroid of the M validated points:

$$\hat{P}_x = \frac{\sum P_{k,x}}{N}, \qquad \hat{P}_y = \frac{\sum P_{k,y}}{N}$$

3. Contact Logic

Following the position estimation step, the system holds a set of unique, validated coordinates $\{\hat{P}_k\}_{k=1}^{K}$ representing the floor location of K distinct individuals in the scene.

To identify potential contact events, we compute the pairwise Euclidean distance between every unique pair of individuals (i, j) where $i \neq j$. The inter-patient distance $D_{i,j}$ is defined as:

$$D_{i,j} = \|\hat{P}_i - \hat{P}_j\|_2 = \sqrt{(\hat{P}_{i,x} - \hat{P}_{j,x})^2 + (\hat{P}_{i,y} - \hat{P}_{j,y})^2}$$

A physical contact event is flagged if the proximity between any two subjects falls below a safety threshold δ:

$$Event(i,j) = \begin{cases} ALARM & D_{i,j} < \delta \\ SAFE & o.w \end{cases}$$

The threshold δ is is empirically calibrated to account for the physical width of the subjects and the intrinsic noise of the thermal projection (typically set to $\delta \approx 0.5$ meters).

2. Deep Learning Classifier I – Multi-View Spatiotemporal Graph Convolutional Network (MV-STGCN)

We elected to evaluate two distinct Deep Learning architectures, rather than contrasting a classical Machine Learning baseline with a Deep Learning approach, due to the complex spatio-temporal nature of Contact Detection. This task necessitates both temporal consistency—to maintain object identity via pre-merge history—and the ability to resolve subtle volumetric and thermal shifts indistinguishable by manual feature extraction. As the Exploratory Data Analysis (EDA) yielded no discriminative explicit features, we determined that Deep Learning's capacity for automated, high-dimensional representation learning is required to capture these intricate dependencies.

General Background

Our second algorithm adopts a **symbolic AI** approach underpinned by deep learning. Instead of asking a neural network to understand "contact" directly from pixels, we first extract structured information (bounding boxes) and then reason about their relationships.

In the domain of action recognition, ST-GCNs have become the state-of-the-art for skeletal data. They represent the human body as a graph of joints. Here, we adapt this concept: instead of joints, our graph nodes are **detected humans**, and the edges represent **potential interactions**.

This fits our exact system needs because of three attributes:

1. **Resolution Independence:** Once the bounding box is detected, the 32x24 pixel grid is abstracted into floating-point coordinates. The logic for contact detection (proximity in 3D space) is independent of the low sensor resolution.
2. **Efficiency:** Graph operations involve multiplying small matrices (e.g., 5×5 adjacency matrices). This is computationally negligible for the Raspberry Pi 5, leaving the bulk of the processing power for the object detector.
3. **Data Efficiency:** Learning a distance threshold via a GCN requires far less data than learning visual textures via a CNN. Even a smaller, labeled dataset like the one offered in this project, should be enough to reliably train a neural network of this architecture, if the object detector is pre-trained.

Mathematical Formulation

The algorithm proceeds in three stages: (1) Object Detection and Tracking, (2) Multi-view Homography Fusion, and (3) Graph Convolution.

Stage 1: Thermal Object Detector (Adapted YOLO)

Object Detector

We employ a lightweight detector, YOLO-Nano, adapted for 32×24 inputs. The network f_θ takes an image $I \in \mathbb{R}^{32\times24}$ and outputs a tensor representing grid cells.

Given the small input, we modify the standard YOLO grid. We divide the image into a 4×3 grid (cells of 8×8 pixels). Each cell predicts B bonding boxes. The loss function is critical. In low resolution, a 1-pixel error in box position causes a massive drop in Intersection over Union (IoU). Standard IoU Loss is unstable here. We utilize **Complete IoU (CIoU) Loss**, which considers overlap, center distance, and aspect ratio:

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(\boldsymbol{b}, \boldsymbol{b}^{gt})}{c^2} + \alpha v$$

Where:

- $\boldsymbol{b}, \boldsymbol{b}^{gt}$ are the center points of the predicted and ground truth boxes.
- $\rho(\cdot)$ is the Euclidean distance $\left(\rho(\boldsymbol{b}, \boldsymbol{b}^{gt}) = \sqrt{\left(\boldsymbol{b}_x - \boldsymbol{b}_x^{gt}\right)^2 + \left(\boldsymbol{b}_y - \boldsymbol{b}_y^{gt}\right)^2}\right)$.
- c is the diagonal length of the smallest enclosing box covering both.
- v measured the consistency of aspect ratio: $v = \frac{4}{\pi^2}\left(\arctan\left(\frac{w^{gt}}{h^{gt}}\right) - \arctan\left(\frac{w}{h}\right)\right)^2$.
- α is a trade-off parameter.

This loss function forces the network to prioritize exact center alignment, which is crucial for the subsequent fusion step.

Tracker

Following detection, we implement a Kalman Filter to address two specific hardware realities of this system:

- **I2C Phase Shift:** The TCA9548A reads sensors sequentially. Camera 3 is read approximately 40-50ms after camera 1. Fast-moving humans will have shifted positions, creating "ghosting" artifacts during multi-view fusion.
- **Quantization Jitter:** In a 32×24 grid, a single pixel represents a large physical area. Thermal noise can cause a static bounding box to "jump" 1 pixel back and forth. This creates artificial velocity signals that confuse the contact detection network.

Assign a distinct Kalman Filter to each detected track using a Constant Velocity Model. The state vector s_t at time t is defined as:

$$s_t = [u, v, \dot{u}, \dot{v}]^T$$

Where (u, v) are the pixel coordinates of the bounding box center and $(\dot{u}, \dot{v})$ are the velocities.

The **Prediction Step** estimates the state at the next time step (or at the specific timestamp of the Camera 3 read):

$$s_{t+\Delta t} = F_{s_t} + w_t$$

Where F is the state transition matrix:

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This allows us to mathematically "fast-forward" the detection from Camera 1 by Δt to align it perfectly with the capture time of Camera 3. Furthermore, the update step (fusing the noisy YOLO measurement with the prediction) acts as a low-pass filter, smoothing out the 1-pixel quantization jitter and providing stable velocity vectors for the Graph Network.

Stage 2: Homographic Projection to Ground Plane

To detect physical contact, we must distinguish between 2D image overlap (occlusion) and 3D physical proximity. We project the "foot point" of each detected human to a common Top-Down Ground Plane map.

Let $p_{img} = [u, v, 1]^T$ be the homogeneous pixel coordinates of the bottom-center of a bounding box (representing the person's location on the floor). Let $p_{world} = [wX_w, wY_w, w]^T$ be the coordinates in the room (in meters). The relationship is governed by a Homography Matrix H_c for camera c:

$$p_{world} \cong H_c \cdot p_{img}$$

Calibration without Checkerboards: Standard checkerboard calibration fails in thermal because printed paper has uniform emissivity. We must perform **"Hot Point Calibration"** as explained in the first method.

Once calibrated, detections from all 3 cameras are projected to the ground plane. Detections falling within a threshold distance (e.g., 0.5m) on the ground plane are fused into a single **Unique Actor Node** via a clustering algorithm (as explained in first method).

Stage 3: Spatiotemporal Graph Convolution

We construct a graph $G = (V, E)$ over a sliding window of time T (e.g., 16 frames).

- **Nodes (V):** Each unique actor identified in the fusion step.
- **Edges (E):**
    - *Spatial Edges:* Connect all actors in frame t.
    - *Temporal Edges:* Connect actor i at time t to actor i at time $t + 1$.

The input feature vector $F^{(0)}$ for each node contains:

- **Global Position:** (X, Y) normalized.
- **Velocity:** (v_x, v_y).
- **Thermal Intensity:** Max and Mean temperature within the bounding box (normalized).
- **Bounding Box Area:** Proxy for height/posture.

The Graph Convolution Layer follows the formulation of Kipf & Welling, adapted for spatiotemporal dynamics:

$$F^{(l+1)} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\tilde{A}\widetilde{D}^{-\frac{1}{2}}F^{(l)}W^{(l)}\right)$$

Where:

- $\tilde{A} = A + I_N$ is the adjacency matrix with self-loops.
- $\widetilde{D}$ is the degree matrix.
- $W^{(l)}$ is the learnable weight matrix.
- σ is a non-linear activation (ReLU).

However, for contact detection, the *topology* of the graph is dynamic (people move). We use an **Adaptive Adjacency Matrix**. The edge weight A_{ij} is not fixed but computed based on the spatial distance between nodes:

$$A_{ij} = \frac{\exp\left(-\frac{dist(i,j)^2}{\sigma^2}\right)}{\Sigma_k \exp\left(-\frac{dist(i,k)^2}{\sigma^2}\right)}$$

This "Softmax Attention" mechanism allows the network to focus strongly on neighbors that are physically close, prioritizing them for the contact classification decision.

The final layer is a global average pooling followed by a dense layer with Softmax output, giving the probability of the event being a Physical Contact event vs. being a No Contact event.

Algorithm Flow

Step 1: Initialization

- Load the 3 calibrated Homography Matrices (H_1, H_2, H_3).
- Load the pre-trained YOLO-Nano model (TFLite format).
- Initialize the tracker (Kalman Filter).

Step 2: Concurrent Acquisition (Pipeline)

- The RPi initiates a loop at 8fps.
- Read Camera 1 → $buffer_1$ $(Timestamp\ t_0)$
- Read Camera 2 → $buffer_2$.
- Read Camera 3 → $buffer_3$ $(Timestamp\ t_0 + 50ms)$.
- Correction: Utilize the Kalman Filter prediction step to project detections from $buffer_1$ forward by $50ms$ to align with $buffer_3$.

Step 3: Detection & Projection

- Run TFLite Inference on $buffer_1, buffer_2, buffer_3$ (using separate cores on the RPi5 CPU).
- Extract centroids (u, v) and project to (X, Y) using H_c.

Step 4: Graph Assembly & Inference

- Cluster points to identify unique actors.
- Update the Graph history (FIFO buffer of 16 frames).
- Calculate features (Velocity, Distance).
- Run the ST-GCN inference (very fast, $< 5ms$).

Step 5: Logic Check

- If GCN probability for "Contact" > 0.7:
    - Check persistence: Has this state lasted more than 3 frames? (Removes noise).
    - Output Alert.

3. Deep Learning Classifier II - Volumetric Thermal Interaction Network (Thermo-X3D)

While the first Deep Learning Classifer is precise, it has a failure mode: **Detector Breakdown**. In scenarios of intimate contact (e.g., a caregiver lifting a patient), the thermal signatures may merge so completely that the object detector sees only one large bounding box. If there is only one node, the Graph Network has no edges to analyze, and Contact Detection fails.

General Background

This classifier, **Thermo-X3D**, is a pixel-based (volumetric) approach designed to act as a failsafe. It does not rely on bounding boxes. Instead, it treats the thermal video as a 3D volume of data ($space \times time$) and uses 3D convolutions to learn the texture of contact.

The hypothesis is that "Contact" has a unique spatiotemporal signature: a specific way the thermal gradients merge and evolve over time, which is distinct from a single person moving or two people standing apart.

Mathematical Formulation

We adapt the X3D (Expand 3D) architecture, which is the current state-of-the-art for efficient video classification, scaling it down to our low resolution.

Factorized 3D Convolutions

Standard 3D convolutions ($k_t \times k_h \times k_w$) are computationally expensive. We employ (2+1)D Factorization. A 3D kernel $W \in \mathbb{R}^{3\times3\times3}$ is approximated by:

- Spatial Convolution $W_s \in \mathbb{R}^{1\times3\times3}$ (spatial features)
- Temporal Convolution $W_t \in \mathbb{R}^{3\times1\times1}$ (temporal features)

This decomposition reduces the number of parameters by roughly 30-40% and adds an extra non-linearity (ReLU) between space and time, increasing the model's discriminative power without increasing computational load.

Thermal Attention Mechanism

Thermal images are sparse, with most pixels being cold background. We introduce a lightweight attention module to force the network to focus on the "hot" active voxels.

Let $X \in \mathbb{R}^{C\times T\times H\times W}$ be the feature map. We compute a Channel Attention Map M_c:

$$M_c = \sigma\left(MLP\big(AvgPool(X)\big) + MLP\big(MaxPool(X)\big)\right)$$

And a spatial Attention Map M_s:

$$M_s = \sigma\left(conv^{7\times7}\big(\big[AvgPool_c(X); MaxPool_c(X)\big)\big]\right)$$

The refined feature map is $X'' = X \cdot M_c \cdot M_s$. This mechanism effectively suppresses the cold background noise (walls, windows) and highlights the humans, making the "Isotherm Fusion" boundary easier for the network to analyze.

Late Fusion Architecture

Since pixel correspondence between cameras is non-linear, we process each view independently and fuse the high-level semantic features.

1. For stream i (i'th Camera): Processes $16 \times 32 \times 24$ input → Feature vector $v_i \in \mathbb{R}^{128}$
2. Fusion Layer: $v_{final} = Concat(v_1, v_2, v_3)$.
3. Classifier: Fully Connected Layer $(384 \rightarrow 64 \rightarrow 2)$.

Algorithm Flow

Stage 1: Data Preprocessing (Rolling Windows)

- The system maintains 3 Rolling Buffers of size 16 frames (2 seconds back).
- Normalization: Crucial for X3D.
    - Compute the global mean μ and std σ of the 459-frame training set.
    - Normalize input frame I: $I_{norm} = \frac{I-\mu}{\sigma}$.
    - This normalization helps the neural network learn temperature changes rather than absolute values.

Stage 2: Micro-X3D Architecture Definition

We define a custom small architecture in Pytorch:

- **Stem:** Conv3D $(1 \times 3 \times 3)$, stride 1, 24 filters (Preserves spatial resolution)
- **Block 1:** Factorized (2+1)D ResBlock, 24 filters.
- **Block 2:** Factorized (2+1)D ResBlock, 48 filters, stride 2 (spatial downsample to 16×12).
- **Block 3:** Factorized (2+1)D ResBlock, 96 filters, stride 2 (spatial downsample to 8×6).
- **Head:** Global Average Pooling → Dense.

Stage 3: Inference Strategy

- Instead of running 3 separate calls, we batch the inputs: Input Tensor shape:

$$(3,16,32,24) \rightarrow (Batch, Time, Heigth, Width)$$

- Run a single inference pass on the Pi 5.
- The output is a tensor of shape:

$$(3,128)$$

- Flatten this to:

$$(1,384)$$

A single vector of 384 features and pass through the final Dense classification layer.

Step 4: Handling Variable Frame Rates

- The sensors' sampling frequency is programmable from 1fps to 64fps.
- X3D is trained in temporal dynamics (velocity). If training is at 8 fps, running at 32fps will dilate time and break the model.
- To solve this problem, we use a fixed-step sampling strategy. Regardless of the capture FPS, the buffer always feeds the network frames sampled at an effective 8Hz rate.
    - If capturing at 64fps, skip 7 frames (take every 8^{th} frame) to fill the 16-frame buffer.
    - If capturing at 4fps, duplicate frames (upsample).

4.4.4 Fire Detection

This section shifts focus from monitoring patient activity to a critical safety function: the early detection of thermal hazards. This includes detecting active fires, electrical sparks and unauthorized ignition sources, such as lighters or cigarettes.

As will be shown later in the Exploratory Data Analysis section (section 5.2.1), the Maximum Temperature feature serves as a strong discriminator for large fires. However, it is insufficient for detecting small ignition sources due to the sensor's limited spatial resolution. Since each pixel must describe a relatively large chunk of physical space, a small flame occupies only a fraction of a sensor pixel. Consequently, its extreme heat is averaged with the cooler background, resulting in a recorded temperature that is significantly lower than the true combustion temperature (initial data recordings show values of 60°~75° C, where the true temperature is over 400° C). This causes the signal to potentially converge with the thermal signatures of static heat sources (such as hot beverages or electronics) or the upper range of human thermal emissions. Therefore, distinguishing these hazards requires use of explored to explore strategies that analyze spatial mass as well as temporal dynamics, in addition to raw intensity.

To address these challenges, we evaluate the following algorithmic strategies. We will start by describing the pipeline-based approach with several layers and then discussing a machine-learning approach.

Pipelined Approach

1) **Adaptive Segmentation (Otsu's Method):**
   This approach serves as the foundational pre-processing step for the fire detection pipeline. Before applying segmentation, it is necessary to address the sensor's sensitivity to thermal noise. This noise can be amplified by contrast stretching in empty rooms. This amplification becomes a problem once Otsu's method is applied, since it depends on the variance on the frame, as explained below. For this reason, the segmentation itself is a pipeline approach, defined by these steps:
   a) **Variance Gating-** We define the dynamic range of the frame as $\Delta T = T_{max} - T_{min}$. If ΔT falls below a minimal noise floor ΔT_{nom} (currently estimated as 1.5°C difference), the frame is immediately classified as background. This prevents the algorithm from "forcing" a threshold on random sensor noise, effectively filtering out False Positives in static, cold environments.
   b) **Segmentation by Otsu's Method-** Unlike fixed thresholding, which uses an arbitrary cutoff (for example, 50° C), Otsu's method is a histogram-based threshold selection technique that automatically selects a global threshold for separating the foreground (suspected fire) and background (non-fire) based on the statistical properties of the current frame. This method assumes that the image has a bimodal histogram (two clearly distinct peaks), making it especially effective for images where the background and foreground indeed have different intensity distributions.
      Mathematical Formulation-Otsu's Method:
      The algorithm selects an optimal threshold by maximizing the variance between these two classes while minimizing the variance within each class, thereby achieving the best possible intensity separation.[35]

[35] Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms.

The thermal pixels are represented by discrete gray levels; the histogram is normalized and thus the probability of occurrences for each gray level is:

$$p_i = \frac{n_i}{N} \ , \ \sum_{i=0}^{L-1} p_i = 1 \ , \ p_i > 0$$

where pixels are represented by discrete gray levels $L \in [0,1,\dots,L-1]$, n_i denotes number of pixels at level i, and the total number of pixels is N. Pixels are divided into two classes $C_0 - background, C_1 - foreground$, separated by threshold k. A probability ω is calculated for each class as a cumulative weight:

$\omega_0(k) = \sum_{i=0}^{k} p_i \ , \ \omega_1(k) = \sum_{i=k}^{L-1} p_i = 1 - \omega_0(k)$
as well as the mean, which is the average thermal intensity withing each class:

$$\mu_0(k) = \sum_{i=0}^{k} \frac{i \cdot p_i}{\omega_0(k)} \quad , \quad \mu_1(k) = \sum_{i=k+1}^{L-1} \frac{i \cdot p_i}{\omega_1(k)}$$

Otsu defines within-class and between-class variances, which aim to describe how wide the values within each class are spread, and how far apart the two classes are from one another. Since the total variance of data in the image is defined in the article to be:

$$\sigma^2_{Total} = \sigma^2_{within} + \sigma^2_{between}$$

To maximize one is to minimize the other. As such, we will focus on the between-class variance:

$$\sigma_B^2 = \omega_0(k)\omega_1(k) \cdot [\mu_1(k) - \mu_0(k)]^2$$

The algorithm aims to find a threshold k^* that maximizes the between-class variance to create the most reliable separation between the two, as σ_B^2 quantifies the separability of the classes. So, the optimal threshold is:

$$k^* = argmax\ \sigma_B^2(k)\ , 0 \le k < L - 1\ .$$

c) Morphological Shaping- The need for morphological shaping became apparent only once we began evaluating Otsu's method over the preliminary dataset**.** Once masking via Otsu is performed, the segmentation produced suffers from some artifacts due to the sensor's low resolution: Salt-and-Pepper noise (high frequency thermal noise that passes the threshold) and under-segmentation (the detected blob often represents only the hottest core of the object). To address this, we use a 3x3 kernel to apply Erosion and Dilation on the frame.
Erosion is a logical 'AND' operation that removes isolated noise pixels. Dilation is a logical 'OR' operation that is applied twice- once to restore the object size after being decimated by Erosion, and once more to expand the mask boundary to overlap with the true geometric boundary of the target.

Algorithm Flow:

a) Variance Check- Calculate dynamic range ΔT. If $\Delta T < \Delta T_{nom}$, output black frame (background).
b) Histogram Computation- Compute the normalized histogram probabilities p_i for each thermal frame.
c) Iterative Evaluation- Calculate the class weights ω and means μ for every possible threshold k between 0 and L-1.
d) Variance Maximization- Compute the between-class variance $\sigma_B^2(k)$ and identify the k value that yields the maximum $\sigma_B^2(k)$ value.
e) Segmentation- Apply binary thresholding using k* to generate the raw mask.
f) Morphological Optimizaition- Apply one iteration of Erosion, followed by two iterations of Dilation, to generate the region of interest (ROI) mask, isolating potential thermal threats.

2) Hierarchical Rule-Based Classifier (Heuristic Decision Tree)

This classifier operates on the binary mask generated by the Adaptive Segmentation stage written above. At this point we have successfully isolated thermal anomalies from the background but have yet to develop a semantic understanding of the object - is the anomaly a human, a static object or a lighter? To address this ambiguity, we will consider other physical attributes, aside from temperature value, to be gathered from the data.

A Decision Tree Classifier (DTC) is a supervised learning method that breaks down a complex decision-making process into a collection of simpler, sequential decisions. It structures the classification problem as a tree where internal nodes represent logical tests and leaf nodes (nodes with no descendants) represent class labels[36]. This classifier functions as a logical filter, and it is hierarchical and interpretable. This means that unlike "black box" models such as deep learning models (CNN, for example) that we are unable to discern the features by which they classify, the DTC ensures that we understand exactly why a classification is made.
This architecture offers computational efficiency, as its complexity is $O(depth)$, where depth describes the number of levels in the DTC. This translates to a relatively low number of scalar comparisons, which is much simpler than the complexity of heavy neural network calculations, which include matrix multiplication and sometimes thousands of parameters. Furthermore, recent studies[37] have shown that DTC can achieve high classification accuracy for fire detection - over 90% scores in accuracy, F1-score and precision.

Mathematical Formulation

We will define the classifier as a piecewise function C(b) leaning on features covered in section 5.2.1 **Feature Hypothesis Generation**, specifically, Maximum Blob Size & Maximum Temperature, to be denoted $A\ (Area)$ & $T\ (Max\ Temp)$ respectively, in this section.
The aim is to differentiate between an ambiguous threat (is it a proper hot object or a growing flame?), immediate threat (an ignition source) and a certain non-issue.

[36] Safavian, S. R., & Landgrebe, D. "A survey of decision tree classifier methodology", (1991).
[37] B. Haranadh Reddy, Karthikeyan P R, "Classification of Fire and Smoke Images using Decision Tree Algorithm in Comparison with Logistic Regression to Measure Accuracy, Precision, Recall, F-score", (2022).

The function is formulated as:

$$C(b) = \begin{cases} Ignition\ Source & , \quad if\ (T_{max} > T_{ign}) \cap (A < A_{limit}) \quad \rightarrow Alert \\ Potential\ Fire & , \quad if\ (T_{max} > T_{fire}) \cap (A \geq A_{limit}) \quad \rightarrow Verify\ Growth \\ Safe & , \quad Otherwise \end{cases}$$

Where B are the sets of distinct blobs identified in the binary mask in the previous stage, T_{max} is the maximum pixel intensity within the blob. T_{ign} is the lower threshold for a small, concentrated heat source, which has been stated before will likely register at a relatively low temperature due to averaging (roughly 45°C). T_{fire} is the higher cutoff used for larger, clearer heat sources (e.g. 60°C). A is the spatial area of the blob, that is the total pixel count. A_{limit} would be the discriminator, a spatial cutoff to distinguish between concentrated sources and undetermined objects (actual fires\person\electronics).
The classification of an *Ignition Source* would trigger an immediate alert, as no benign object in a hospital room should be both minute ($< A_{limit}$) and hot ($> T_{ign}$). This logic captures high-intensity, small-scale hazards such as socket failures, as well as unauthorized items such as lighters and cigarettes.
The Classification of a *Potential Fire* would represent large, hot objects. Since these attributes could belong to either a spreading fire or a static device, this triggers further analysis rather than an immediate alarm to minimize false positives.

Algorithm Flow:

a) Feature Extraction - For each blob in the binary mask, calculate the Area (A) and Max Temp (T_{max})
b) Gating - Discard any blobs where $T_{max} < T_{ign}$ (human\background)
c) Classification - Apply the piecewise function C(b). If "*Ignition Source*", alert immediately. If "*Potential Fire*", pass the object to the next step (Growth Rate tracker).

3) Temporal Mass Gradient Analysis (Growth Rate)

The final layer of the pipeline approach is triggered when the DTC identifies a "Potential Fire" candidate - a thermal anomaly that is both hot ($T_{max} > T_{fire}$) and large ($A \geq A_{limit}$). Its function is to resolve the ambiguity between *Static Heat Sources* and a spreading fire - an *Active Combustion*.

The distinction lies in the physical behavior of the source over time. As described in the fire detection article by Celik et al.[38], combustion is a dynamic process characterized by rapid release of energy and spatial expansion. In contrast, *Static Heat Sources* maintain relatively constant thermal footprints over short time intervals. Therefore, looking at a single frame is insufficient. Furthermore, a combustion's growth rate might not be strictly linear, so demanding a strict growth over *consecutive* frames might lead to false negatives.

To differentiate between these cases, we analyze the Temporal Mass Gradient - the rate of change of the suspected object's spatial area ($\frac{dA}{dt}$) over a short sliding window, and through it- the net growth trend. An *Active Combustion* will exhibit a positive gradient as it grows, and a *Static Heat Source* should exhibit a near-zero gradient.

Mathematical Formulation

The gradient analysis is initiated only when the DTC flags a tracked anomaly as a *Potential Fire*.

We define A_n as the spatial area A (pixel count) of the tracked anomaly at the current frame n. To evaluate expansion, we will use a sliding window approach defined by the following parameters.

1. Window Interval (Δt): The look-back period for calculating size difference.
2. Frame Lag (k): The discrete difference between frames, corresponding to the window interval and frame sample frequency- $k = \lfloor \Delta t \cdot f_s \rfloor$

The discrete temporal gradient G_n will be calculated by comparing the area of the object between the current frame n against the frame $n - k$

$$G_n = \frac{A_n - A_{n-k}}{\Delta t}$$

Given that the **current** working sampling rate is $f_s = 8\ Hz$ (we expect to increase value), we select a window lag of $k = 8$ frames to define a physical time interval of $\Delta t = 1\ sec$. With that said, Δt, as well as k, remain hyper-parameters to be tuned and reassessed as we move to implementation phases.

The time interval of a 1-second window is a temporary selection stems from a critical tradeoff between sensitivity and robustness:

1. Noise Suppression: Fire is turbulent. If the window interval is too short (e.g. 100ms), the natural high-frequency flickering of a flame might result in momentary shrinking and thus a negative gradient, causing false negatives. A longer time interval acts as a low-pass filter, ignoring these fluctuations and allowing focus on the underlying expansion trend.
2. Measurable Margin & Early Warning: Considering the low resolution we are working with, the growth of a fire is significant but not instantaneously apparent. A very short

[38] Celik T, Demirel H, Ozkaramanli H, Uyguroglu M., "Fire detection using statistical color model in video sequences", (2007).

time interval might show a growth rate of near-zero, appearing static. This would potentially result in false negatives as well. A large enough Δt window ensures that a flame would have had enough time to expand by a measurable margin (registering new pixels as part of the blob) if it is indeed spreading. It is of course of the highest priority not to delay detection. Therefore, extending the window interval too much (e.g. 10 seconds) would pose a critical safety concern and a major risk.

We define a persistence filter to confirm the growth trend in the form of a counter S. For this we define:

1. ϵ_{growth} - The minimal pixel difference to register as potential expansion. Defined to take thermal noise into consideration and allow scaling for higher spatial resolutions. Given the low spatial resolution of the current sensor, we set this to 1 pixel.
2. τ_{step} - The interval between gradient calculations. To keep indices consistent, this will be described by the frame difference and is currently chosen to be 4 frames, corresponding to 0.5 seconds. This is a separate consideration from the window interval (Δt).
3. $T_{measure}$ - The maximum measuring period allowed between a flag raised by the DTC and reaching a decision.
4. $S_{threshold}$ – The minimal accumulated score over the measuring period to confidently classify as *Active Combustion* and send an alert, to be derived from T & τ .

$$S_n = \begin{cases} S_{n-1} + 1 & if \quad G_n > \epsilon \quad (Growth\ Detected) \\ S_{n-1} - 1 & if \quad G_n \leq \epsilon \quad (Static \backslash Shrinking) \end{cases}$$

If the gradient is positive and surpasses the minimal difference, we increment the counter, signifying a "growth frame". Otherwise , we decrement the counter, signifying a "static\shrinking frame". The system triggers an alarm only if this counter exceeds a pre-defined threshold $S_{threshold}$, confirming that the expansion is not a transient artifact.

$$Decision = \begin{cases} Active\ Combustion \rightarrow Alert & if\ S_n \geq S_{threshold} \\ Safe \rightarrow No\ alert & otherwise \end{cases}$$

Algorithm Flow

a) Initialization - The process begins only when the DTC flags a specific object as a *Potential Fire*. The system initializes a "history buffer" of length $f_s \cdot T_{measure}$ to store this object's area values over the entire measuring period.
b) Buffer Update – Retrieve the object's Area A_n for every frame and append it. If less than Δt seconds have passed, no gradient can be calculated yet and the system waits.
c) Gradient Calculation – For every τ_{step} frames, retrieve the current and old Area A_n & A_{n-k} and calculate the growth gradient.
d) Persistence Check - If Area is growing ($G_n > \epsilon$), increment the growth counter. If Area is stable or shrinking ($G_n \leq \epsilon$), decrement the counter (minimum value 0).
e) Final Decision -Once $S_{threshold}$ has been surpassed, the object is validated as *Active Combustion*, the classification is elevated from *Potential Fire* to *Immediate Threat* and triggers an alert. If $T_{measure}$ seconds have passed and the condition hasn't been

met, the object is identified as a *Static Heat Source* and classified as *Safe*. Once a classification is made, the counter is reset and tracking is ceased.

This concludes the pipelined approach for fire detection.

While the primary metric for algorithm selection is detection reliability, we must also consider the system's operational constraints, as we aim to process data from three synchronized sensors in real-time. Given that fire detection is semantically simpler than human behavior analysis, allocating heavy Deep Learning resources (such as CNN-based detection) to this task would introduce unnecessary computational overhead, potentially bottlenecking the more complex contact detection algorithms.

Therefore, we propose a feature-based machine learning approach as a lightweight alternative.

Feature-Based Machine Learning Approach

The core of this approach lies in transforming the raw thermal blobs into meaningful mathematical descriptors that can be extracted from the data, as described in the EDA (Section 5.2). Based on our EDA findings, our models will be trained on a candidate pool of radiometric and statistical features. These include but not limited to:

- Intensity Descriptors: Such as Maximum Temperature and Mean Intensity
- Statistical Moments: Several higher-order metrics like Variance, Skewness and Kurtosis.

- **SVM (Support Vector Machine)**
  The ML approach for fire detection utilizes a Support Vector Machine, a supervised learning model designed for binary classification (Fire vs. Non-Fire). The rule-based pipeline, which relies on statistical properties of the current frame and immediate past frames alone, does not 'learn' from historical data. In contrast, the SVM operates by finding the optimal decision boundary (hyperplane) in an N-dimensional feature space, derived from the training data. This allows the system to identify complex, non-linear relationships between features that simple thresholding might miss.
  This approach is particularly suitable for fire detection because fire signatures (such as specific color distributions, growth rates and flickering frequencies), can be represented as a feature vector. The SVM analyzes these vectors to construct a hyperplane that maximizes the 'margin' (the distance) between positive samples (Fire) and negative samples (Non-Fire), ensuring the system effectively generalizes to new, unseen environments rather than just memorizing the training data.
  <u>Mathematical Formulation:</u>
  The goal of the SVM is to find an optimal separating hyperplane that maximizes the margin (the geometric distance) between the "Fire" ($y = +1$) and "Non-Fire" ($y = -1$) classes.
  Let the input data be represented as feature vectors $x \in \mathbb{R}^n$. Since fire signatures may not be linearly separable in the raw feature space, we first define a mapping function $\phi(\cdot)$ that projects the input vectors into a higher-dimensional feature space:
  $$x \rightarrow \phi(x)$$
  We define the decision boundary such that for each training sample x_i, the following conditions must hold:
  - For Fire samples $y_i = +1$:
    $$w^T \phi(x_i) + b \geq +1$$
  - For Non-Fire samples $y_i = -1$:

$$w^T\phi(x_i) + b \leq -1$$

These inequalities can be combined into a single constraint for all i:

$$y_i(w^T\phi(x_i) + b) \geq +1$$

The decision function for a new input x predicts the class $\hat{y}$ based on which side of the margin it falls:

$$\hat{y} = sign(w^T\phi(x_i) + b)$$

This formulation ensures that the margin between the classes has a width of $\frac{2}{||w||}$. The optimization goal is to minimize $||w||$ thus maximizing the width, while satisfying the constraints above.

While this is straightforward enough, as we operate in an N-dimensional space, where N is the size of the feature vector, the data might not be separable by a straight line\flat plane in its original form. To avoid the computational cost of explicitly calculating the high-dimensional mapping $\phi(x)$, we employ the Kernel Trick. The kernel function K computes the inner product directly in the original low-dimensional space:

$$K(x_i, x_j) = \phi(x_i)^T\phi(x_j)$$

Common kernel candidates for this implementation include the Linear Kernel (for computational efficiency) or the Radial Basis Function (for complex non-linear distributions). The specific kernel selection will be determined during the model training phase.

Algorithm Flow

1. Region Proposal: The system identifies candidate regions (blobs) using the initial motion and color detection stages.
2. Feature Extraction: For each candidate region of interest (ROI), the N features defined in the EDA are computed. These features form the input vector x
3. Standardization: The vector is normalized using pre-computed mean (μ) and standard deviation (σ) values derived from the training set-
   $\hat{x} = \frac{x - \mu}{\sigma}$
   This is crucial for preventing disproportionate values to dominate features with with small ranges, which might cause biasing.
4. Classification: The system computes the decision function score using the trained model parameters (w, b) and the selected kernel function K.
5. Decision Logic:

$$if\ \hat{y} = +1 \rightarrow Classify\ as\ Fire\ (Alert)$$
$$if\ \hat{y} = -1 \rightarrow Classify\ as\ Non - Fire\ (No\ Alert)$$

5 Preliminary Results and Analysis

5.1 Prototype Configuration and Testing

5.1.1 Test Environment

To validate the system's detection capabilities in realistic thermal conditions, the preliminary dataset was collected in a residential environment (indoor living room) rather than a sterile laboratory. This ensured the presence of natural background thermal noise (furniture, ambient air currents) representative of a real-world scenario.

Ambient Conditions: Tests were conducted at an ambient temperature of approximately 17°C. This provides a fair thermal contrast against the human body temperature ($\Delta T \cong 20°C$). Data was collected over two separate occasions, both during evening time, so the room was artificially lit.

Sensor Geometry: Sensors were positioned at chest height (approx. 1.4m), leveled parallel to the floor. Due to the lack of a final enclosure, sensors were hand-stabilized or statically mounted using temporary fixtures.

5.1.2 Data Acquisition Software

A custom Python-based GUI was developed for the acquisition. The software provides real-time feedback (video stream) for all three sensors, logs the data as raw CSV thermal matrices and parsed .mp4 video files, and allows for immediate 3/5/7 second recordings or manual star/stop selection.

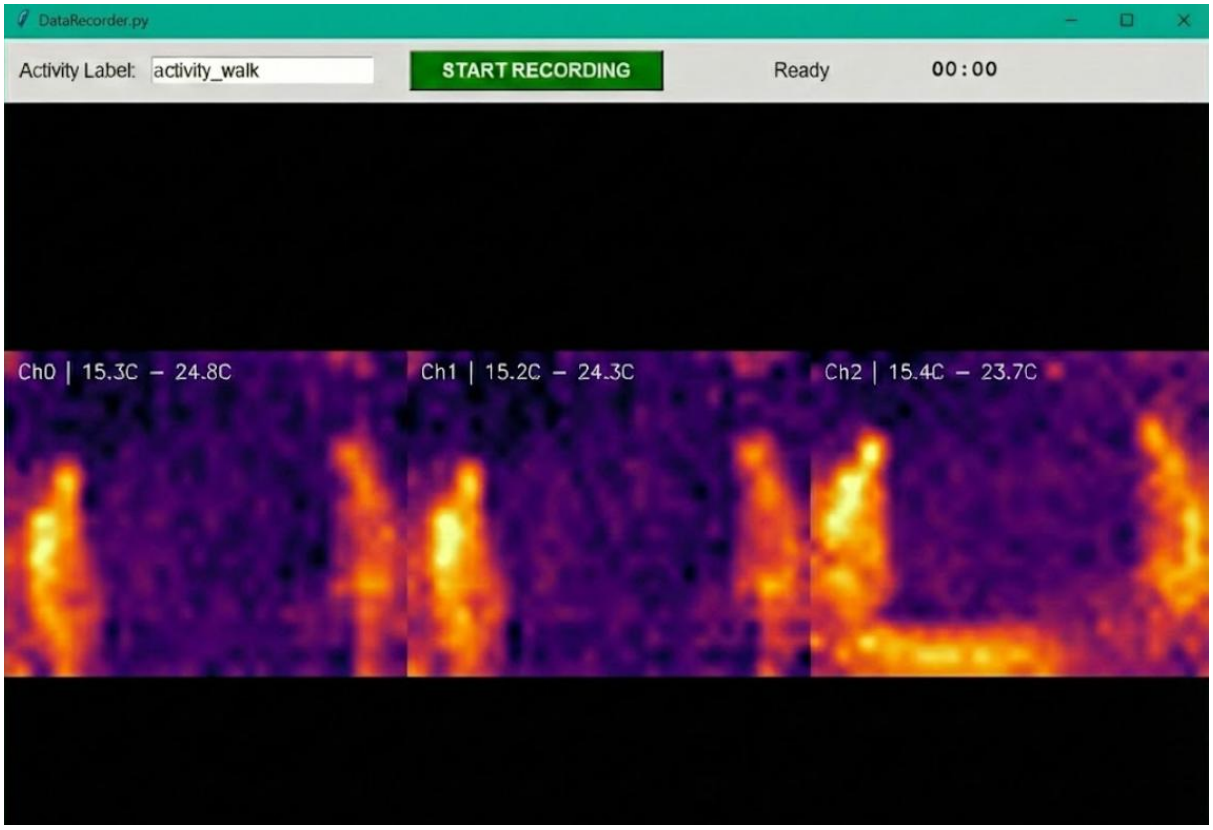


Figure 12: Custom Data Acquisition GUI showing real-time sensor feeds.

5.1.3 Test Scenarios

Data collection was structured into three distinct detection categories, with an added baseline recording. The background environment was held constant to isolate the subject signatures.

Baseline: Empty room (Background noise calibration).

Thermal Anomaly (Fire): A controlled flame (lighter) triggered by an operator just outside the Field of View (FOV), ensuring only the thermal signature of the flame and a partial hand were visible.

Human Presence: Single subjects performing standard movements: static standing, walking laterally across the FOV, and vertical motion (jumping).

Multi-Person Interaction: Two subjects engaging in non-violent proximity (standing close) and contact (hugging).

A photograph of the assembled prototype is featured.

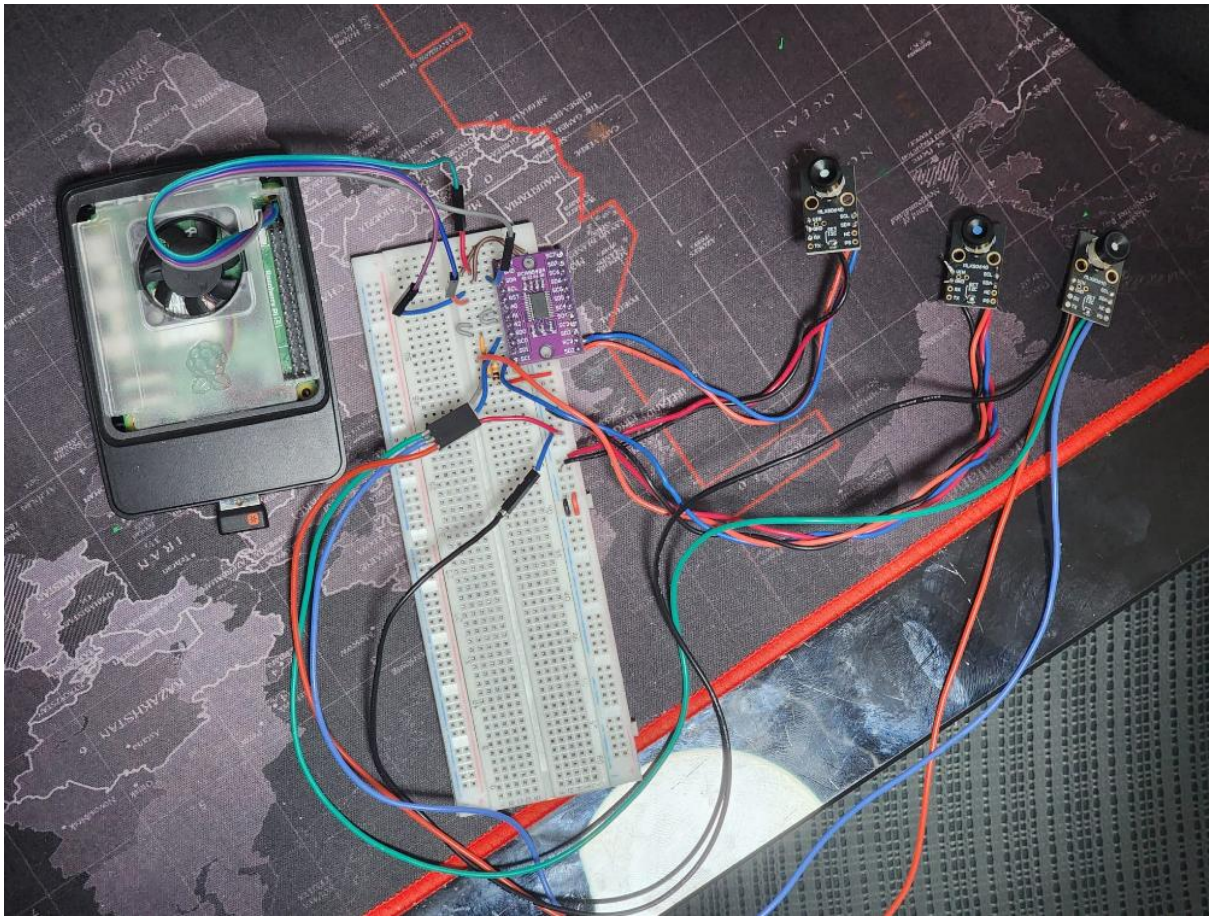


Figure 13: The assembled prototype

5.2 Exploratory Data Analysis

Following data acquisition, we prioritized defining interpretable thermal features that could characterize our three anomaly classes. First, we verified the integrity of the dataset by cross-referencing the loaded frame counts with our experimental sequence logs:

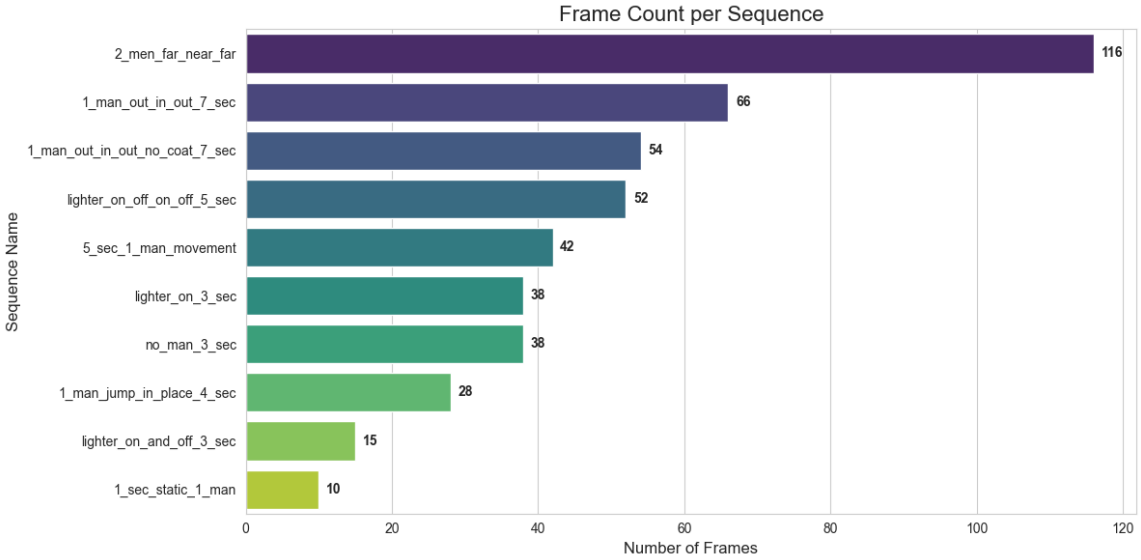


Figure 14: Frame Count per Sequence

5.2.1 Feature Hypothesis Generation

We postulated that specific physical properties would manifest as distinct statistical features in the thermal domain:

- **Maximum Temperature:** We hypothesized that fire would exhibit significantly higher peak pixel intensities than human subjects.
- **Maximal Blob Size:** We expected the thermal signature of a human to occupy a larger spatial area than that of a small fire source (e.g., a lighter).
- **Blob Geometry:** To distinguish 'Human' events, we analyzed the **Width-Height Ratio** of a blob, assuming that a human would be slimmer and taller than a fire.
- **Statistical Moments:** We anticipated that fire would display a distinct **Mean-Variance signature** (low mean, high variance) compared to the more uniform heat distribution of a human body.
- **HoG (Histogram of Gradients):** HoG is a famous feature that allows us to describe texture and edges in an image and might even show some differences between the 'Human' and 'Touch' classes.
- **Skewness and Kurtosis:** We expect that skewness will be reduced when two hot blobs merge (since the ratio of "hot" to "cold" pixels changes). We expect an image with multiple objects to reduce kurtosis, which means there are many extreme "outliers", when compared to a sparse image.

5.2.2 Dynamic Range & Noise Analysis

An initial analysis of the sensor's dynamic range revealed a global minimum of:

$$Temp_{min} = 5.7915\ C, \qquad Temp_{max} = 76.4642\ C$$

However, visual inspection of the raw histograms (see chapter 7.3.1) revealed that these extremes were often flanked by a substantial noise floor. The raw pixel distribution (gray histogram) showed a significant overlap between background noise and potential signals.

To mitigate this and ensure our proposed features could be extracted reliably, we implemented a preprocessing pipeline inspired by S. Tateno et al.,[39] designed to suppress environmental noise and enhance feature distinctness (detailed in Chapter 6.4.1).

5.2.3 Feature Analysis – Maximum Temperature

To evaluate the discriminative power of thermal intensity, we plotted the distribution of the maximum pixel value per frame across all labeled classes. We sought to verify if fire incidents exhibit a statistically distinct peak temperature range compared to human subjects, which would allow for robust threshold-based detection.

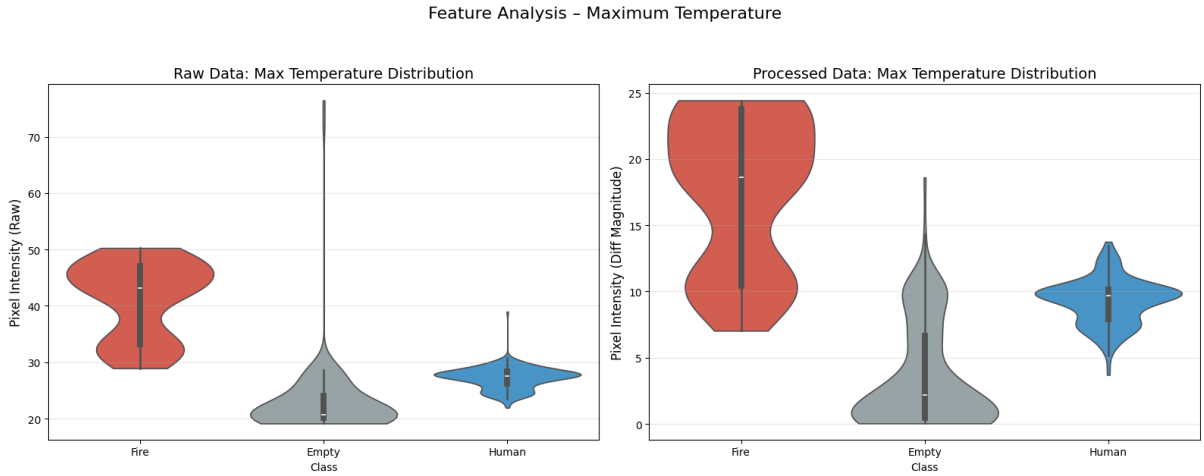


Figure 15: Feature Analysis - Maximum Temperature

The comparative violin plots demonstrate a marked improvement in signal separability after preprocessing. In the raw domain (left), the 'Empty' class distribution heavily overlaps with 'Human' and 'Fire' signals due to the high noise floor, making reliable detection difficult. In the processed domain (right), the 'Empty' class is effectively suppressed to near-zero, proving the pipeline's efficacy as a robust activity detector.

However, a significant ambiguity remains. While high-intensity outliers (> 15) are exclusively indicative of Fire, the 'Human' distribution (centered ≈ 10) is almost entirely contained within the lower quartile of the 'Fire' distribution. This indicates that thermal intensity alone cannot distinguish a human from a small fire (e.g., a lighter). Consequently, this motivates the need for spatial analysis—specifically Maximal Blob Size—to differentiate between these two classes based on their physical footprint rather than just their peak temperature.

[39] Tateno, S. et al., (2020), Privacy-preserved fall detection method with three-dimensional convolutional neural network using low-resolution infrared array sensor, Sensors

5.2.4 Feature Analysis – Blob Maximal Size

Next, we analyzed the spatial extent of the thermal anomalies to validate the hypothesis that a human body's thermal footprint is consistently larger than smaller, concentrated heat sources such as a lighter. To establish this ground truth, we calculated the area (in pixels) of the manually labeled bounding boxes for each class.

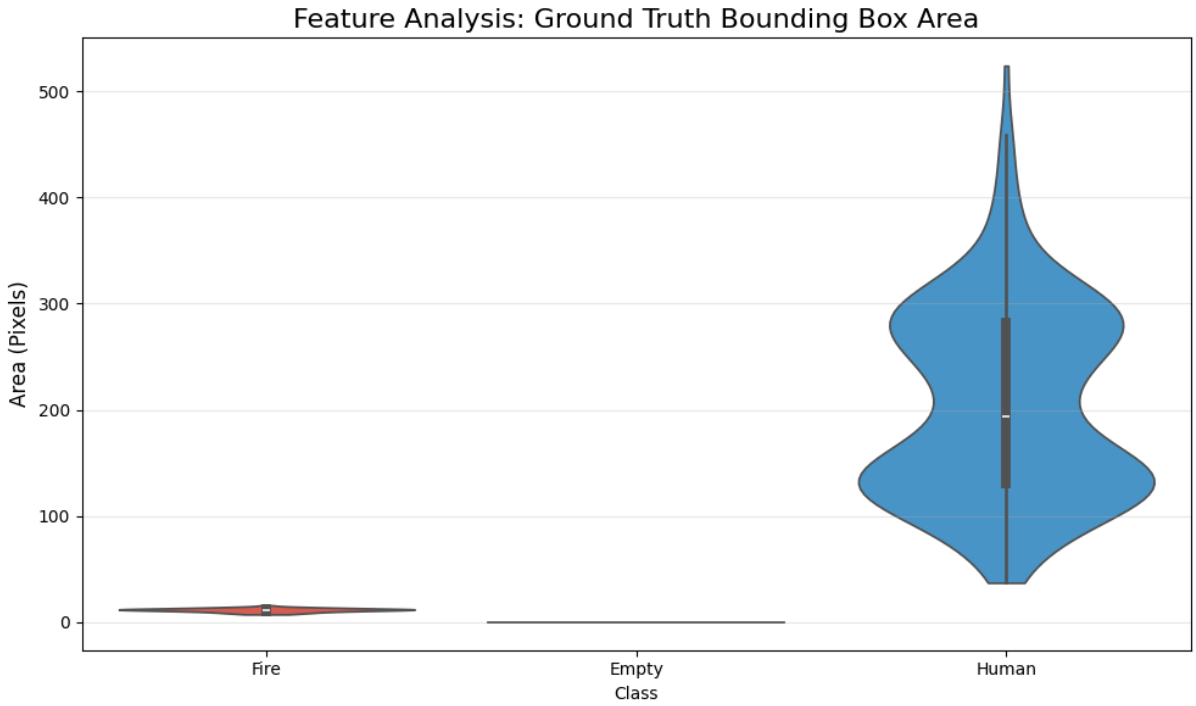


Figure 16: Blob Area Visualization

As observed in the violin plot above, the Human category exhibits a significantly larger spatial area compared to the Fire and Empty categories, with minimal overlap. This distinct separation demonstrates that spatial area is a robust discriminative feature for classifying these thermal sources.

5.2.5 Feature Analysis – Width-Height Ratio

We further characterized the geometry of the detections by computing the bounding box width and height for the primary thermal blob. This analysis tests the assumption that a standing human presents a vertically oriented profile (meaning height > width), whereas the combined shape of two subjects touching creates a wider, more horizontal distribution.

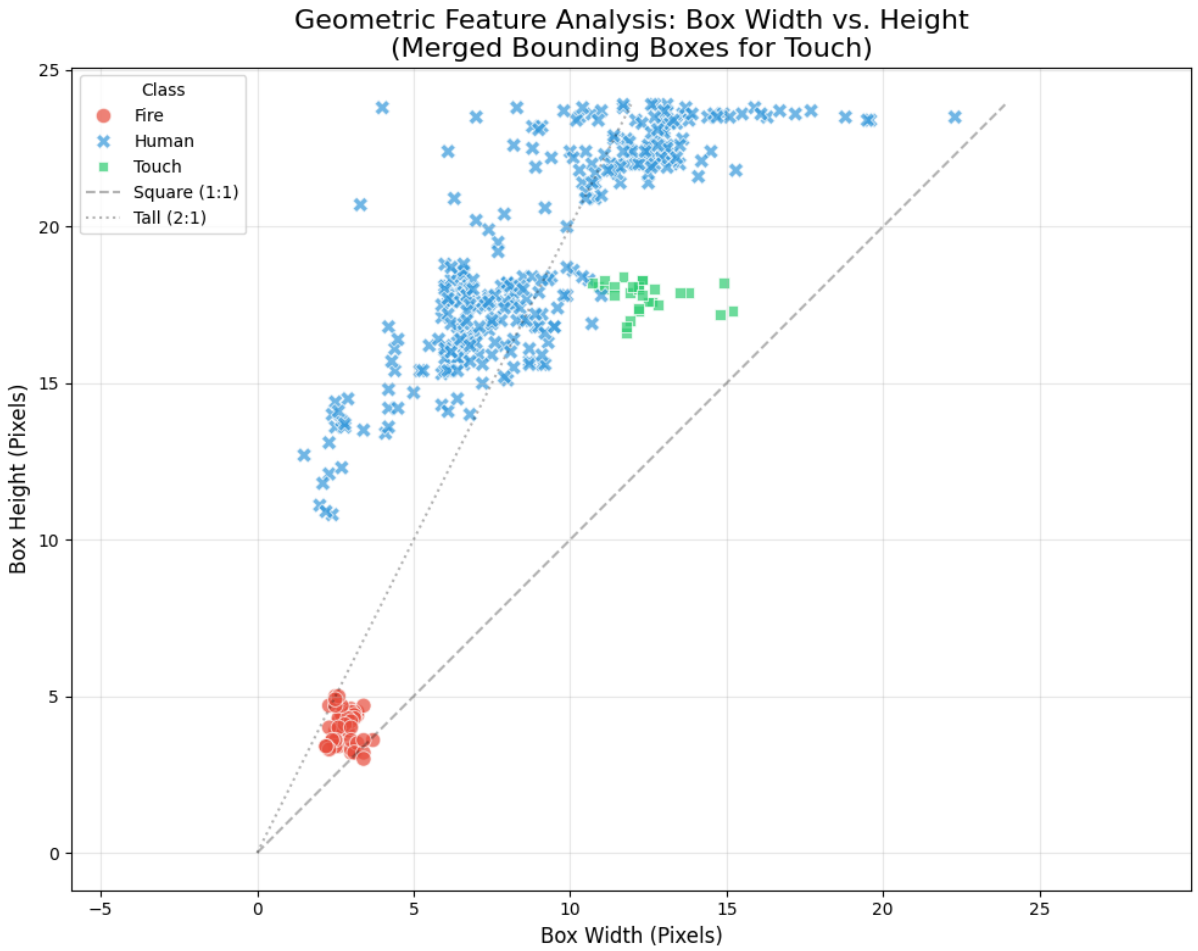


Figure 17: Bounding Box Width vs. Height

By merging the bounding boxes for "Touch" interactions to capture the full footprint of the event, the analysis now strongly validates our geometric hypothesis. As seen in the updated scatter plot, the Touch samples (green) have shifted significantly along the horizontal axis, clustering between 10–15 pixels in width. This forms a distinct group separated from the narrower Human distribution (blue), which typically remains under 10 pixels in width. This shift demonstrates that the combined profile of interacting subjects presents a much wider aspect ratio—approaching 1:1 or even wider—compared to the vertically oriented (approx. 2:1) profile of a single standing human. Meanwhile, the Fire class remains isolated near the origin, confirming that geometric features can now effectively discriminate between all three categories.

5.2.6 Feature Analysis – Mean and Standard Deviation

We investigated the global pixel intensity statistics—specifically the mean and standard deviation—within the region of interest. We anticipated that fire would manifest as a high variance signal due to its small but intense hotspot, contrasting with the more uniform and widely distributed thermal signature of a human subject.

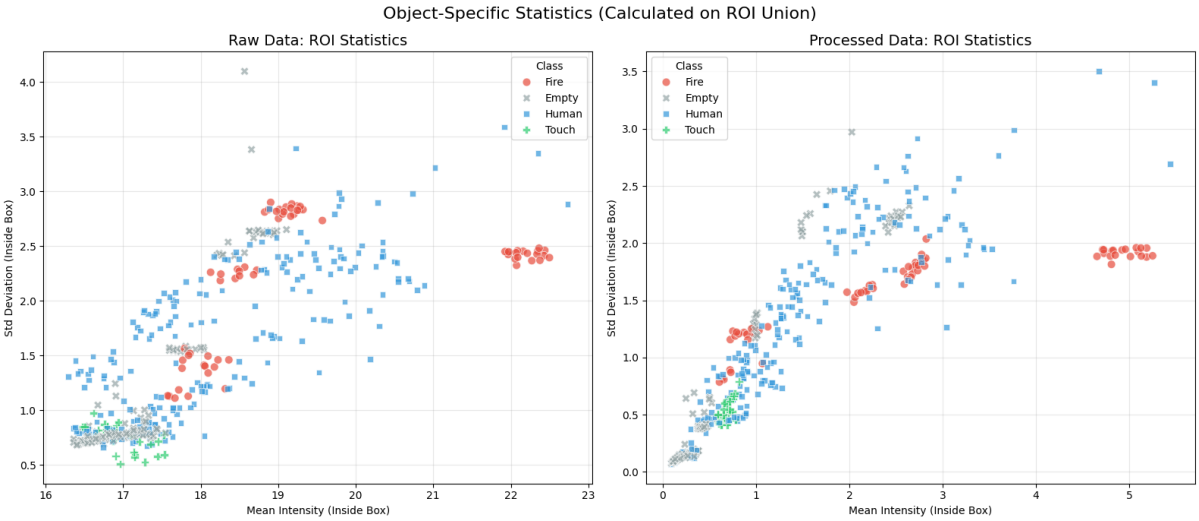


Figure 18: Mean and Standard Devation Visualization

Here, we can see that even trying to look only at the ROI (region-of-interest), the mean and standard deviation aren't a good way to differentiate between the categories, even though we can see the effect of the pre-processing pipeline on the data: more images are closely aligned to a linear straight line that can be fit between them, while in the raw data, the frames are more scattered and chaotic.

5.2.7 Feature Analysis – Histogram of Gradients

HoG is a feature descriptor widely used in computer vision for object detection. It works by dividing the image into small spatial cells and computing a histogram of gradient directions (edge orientations) for the pixels within each cell. The result is a feature vector that effectively encodes the "shape" and "structure" of the object while being relatively invariant to local illumination changes.

We hypothesize that a single standing person will exhibit a distinct gradient signature dominated by vertical edges (representing the torso and legs). in contrast, a "Contact" event (two people touching or hugging) should produce a more complex gradient distribution with significant horizontal or diagonal components where the two shapes merge.

A standard HoG feature vector for our image resolution contains thousands of dimensions, making it impossible to visualize directly. To analyze the separability of our classes, we utilized Principal Component Analysis (PCA). PCA is a statistical technique that transforms high-dimensional data into a lower-dimensional space (e.g., 2D) while preserving as much variance (information) as possible. By projecting our dataset onto the two principal components, we can visually inspect if "Contact" and "No Contact" samples form distinct clusters.

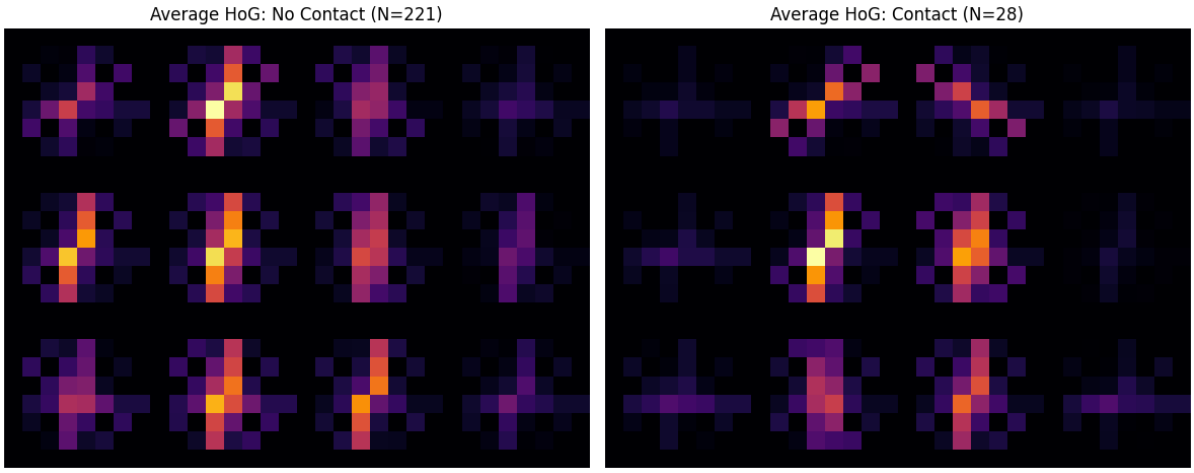


Figure 19: Average HoG of Contact vs. No Contact

The "Average HoG" visualizations reveal a clear structural distinction between the classes:

- **No Contact (Left):** The average representation shows a strong, vertical "pillar" of energy. The gradients are highly organized, reflecting the distinct vertical edges of single, standing subjects. The background regions are effectively suppressed, indicating that our preprocessing pipeline successfully removed thermal noise.
- **Contact (Right):** The averaged representation for contact events is significantly wider and more chaotic. The strong vertical definition is replaced by a complex "X" pattern with substantial diagonal and horizontal energy. This confirms that the geometric merger of two thermal blobs fundamentally alters the gradient landscape.

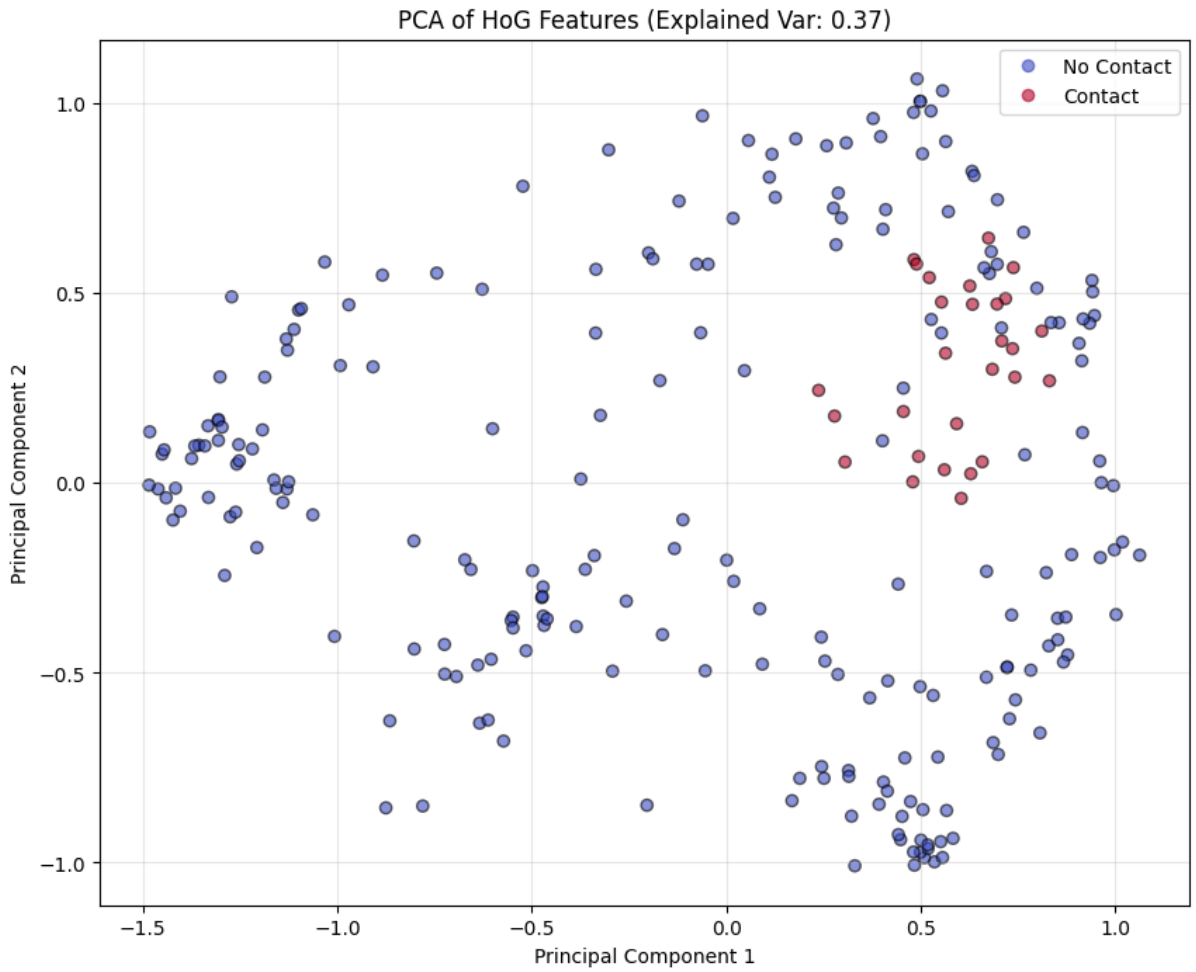


Figure 20: PCA of HoG Features

The PCA scatter plot demonstrates that these visual differences translate into a separable feature space:

- The "Contact" samples (Red) are not randomly distributed; they form a cohesive cluster in the upper-right quadrant of the latent space ($PC1 > 0.25$). This indicates that contact events share a specific, mathematical "shape signature."
- While the contact cluster is distinct, it is not perfectly isolated. The "No Contact" samples (Blue) are spread widely, with some samples overlapping the contact region. This suggests that while HoG is a powerful discriminator for complex shapes, it may struggle to distinguish a "wide" single person (e.g., arms outstretched) from two merged people. This finding motivates the inclusion of additional shape-specific descriptors, such as Hu Moments, to resolve these edge cases.

5.2.8 Feature Analysis – Skewness & Kurtosis

To determine if global thermal intensity distributions could serve as discriminative features, we analyzed the **Skewness** (measure of asymmetry) and **Kurtosis** (measure of tailedness) of the image histograms. We hypothesized that the merger of two warm bodies might alter the ratio of "foreground" to "background" pixels sufficiently to distinguish contact events from single subjects.

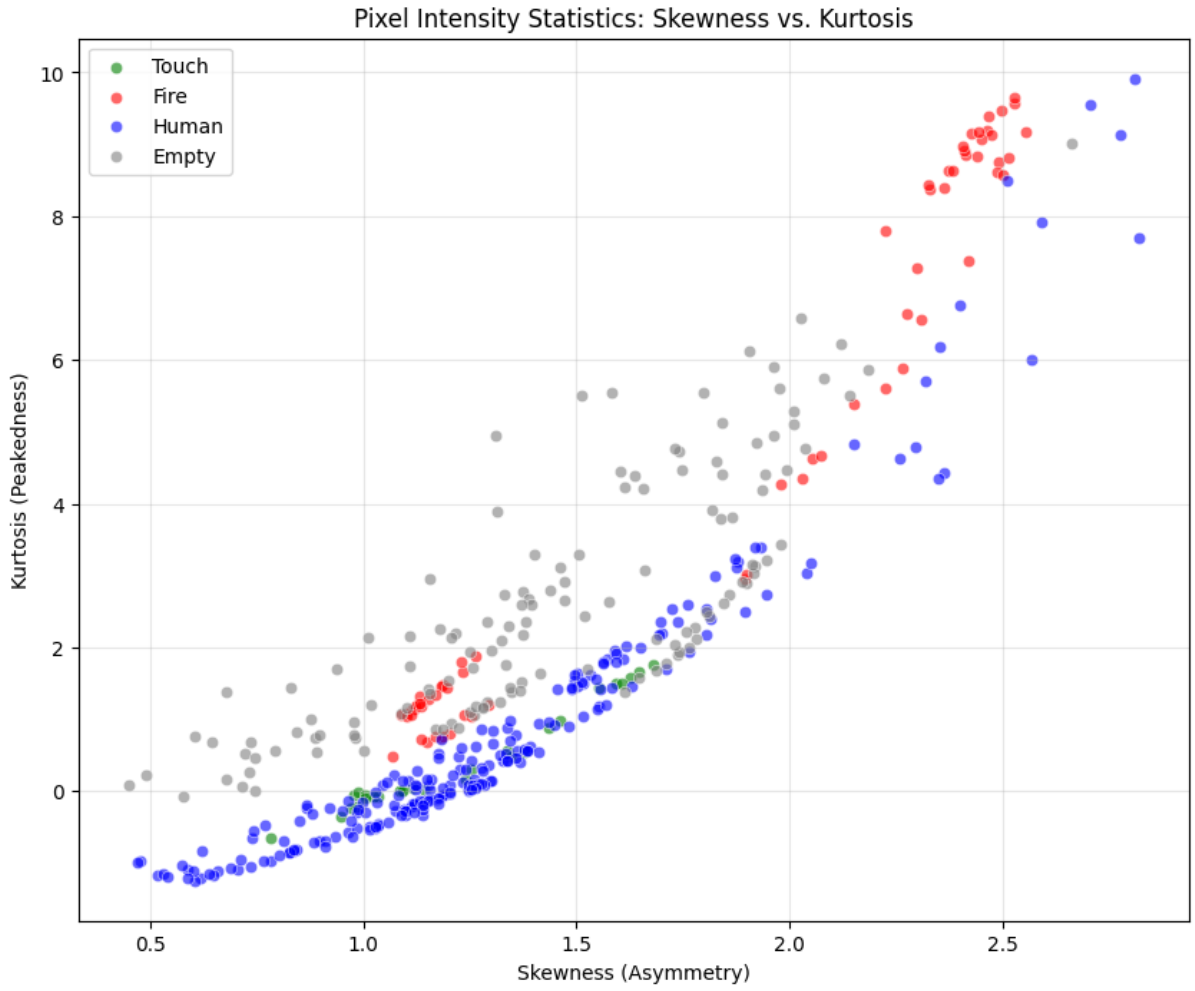


Figure 212: Skewness and Kurtosis Visualization

Here, the "Fire" class (Red) generally occupies the upper-right quadrant, characterized by high skewness and kurtosis. This trend aligns with the physical properties of fire as an intense, small-area thermal anomaly. However, separability is not absolute; a subset of fire samples in the range of $1.0 < Skewness < 1.5$ and $0 < Kurtosis < 2$ is more closely aligned with the "Human" and "Touch" clusters. This indicates that while a decision boundary between fire and humans exists, it is likely **non-linear**. Consequently, simple linear thresholding is insufficient to perfectly filter thermal noise without potentially discarding valid human detections.

It can also be seen that the "Touch" samples (Green) are completely embedded within the "Human" distribution (Blue). Both classes share identical statistical ranges for pixel intensity. This confirms that global histogram statistics discard the spatial information necessary to distinguish two separate individuals from a single merged shape.

5.2.9 Exploratory Data Analysis – Summary

This section presented a comprehensive analysis of the thermal dataset to determine the viability of classical feature engineering for contact detection. We evaluated a diverse set of radiometric, geometric, and statistical descriptors to characterize the thermal signatures of human subjects and contact events.

- **Radiometric & Statistical Analysis:**
  - **Dynamic Range & Noise:** We assessed the sensor's dynamic range and noise floor, confirming the need for robust background subtraction to isolate valid thermal targets from environmental static.
  - **Maximum Temperature:** Peak temperature analysis proved effective for distinguishing high-intensity non-human anomalies (e.g., fire) but lacked the resolution to differentiate individual humans from interacting pairs.
  - **Pixel Intensity Statistics:** Global histogram descriptors, specifically Skewness and Kurtosis, showed some correlation with extreme thermal events but failed to separate the "Contact" class from the "Human" class due to nearly identical intensity distributions.
  - **Mean & Standard Deviation:** Basic statistical moments provided insight into overall scene energy but were insufficient for fine-grained classification.
- **Geometric & Structural Analysis:**
  - **Blob Maximal Size & Aspect Ratio:** While contact events theoretically exhibit larger bounding boxes and higher width-to-height ratios, analysis revealed significant overlap with single subjects in specific poses (e.g., outstretched limbs), rendering simple dimensional thresholds unreliable.
  - **Histogram of Oriented Gradients (HoG):** Dimensionality reduction via PCA on HoG features demonstrated that while contact events possess a distinct gradient signature (complex, multi-directional edges), the class boundary is not linearly separable from single-subject samples.

While specific features successfully segregated environmental hazards like fire, **none of the evaluated hand-crafted features provided a robust discriminative signal for Contact Classification.** The structural and geometric ambiguity between a single "wide" human and two interacting subjects cannot be resolved through linear combinations of classical features. This necessitates the adoption of a data-driven **Deep Learning** approach, capable of learning hierarchical, non-linear representations directly from the raw thermal data.

5.3 Algorithmic Performance Evaluation

5.3.1 Preprocessing Pipeline

To enhance feature visibility and suppress environmental noise, we implemented a three-stage preprocessing pipeline consisting of **Spatial Denoising**, **Baseline Subtraction**, and **Residual Rectification**. We utilized a standard Gaussian smoothing filter ($3x3$ kernel) to reduce high-frequency speckle noise and improve the spatial coherence of the target. This was followed by subtracting a static background reference to filter out environmental artifacts, using the absolute difference (L1 norm) to capture the magnitude of all thermal/radar anomalies regardless of polarity.

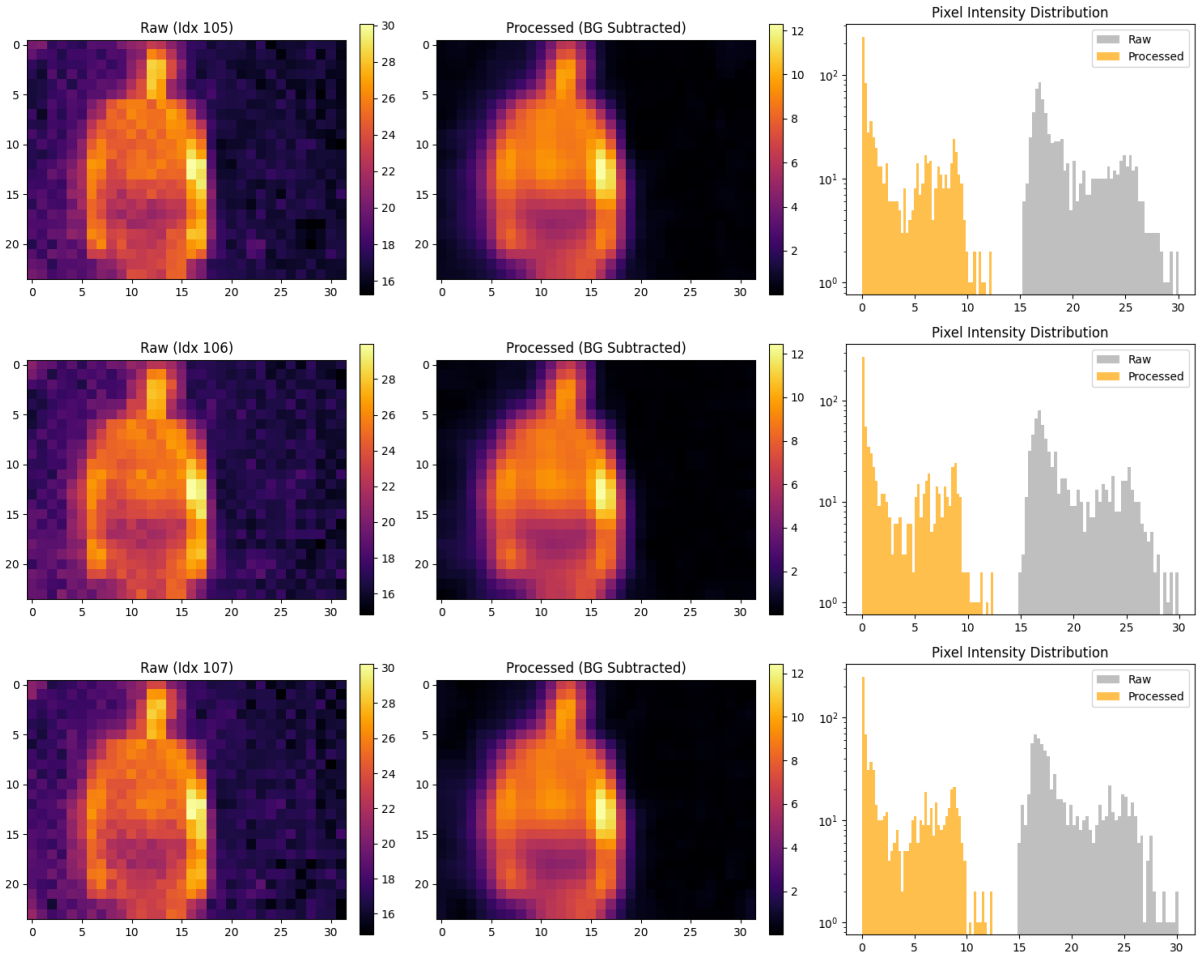


Figure 22: Preprocessing results and histograms

The effectiveness of this pipeline was validated both subjectively and quantitatively. Visual inspection confirms that the processed frames exhibit a significantly cleaner background with the human subject isolated as a distinct, coherent region. Mathematically, the pipeline achieved a **5.61x improvement in Signal-to-Background Ratio (SBR)**, raising the average SBR from **1.21** in the raw data to **6.80** in the processed output. This substantial gain in contrast confirms that the pipeline successfully amplifies the target signal relative to background clutter, providing a robust foundation for subsequent exploratory data analysis and model training.

5.3.2 Human Detection

To evaluate the robustness of the human detection system, the **Adaptive Gaussian thresholding algorithm** was implemented and benchmarked across six distinct experimental scenarios from the 'man_presence_easy' dataset, which includes a total of 316 frames. The primary objectives were to quantify the performance uplift provided by the Preprocessing Pipeline, as well as to test the robustness and success of the algorithm itself.

Performance Summary and Metrics

The comparative analysis utilized a **confusion matrix**, out of which we derived four statistical metrics: **Accuracy (Acc)**, **Precision (Prec)**, **Recall (Rec)**, and **F1-Score**. Additionally, the **Correct/Total** metric was included to provide the raw frame count of successful classifications for each experiment:

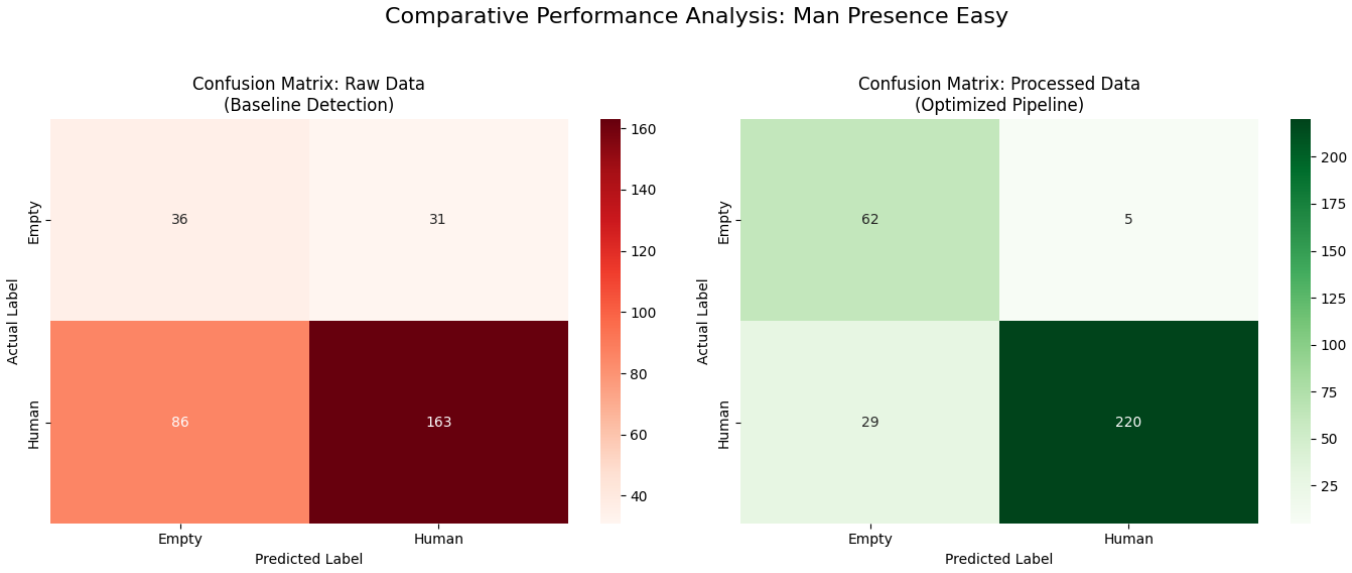


Figure 23: Confusion matrices - Adaptive Gaussian Thresholding, raw vs preprocessed data

The confusion matrices provide a high-level view of how the system's reliability is scaled with preprocessing.

- **Raw Data (Baseline Detection):** The red matrix reveals significant classification errors. Specifically, 86 False Negatives (Actual Human predicted as Empty) and 31 False Positives (Actual Empty predicted as Human) indicate that the raw signal is often too weak or too noisy for consistent detection.
- **Processed Data (Optimized Pipeline):** The green matrix shows a dramatic reduction in errors. False Negatives dropped from 86 to 29, and False Positives were nearly eliminated, falling from 31 to just 5. This confirms that the pipeline successfully "silences" background noise while amplifying the target heat signature.

Folder Name	Mode	Accuracy	Precision	Recall	F1	Correct/Total
1_man_jump_in_place_4_sec	Raw	100.0%	100.0%	100.0%	100.0%	28/28
	Proc	100.0%	100.0%	100.0%	100.0%	28/28
1_man_out_in_out_7_sec	Raw	63.6%	60.0%	67.7%	63.6%	42/66
	Proc	90.9%	100.0%	80.6%	89.3%	60/66
1_man_out_in_out_no_coat_7_sec	Raw	66.7%	55.3%	95.5%	70.0%	36/54
	Proc	88.9%	80.8%	95.5%	87.5%	48/54
1_sec_static_1_man	Raw	40.0%	100.0%	40.0%	57.1%	4/10
	Proc	70.0%	100.0%	70.0%	82.4%	7/10
2_men_far_near_far	Raw	58.6%	100.0%	58.6%	73.9%	68/116
	Proc	90.5%	100.0%	90.5%	95.0%	105/116
5_sec_1_man_movement	Raw	50.0%	100.0%	50.0%	66.7%	21/42
	Proc	81.0%	100.0%	81.0%	89.5%	34/42

Table 3: Human Detection - Algorithm Evaluation Table

Analysis of Results

The results validate that the preprocessing pipeline provides a robust foundation for subsequent data analysis by significantly amplifying the target signal relative to background clutter:

- **Precision and Background Suppression:** In the more complex scenarios like '1_man_out_in_out_7_sec', Raw mode Precision was limited to **60.0%** due to thermal artifacts. The Processed mode achieved 100.0% Precision, confirming that baseline subtraction effectively eliminated static environmental noise.
- **Sensitivity and Object Isolation:** The pipeline achieved a 5.61x improvement in Signal-to-Background Ratio (SBR). This is most evident in the '2_men_far_near_far experiment', where the total number of correctly identified frames jumped from 68/116 to 105/116, demonstrating higher sensitivity to distal targets.
- **F1-Score Reliability:** Across all experiments, the F1-Score in Processed mode significantly outperformed Raw mode. Even in short duration tests like 1_sec_static_1_man, the F1-score improved from 57.1% to 82.4%, indicating a much more balanced and dependable classification output.

Conclusion

The algorithmic evaluation proves that while the adaptive Gaussian logic can identify thermal anomalies, it requires the refined signal provided by the preprocessing pipeline to function reliably. By isolating the human subject as a distinct region, the system increased its average F1-Score **from ~71% to ~89%**, providing a robust foundation for real-time monitoring and model deployment.

5.3.3 Contact Detection

While the theoretical architecture for Contact Detection is detailed in the preceding Methods section, their full-scale implementation and empirical validation are strategically scheduled for the system integration phase. The operational efficacy of these specific algorithms is intrinsically linked to the spatial synchronization and overlapping fields of view inherent in a multi-view sensor network.

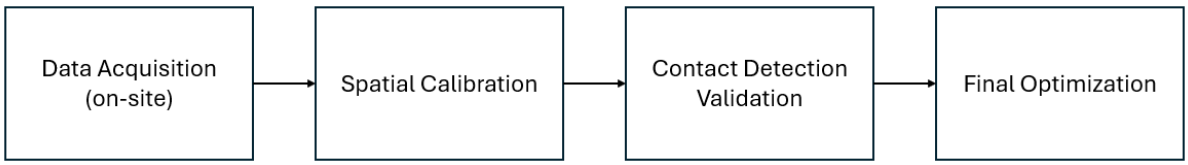


Figure 24: Algorithm Development Workflow

Consequently, high-fidelity testing requires a dataset that accurately reflects the unique occlusions and perspective-dependent thermal signatures found within the Mental Health Center's specific layout. Since synthetic or single-view data cannot sufficiently replicate the complex spatial relationships required for robust contact detection, finalized model training and performance benchmarking will be conducted *in situ*. This phased approach ensures that the algorithms are fine-tuned to the actual environmental geometry and sensor orientations of the live facility, ultimately enhancing the reliability of the patient monitoring system.

5.3.4 Fire Detection

At this phase of development, the evaluation focuses exclusively on the **Adaptive Segmentation (Otsu's Method)** stage. While the complete system includes a Machine Learning classifier (Decision Tree/SVM), the later stages of the pipeline approach both depend on the quality of the initial region proposal stage, as well as requiring significantly larger and more diverse datasets for training and validation, which will be collected during the upcoming system integration phase. Therefore, this section validates the region proposal capability, ensuring the system can reliably isolate potential thermal anomalies from the background before attempting to classify them. The segmentation algorithm was tested on the full preliminary dataset (459 frames), which includes both active thermal events (fire & human presence) and empty background frames. The distinction between a fire and a non-problematic thermal source will be made by the later stages.

Performance Summary and Metrics

Similarly to the human detection evaluation section, we utilize a confusion matrix and derive the same four metrics- Recall, Accuracy, Precision and F1-Score, on a frame-by-frame basis. Additionally, we also use IoU (Intersection over Union), which is a geometric metric that measures the overlap between the predicted region and the ground truth bounding box, divided by the area of their union- $\frac{Area\ of\ overlap}{Area\ of\ union}$.

It is worth noting that for preliminary fire hazard detection, Recall is a primary metric- a recall of under 1.0 implies a failure to detect a potential fire. IoU serves as an important metric as well, as it ensures that the system is not 'guessing' the presence of an object, but is accurately localizing the thermal signature.

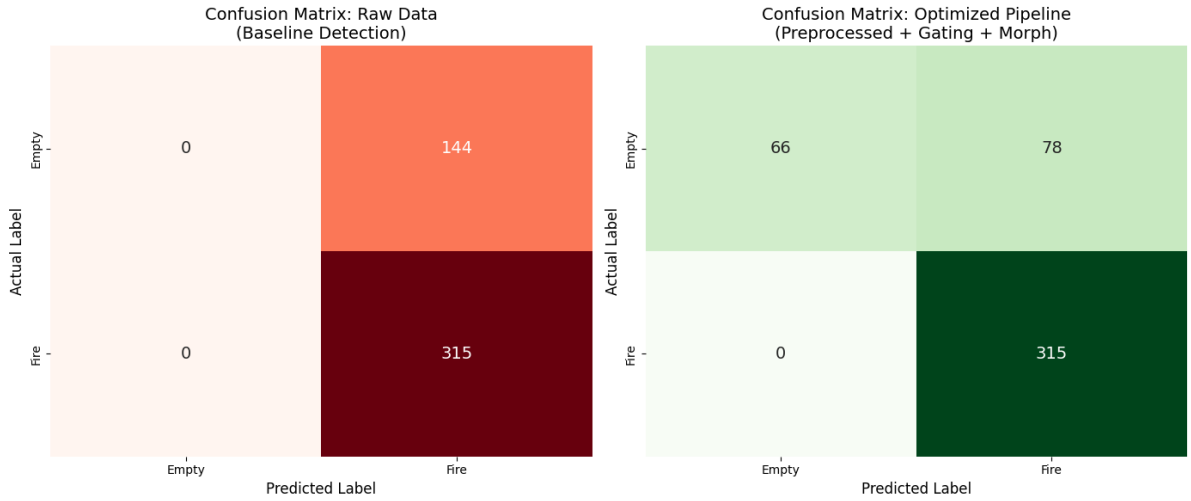


Figure 25: Confusion matrices - Adaptive Segmentation, raw vs preprocessed data

- **Raw Data (Baseline):** The matrix on the left reveals a critical failure mode: the system predicted "Fire" for every single frame (315 True Positives, 144 False Positives). This confirms that without the Variance Gate, Otsu's method forces a threshold on sensor noise in empty rooms, rendering the raw signal unusable for automated detection.
- **Optimized Pipeline:** The matrix on the right demonstrates the pipeline's ability to filter noise while preserving sensitivity.

- **Safety Verification (0 False Negatives):** The most critical result is the retention of 100% Recall. Every frame containing an active thermal signature was correctly identified. The pipeline did not miss a single true hazard.
- **Noise Rejection:** The Variance Gate successfully converted 66 empty frames from False Positives to True Negatives, effectively silencing ~46% of the background noise that plagued the baseline model.

Folder Name	Mode	IoU	Accuracy	Precision	Recall	F1	Correct/Total
lighter_on_3_sec	Raw	5.3%	100.0%	100.0%	100.0%	100.0%	38/38
	Proc	5.1%	100.0%	100.0%	100.0%	100.0%	38/38
lighter_on_and_off_3_sec	Raw	3.0%	40.0%	40.0%	100.0%	57.1%	6/15
	Proc	2.9%	40.0%	40.0%	100.0%	57.1%	6/15
lighter_on_off_on_off_5_sec	Raw	2.5%	42.3%	42.3%	100.0%	59.5%	22/52
	Proc	1.9%	42.3%	42.3%	100.0%	59.5%	22/52
1_man_jump_in_place_4_sec	Raw	68.5%	100.0%	100.0%	100.0%	100.0%	28/28
	Proc	77.0%	100.0%	100.0%	100.0%	100.0%	28/28
1_man_out_in_out_7_sec	Raw	30.2%	47.0%	47.0%	100.0%	63.9%	31/66
	Proc	59.8%	69.7%	60.8%	100.0%	75.6%	46/66
1_man_out_in_out_no_coat_7_sec	Raw	27.4%	40.7%	40.7%	100.0%	57.9%	22/54
	Proc	55.9%	64.8%	53.7%	100.0%	69.8%	35/54
1_sec_static_1_man	Raw	66.4%	100.0%	100.0%	100.0%	100.0%	10/10
	Proc	78.1%	100.0%	100.0%	100.0%	100.0%	10/10
2_men_far_near_far	Raw	53.5%	100.0%	100.0%	100.0%	100.0%	116/116
	Proc	63.6%	100.0%	100.0%	100.0%	100.0%	116/116
5_sec_1_man_movement	Raw	64.2%	100.0%	100.0%	100.0%	100.0%	42/42
	Proc	78.6%	100.0%	100.0%	100.0%	100.0%	42/42
no_man_3_sec	Raw	0.0%	0.0%	0.0%	0.0%	0.0%	0/38
	Proc	100.0%	100.0%	0.0%	0.0%	0.0%	38/38

Table 4: Fire Detection - Algorithm Evaluation Table

From the table we derive the following performance metrics:
Mean IoU: 0.5387
F1-Score: 0.8898
Recall: 1.0000
Precision: 0.8015
Accuracy: 0.8301

A closer inspection of the results reveals that the performance gains are primarily concentrated in noise rejection and background stabilization. For example, the no_man_3_sec dataset improved from 0% to 100% accuracy, proving the Variance Gate's effectiveness at silencing sensor noise in static environments.
However, a distinct behavior is observed in the lighter_on_off experiments, where accuracy and precision are noticeably lower (approx. 42%). This occurs because the Variance Gate is designed to filter environmental noise, not thermal objects. During the "Lighter Off" periods, the algorithm correctly detected the operator's hand/arm, which remained in the frame. Since the ground truth for these frames is labeled '0' (Empty/Safe, not flagged for any thermal presence), this detection is statistically recorded as a False Positive. This finding highlights the architectural role of the segmentation stage: it functions as a "suspect detector" (finding anything hot).
Another difference is observed between the Mean IoU for human subjects and small fire sources. This stems from the target ambiguity, since the ground truth labels isolate the flame while the segmentation sees the entire thermal signature, creating a mismatch between them.

Another reason would be the small dimensions of the flame, causing so that every deviation translates into a large error margin.

Conclusions

The evaluation confirms that the proposed optimized segmentation pipeline transforms the raw sensor output into a reliable safety tool. By delivering 100% detection of active threats (Recall 1.0) while ensuring that ~80% of the generated alerts are valid threats (Precision 0.80), the segmentation stage successfully fulfills its role as a "suspect detector," providing a stable input for the next stage in the fire detection system.

5.4 System Integration and Hardware Status

5.4.1 Hardware Configuration

The current prototype is built around a Raspberry Pi 5 Model B. Thermal management is handled by the official active cooler (fan). Power is delivered via the USB-C port using an official 27W power supply.

5.4.2 Peripherals & Interface

During development and debugging, the system is operated with a standard monitor via micro-HDMI and wireless Bluetooth mouse & keyboard via USB. For headless operation during data collection, remote access was maintained via OpenSSH over a local network.

5.4.3 Sensor Integration & Wiring

Integration of the three MLX90640 sensors is achieved using a TCA9548A I2C Multiplexer to address address-conflict limitations. The connection topology is illustrated in *Figure 26*.

- **Physical Connections:** All connections utilize standard jumper wires (approx. 30cm length) routed through a solderless breadboard. To mitigate mechanical instability common to prototyping, critical header pins on the sensors were soldered directly to the wire terminations where possible.
- **Protocol Enforcement:** A hardware modification was required on the sensor breakout boards. The UART interface pins were physically grounded. This pull-down ensures the sensors initialize strictly in I2C mode, preventing communication protocol errors during the boot sequence.
- **Pinout Mapping:** The Multiplexer connects to the Pi's I2C bus on Physical Pins 3 (SDA) and 5 (SCL). Power is drawn from Physical Pin 2 (5V) and Ground from Physical Pin 6.
- **Multiplexer Configuration:** To minimize wiring complexity, the TCA9548A is seated directly into the breadboard. The address selection pins (A0-A2) are grounded, setting the default I2C address. The sensors occupy channels 0, 1, and 2.

5.4.4 Prototype Topology & Component GPIO

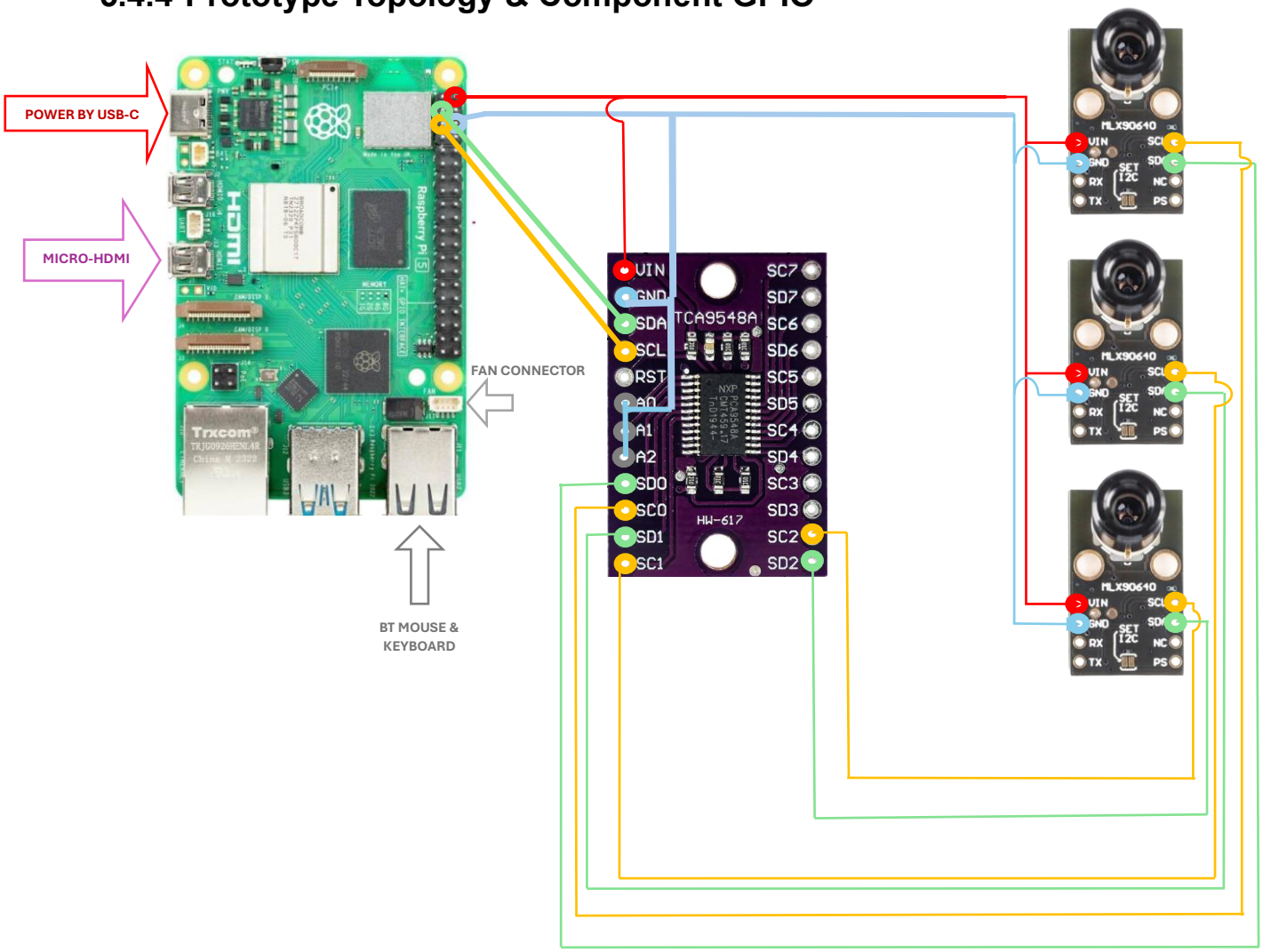


Figure 26: Prototype Topology

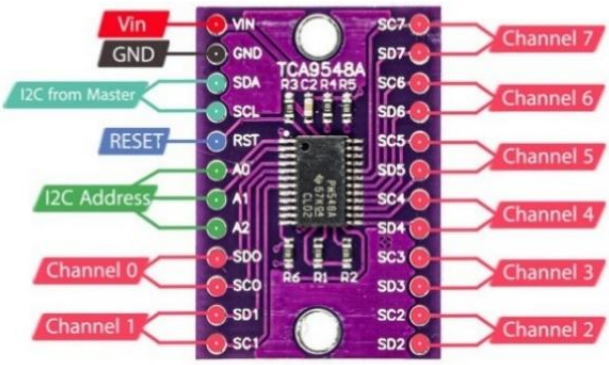


Figure 27: TCA9548A I2C Mux Pinout

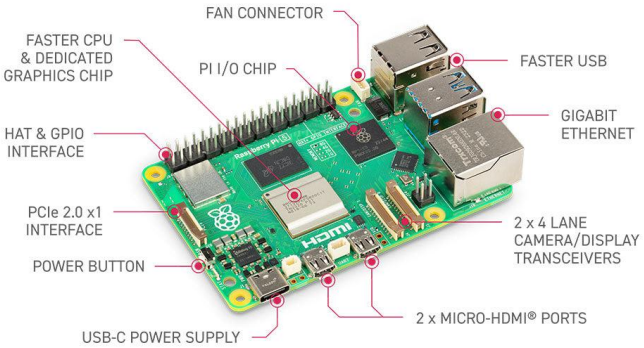


Figure 28: Raspberry Pi 5 On-Board Connections

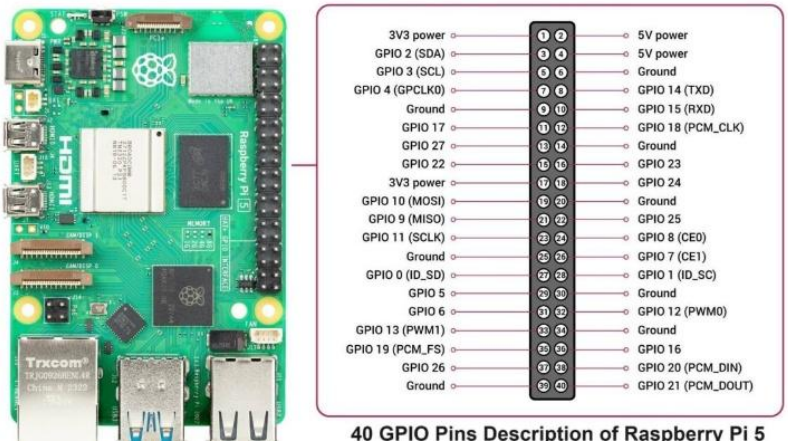


Figure 29: Raspberry Pi 5 Pinout

6 Next Steps and Interim Conclusions

Hardware Stabilization and Selection
Following the identification of the signal integrity risks associated with the required 2–3 meter sensor spacing, our immediate focus is to validate the physical infrastructure. We will proceed with stress-testing I2C communication over extended cable runs, potentially implementing I2C buffers or repeaters to ensure data stability. Concurrently, we will benchmark the Waveshare USB-C thermal camera against our current MLX setup to determine if switching to a USB-based architecture offers the necessary improvements in resolution and signal integrity.

Data Campaign and Algorithmic Shift
Having determined that our classical algorithms, while functional - do not meet the high robustness threshold required for a clinical environment, we will implement and test different learning-based architectures. To support this transition, and for system validation purposes, we will execute a comprehensive data expansion phase. This includes capturing in-site, multi-view footage within the mental health center to capture real-world variables, alongside applying data augmentation techniques to create a sufficiently diverse dataset.

System Integration and Interface Deployment
Once the hardware configuration is finalized and the model is trained, we will move to final integration. This entails fabricating the 3D-printed enclosures to house the sensor array and implementing the Ethernet interface for reliable communication with the external workstation. The project will conclude with a pilot evaluation in a single ward room, where a graphical user interface will monitor real-time performance. This test will serve as a proof of concept to validate the system’s detection accuracy and privacy-preserving capabilities in a real-world setting.

7 References

[1] A. Anderson and S. G. West, “Violence against mental health professionals: When the treater becomes the victim,” *Innovations in Clinical Neuroscience*, vol. 14, no. 11–12, pp. 34–40, Nov.–Dec. 2017. [Online]. Available: (link)

[2] Mental Health Treatment Act, Israel, 1991. [Online]. Available: (link)

[3] Kintronics, “Thermal versus optical IP cameras,” *Kintronics*, Nov. 11, 2022. [Online]. Available: (link)

[4] Visionify.ai, “Infrared thermal imaging for people detection,” *Visionify.ai*, Apr. 6, 2022. [Online]. Available: (link)

[5] Treatment of Mentally Ill Act [חוק טיפול בחולי נפש, התשנ״א-1991], Nevo – Israeli Legal Database. Accessed: Jan. 20, 2026. [Online]. Available: (link).

[6] Protection of Privacy Act [חוק הגנת הפרטיות, התשמ״א-1981], Nevo – Israeli Legal Database. Accessed: Jan. 20, 2026. [Online]. Available: (link).

[7] Patients’ Rights Act [חוק זכויות החולה, התשנ״ו-1996], Nevo – Israeli Legal Database. Accessed: Jan. 20, 2026. [Online]. Available: (link).

[8] Standards for Filming in Trauma and Intensive Care Rooms in the Emergency Department [אמות מידה לצילום בחדרי טראומה וטיפול נמרץ במחלקה לרפואה הדחופה], Ministry of Health Circular 09/2024 [חוזר מנכ"ל משרד הבריאות], Ministry of Health, Israel, 2024. Accessed: Jan. 20, 2026. [Online]. Available: (link)

[9] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022. [Online]. Available: (link)

[10] A. Pajankar, *Raspberry Pi Computer Vision Programming*. Packt Publishing, 2019.

[11] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*. Cambridge, UK: Cambridge University Press, 2023.

[12] N. Dubagunta, "Using machine learning for real-time detection of violence in video footage", 2022. [Online]. Available: (link)

[13] R. Vijeikis, "Efficient violence detection in surveillance," Sensors, 2022. [Online]. Available: (link)

[14] S. Tateno, F. Meng, R. Qian, and Y. Hachiya, "Privacy-preserved fall detection method with three-dimensional convolutional neural network using low-resolution infrared array sensor," *Sensors*, vol. 20, no. 20, Art. no. 5957, Oct. 2020 (link)

[15] Raspberry Pi Foundation, “Raspberry Pi 4 Model B Specifications,” *Raspberry Pi*, Accessed: Jun. 20, 2025. [Online]. Available: (link)

[16] Piitel, “Raspberry Pi 4 Model B,” *Piitel*, Accessed: Jun. 20, 2025. [Online]. Available: (link)

[17] Raspberry Pi Foundation, “Raspberry Pi 5 Specifications,” *Raspberry Pi*, Accessed: Jun. 20, 2025. [Online]. Available: (link)

[18] Piitel, “The Next Generation – Raspberry Pi 5,” *Piitel*, Accessed: Jun. 20, 2025. [Online]. Available: (link)

[19] Espressif Systems, *ESP32 Series Datasheet – Version 4.9*, Espressif, Mar. 2025. Accessed: Jun. 20, 2025. [Online]. Available: (link)

[20] Texas Instruments, “TCA9548A Low-Voltage 8-Channel I^2C Switch with Reset,” *Texas Instruments*, Accessed: Jun. 20, 2025. [Online]. Available: (link)

[21] “Adafruit AMG8833 IR Thermal Camera Breakout [STEMMA QT],” Adafruit Industries. (accessed Feb. 1, 2026) (link).

[22] “Grove - AMG8833 8x8 Infrared Thermal Temperature Sensor Array,” Seeed Studio. (accessed Feb. 1, 2026). (link)

[23] Melexis, Far infrared thermal sensor array (16x12 RES) - product page, Accessed: Jul. 7, 2025. (link)

[24] Pimoroni, MLX90640 Thermal Camera Breakout - Product Page, Accessed: Jul. 7, 2025. (link)

[25] Melexis, Far infrared thermal sensor array (32x24 RES) – Product page, Accessed: Jul. 7, 2025. (link)

[26] Waveshare, "Long-wave IR Thermal Imaging Camera Module" - Product Page, Waveshare, Accessed: Jan. 21, 2026. [Online]. Available (link)

[27] NXP, P82B715 I2C-bus extender, datasheet. Accessed: Jan. 20, 2026. [Online]. Available: (link)

[28] NXP, PCA9600 Dual bidirectional bus buffer, datasheet. Accessed: Jan. 20, 2026. [Online]. Available: (link)

[29] NXP, AN10710 Features and applications of the P82B715 I2C-bus extender. Accessed: Jan. 20, 2026. [Online]. Available: (link)

[30] All About Circuits, Guide to USB-C Pinout and Features. Accessed: Jan. 21, 2026. [Online]. Available: (link)

[31] USB.org, USB Type-C.. Accessed: Jan. 21, 2026. [Online]. Available: (link)

[32] Rocktech, I2C vs USB: Choosing the Right Interface for PCAP Touch Screens. Accessed: Jan. 21, 2026. [Online]. Available: (link)

[33] “Smoothing Images,” OpenCV Documentation. (accessed Feb. 1, 2026). (link)

[34] H. Farid, "Image analysis: homography: planar homography," *YouTube*, 21 min., 09 sec., Jun. 19, 2023. [Online]. Available: (link). [Accessed: Jan. 25, 2026]

[35] Otsu, N. (1979). "A Threshold Selection Method from Gray-Level Histograms", *IEEE Transactions on Systems, Man, and Cybernetics*. Accessed: Jan. 15, 2026. [Online]. Available: (link).

[36] Safavian, S. R., & Landgrebe, D. (1991), "A survey of decision tree classifier methodology", IEEE Transactions on Systems, Man, and Cybernetics. Accessed: Jan. 15, 2026. [Online]. Available: (link).

[37] B. Haranadh Reddy, Karthikeyan P R, (2022) "Classification of Fire and Smoke Images using Decision Tree Algorithm in Comparison with Logistic Regression to Measure Accuracy, Precision, Recall, F-score", 2022 14th International Conference on MACS (IEEE). Accessed: Jan. 17, 2026. [Online]. Available: (link).

[38] Celik T, Demirel H, Ozkaramanli H, Uyguroglu M. (2007), "Fire detection using statistical color model in video sequences", Journal of Visual Communication and Image Representation, Accessed: Jan. 17, 2026. Available: (link)

[39] See reference [14].

8 Appendices

8.1 Roles and Responsibilities

Section	Chapter	Guy Chen	Roy Lieberman	Yaniv Blau
Supervisor's Approval				
Acknowledgements			X	
Table of Contents				X
1. Abstract	1.1 תקציר (עברית)	X		
	1.2 Abstract	X		
2. Introduction	2.1 Opening		X	
	2.2 Goals, Objectives and Metrics		X	
	2.3 Requirements			X
	2.4 Project Status			X
3. Literature and Market Review	3.1 Literature Review			X
	3.2 Market Review	X		
4. Methods	4.1 Component Selection		X	
	4.2 System Architecture			X
	4.3 Data Collection and Labelling	X		
	4.4 Algorithm Description	X	X	X
5. Preliminary Results and Analysis	5.1 Prototype Configuration and Testing		X	
	5.2 Exploratory Data Analysis	X		
	5.3 Algorithmic Performance Evaluation	X	X	X
	5.4 System Integration and Hardware Status		X	
6. Next Steps and Interim Conclusions		X	X	X
7. References		X	X	X
8. Appendices	8.1 Roles and Responsibilities	X		
	8.2 Python Scripts and Algorithms	X	X	X
	8.3 Scientific Abstract for Research Paper	X		
	8.4 Work Plan (Gantt)			X
	8.5 Table of Changes		X	

8.2 Python Scripts and Algorithms

Here, we show all python scripts used to reach the results we have shown up until now.

8.2.1 Exploratory Data Analysis Codes

1. Imports and local file paths

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import seaborn as sns
import os
import pandas as pd
from tqdm import tqdm
import cv2
import numpy as np
import torch
from torch.utils.data import Dataset
from scipy.ndimage import label, sum as ndi_sum
from skimage.feature import hog
from scipy.stats import skew, kurtosis
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import warnings

ROOT = r"c:\Users\guych\Desktop\Final Project\Dataset\dataset_raw"
LABELS = r"c:\Users\guych\Desktop\Final
Project\Dataset\dataset_raw\labels.xlsx"
```

2. Class definitions for the raw and processed datasets

```
# ========================================
# 1. Raw Dataset (The Baseline)
# ========================================
class RawProjectDataset(Dataset):
    def __init__(self, root_dir, excel_file):
        self.root_dir = root_dir
        self.samples = []
        self.data_cache = {}

        # Map folders
        self.exp_path_map = {}
        for root, dirs, files in os.walk(root_dir):
            for d in dirs:
                self.exp_path_map[d] = os.path.join(root, d)

        # Load Labels
        xls = pd.read_excel(excel_file, sheet_name=None)

        print("Loading Raw Data & Indexing...")
        for exp_name, df in xls.items():
            if exp_name not in self.exp_path_map: continue
```

```
22.
23.             # 1. Fix Labels
24.             if 'touch' in df.columns:
25.                 df.loc[df['touch'] == 1, 'human'] = 1
26.
27.             # 2. Load NPZ Files
28.             for ch in df['channel'].unique():
29.                 cache_key = f"{exp_name}_ch{ch}"
30.                 npz_path = os.path.join(self.exp_path_map[exp_name],
    f"ch{ch}_raw_data.npz")
31.
32.                 if cache_key not in self.data_cache and
    os.path.exists(npz_path):
33.                     try:
34.                         loaded = np.load(npz_path)
35.                         self.data_cache[cache_key] =
    loaded[list(loaded.keys())[0]].astype(np.float32)
36.                     except Exception as e:
37.                         print(f"Error loading {npz_path}: {e}")
38.
39.             # 3. Build Index
40.             for _, row in df.iterrows():
41.                 ch = int(row['channel'])
42.                 frame_idx = int(row['frame'])
43.                 cache_key = f"{exp_name}_ch{ch}"
44.
45.                 if cache_key in self.data_cache:
46.                     if frame_idx < len(self.data_cache[cache_key]):
47.                         self.samples.append({
48.                             'cache_key': cache_key,
49.                             'frame_idx': frame_idx,
50.                             'labels': [row.get('human', 0),
    row.get('fire', 0), row.get('touch', 0)],
51.                             'exp_name': exp_name,
52.                             'channel': ch
53.                         })
54.
55.     def load_yolo_boxes(self, sample, img_height, img_width):
56.         """
57.         Parses YOLO .txt file for the specific frame and converts to
58.         absolute pixel coordinates (x1, y1, x2, y2).
59.         """
60.         exp_name = sample['exp_name']
61.         ch = sample['channel']
62.         f_idx = sample['frame_idx']
63.
64.         # Construct path: dataset/exp_name/ch0_frames/frame_00000.txt
```

```
        label_dir = os.path.join(self.exp_path_map[exp_name], f"ch{ch}_frames")
        label_path = os.path.join(label_dir, f"frame_{f_idx:05d}.txt")

        boxes = []
        labels = []

        if os.path.exists(label_path):
            with open(label_path, 'r') as f:
                lines = f.readlines()

            for line in lines:
                # YOLO format: class_id x_center y_center width height (Normalized 0-1)
                parts = list(map(float, line.strip().split()))
                cls_id = int(parts[0])
                xc, yc, w, h = parts[1], parts[2], parts[3], parts[4]

                # Convert to Absolute Pixels (x_min, y_min, x_max, y_max)
                x1 = (xc - w / 2) * img_width
                y1 = (yc - h / 2) * img_height
                x2 = (xc + w / 2) * img_width
                y2 = (yc + h / 2) * img_height

                # Clamp to image boundaries
                x1 = max(0, x1)
                y1 = max(0, y1)
                x2 = min(img_width, x2)
                y2 = min(img_height, y2)

                boxes.append([x1, y1, x2, y2])
                labels.append(cls_id + 1) # Add 1 because 0 is usually reserved for background in PyTorch

        # Return as Tensors
        if len(boxes) > 0:
            boxes_t = torch.tensor(boxes, dtype=torch.float32)
            labels_t = torch.tensor(labels, dtype=torch.int64)
              else:
                # No boxes found (Empty frame)
                boxes_t = torch.empty((0, 4), dtype=torch.float32)
                labels_t = torch.empty((0,), dtype=torch.int64)

              target = {
                "boxes": boxes_t,
                "labels": labels_t
              }
```

```
109.             return target
110.
111.         def __len__(self):
112.             return len(self.samples)
113.
114.         def __getitem__(self, idx):
115.             sample = self.samples[idx]
116.             raw_data =
    self.data_cache[sample['cache_key']][sample['frame_idx']]
117.
118.             # Get Dimensions for box scaling
119.             H, W = raw_data.shape
120.
121.             # Get Detection Targets (Boxes)
122.             target = self.load_yolo_boxes(sample, H, W)
123.
124.             # Get Classification Labels (Frame-level)
125.             class_labels = torch.tensor(sample['labels']).float()
126.
127.             # Return: Image, Classification Labels, Detection Targets
128.             return torch.tensor(raw_data).float().unsqueeze(0),
    class_labels, target
129.
130.     # ========================================
131.     # 2. Processed Dataset (The Experiment)
132.     # ========================================
133.     class ProcessedProjectDataset(RawProjectDataset):
134.         def __init__(self, root_dir, excel_file,
    bg_exp_name='no_man_3_sec'):
135.             super().__init__(root_dir, excel_file)
136.
137.             self.bg_ref = {}
138.             bg_found = False
139.
140.             print("Computing Background References...")
141.             for key, data_array in self.data_cache.items():
142.                 if key.startswith(bg_exp_name + "_ch"):
143.                     bg_found = True
144.                     try:
145.                         ch = int(key.split('_ch')[-1])
146.                         mean_bg = np.mean(data_array, axis=0)
147.                         self.bg_ref[ch] = cv2.GaussianBlur(mean_bg,
    (3, 3), 0)
148.                     except Exception as e:
149.                         print(f"Error processing background for
    {key}: {e}")
150.
151.             if not bg_found:
```

```
                print(f"Warning: Background experiment '{bg_exp_name}' not found.")
            else:
                print(f"Background reference created for channels: {list(self.bg_ref.keys())}")

        def __getitem__(self, idx):
            # 1. Get Raw Data AND Boxes from Parent
            raw_tensor, class_labels, target = super().__getitem__(idx)
            raw_numpy = raw_tensor.squeeze(0).numpy()

            # 2. Retrieve Channel info
            sample_info = self.samples[idx]
            ch = int(sample_info['cache_key'].split('_ch')[-1])

            # 3. Apply Pipeline
            blurred = cv2.GaussianBlur(raw_numpy, (3, 3), 0)

            if ch in self.bg_ref:
                processed = np.abs(blurred - self.bg_ref[ch])
            else:
                processed = blurred

            # Return Processed Tensor + Original Boxes
            return torch.tensor(processed).float().unsqueeze(0), class_labels, target
```

3. Create datasets and show difference in histogram between processed and raw images

```
# 1. Initialize both
print("Initializing Raw...")
ds_raw = RawProjectDataset(ROOT, LABELS)
print("Initializing Processed...")
ds_proc = ProcessedProjectDataset(ROOT, LABELS)

# 2. Find interesting samples (e.g., where Human is present)
# Note: s['labels'][0] is the Human flag
human_indices = [i for i, s in enumerate(ds_raw.samples) if s['labels'][0] == 1]
indices_to_plot = human_indices[:3] if len(human_indices) >= 3 else range(3)

# 3. Plot Comparison
fig, axes = plt.subplots(len(indices_to_plot), 3, figsize=(15, 5 * len(indices_to_plot)))
# Columns: Raw Image | Processed Image | Histogram Comparison

def draw_bboxes(ax, target, color='lime'):
```

```
    """Helper to draw boxes on a matplotlib axis"""
    boxes = target['boxes']
    if len(boxes) > 0:
        for box in boxes:
            x1, y1, x2, y2 = box.numpy()
            width = x2 - x1
            height = y2 - y1
            # Create a Rectangle patch
            rect = patches.Rectangle(
                (x1, y1), width, height,
                linewidth=2, edgecolor=color, facecolor='none'
            )
            ax.add_patch(rect)

for row_idx, data_idx in enumerate(indices_to_plot):
    # Get Data (Unpack 3 items now: image, labels, box_targets)
    raw_img, _, raw_target = ds_raw[data_idx]
    proc_img, _, proc_target = ds_proc[data_idx]

    raw_np = raw_img.squeeze().numpy()
    proc_np = proc_img.squeeze().numpy()

    # Col 1: Raw
    ax1 = axes[row_idx, 0]
    im1 = ax1.imshow(raw_np, cmap='inferno')
    ax1.set_title(f"Raw (Idx {data_idx})")
    draw_bboxes(ax1, raw_target, color='cyan') # Cyan for Raw
    plt.colorbar(im1, ax=ax1, fraction=0.046, pad=0.04)

    # Col 2: Processed
    ax2 = axes[row_idx, 1]
    im2 = ax2.imshow(proc_np, cmap='inferno')
    ax2.set_title(f"Processed (BG Subtracted)")
    draw_bboxes(ax2, proc_target, color='lime') # Lime for Processed
    plt.colorbar(im2, ax=ax2, fraction=0.046, pad=0.04)

    # Col 3: Histogram
    ax3 = axes[row_idx, 2]
    ax3.hist(raw_np.flatten(), bins=50, alpha=0.5, label='Raw',
color='gray')
    ax3.hist(proc_np.flatten(), bins=50, alpha=0.7, label='Processed',
color='orange')
    ax3.set_yscale('log')
    ax3.legend()
    ax3.set_title("Pixel Intensity Distribution")

plt.tight_layout()
plt.show()
```

4. Show statistical parameters that prove that the preprocessing pipeline has indeed improved upon the raw data

```
def evaluate_comprehensive_metrics(ds_raw, ds_proc):
    """
    Computes SBR (Visual Contrast) and Background Suppression.
    """
    results = {
        'raw_sbr': [], 'proc_sbr': [],
        'raw_mean_bg': [], 'proc_mean_bg': []
    }

    human_indices = [i for i, s in enumerate(ds_raw.samples) if s['labels'][0] == 1]
    print(f"Computing Metrics for {len(human_indices)} verified human samples...")

    for idx in human_indices:
        raw_tensor, _, raw_target = ds_raw[idx]
        proc_tensor, _, _ = ds_proc[idx]

        raw_np = raw_tensor.squeeze().numpy()
        proc_np = proc_tensor.squeeze().numpy()

        boxes = raw_target['boxes'].numpy()
        if len(boxes) == 0: continue

        # Masks
        mask_signal = np.zeros_like(raw_np, dtype=bool)
        for box in boxes:
            x1, y1, x2, y2 = box.astype(int)
            x1, y1 = max(0, x1), max(0, y1)
            x2, y2 = min(raw_np.shape[1], x2), min(raw_np.shape[0], y2)
            mask_signal[y1:y2, x1:x2] = True

        mask_bg = ~mask_signal

        # --- METRIC CALCULATIONS ---

        def get_metrics(img, m_sig, m_bg):
            # Avoid divide by zero
            sig_mean = np.mean(img[m_sig]) if np.any(m_sig) else 0
            bg_mean = np.mean(img[m_bg]) if np.any(m_bg) else 0

            # SBR: Signal / Background (Add epsilon to avoid div/0)
            # Use absolute values since thermal diffs can be negative in math but positive in magnitude
            sbr = abs(sig_mean) / (abs(bg_mean) + 1e-6)
```

```
            return sbr, bg_mean

        if np.any(mask_signal) and np.any(mask_bg):
            # Raw
            r_sbr, r_bg = get_metrics(raw_np, mask_signal, mask_bg)
            results['raw_sbr'].append(r_sbr)
            results['raw_mean_bg'].append(r_bg)

            # Processed
            p_sbr, p_bg = get_metrics(proc_np, mask_signal, mask_bg)
            results['proc_sbr'].append(p_sbr)
            results['proc_mean_bg'].append(p_bg)

    # Statistics
    avg_raw_sbr = np.mean(results['raw_sbr'])
    avg_proc_sbr = np.mean(results['proc_sbr'])

    avg_raw_bg = np.mean(results['raw_mean_bg'])
    avg_proc_bg = np.mean(results['proc_mean_bg'])

    # Calculate Improvements
    sbr_gain = avg_proc_sbr / avg_raw_sbr

    # Background Suppression (in dB)
    # How much did we lower the floor?
    bg_suppression = 20 * np.log10(avg_raw_bg / (avg_proc_bg + 1e-6))

    return avg_raw_sbr, avg_proc_sbr, sbr_gain, bg_suppression

# --- Run ---
r_sbr, p_sbr, gain, bg_db = evaluate_comprehensive_metrics(ds_raw,
  ds_proc)

print("\n" + "="*40)
print("   VISUAL CONTRAST & SUPPRESSION")
print("="*40)
print(f"Raw SBR:                {r_sbr:.2f}")
print(f"Processed SBR:          {p_sbr:.2f}")
print("-" * 40)
print(f"CONTRAST GAIN:          {gain:.2f}x")
print(f"BACKGROUND SUPPRESSION: {bg_db:.2f} dB")
print("="*40)
```

5. Feature Analysis - Maximum temperature in frame

```
# Suppress specific future warnings just in case environment differs
warnings.simplefilter(action='ignore', category=FutureWarning)

def extract_max_temp(dataset, dataset_name):
    """
    Iterates through the dataset and extracts the maximum pixel value
    (max temperature) for each frame, along with its classification.
    """
    data = []

    print(f"Extracting features from {dataset_name}...")

    for i in range(len(dataset)):
        # --- THE FIX ---
        # The dataset now returns 3 items: (image, labels, boxes)
        # We only need the image and labels for this plot.
        img_tensor, label_tensor, _ = dataset[i]

        # Convert to numpy (Squeeze to remove channel dim: 1xHxW -> HxW)
        img_np = img_tensor.squeeze().numpy()
        labels = label_tensor.numpy() # [Human, Fire, Touch]

        # Determine Class Label (Hierarchy: Fire > Human > Empty)
        if labels[1] == 1:
            category = "Fire"
        elif labels[0] == 1:
            category = "Human"
        else:
            category = "Empty"

        # Calculate Max Temperature
        max_val = np.max(img_np)

        data.append({
            "Max Temp": max_val,
            "Class": category,
            "Dataset": dataset_name
        })

    return pd.DataFrame(data)

# --- 1. Extract Data ---
df_raw = extract_max_temp(ds_raw, "Raw Data")
df_proc = extract_max_temp(ds_proc, "Processed Data")

```

```
# --- 2. Generate Violin Plots ---
fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=False)

# Define a consistent color palette
my_palette = {"Empty": "#95a5a6", "Human": "#3498db", "Fire": "#e74c3c"}

# Plot A: Raw Data
sns.violinplot(
    data=df_raw,
    x="Class",
    y="Max Temp",
    hue="Class",
    palette=my_palette,
    legend=False,
    ax=axes[0],
    cut=0
)
axes[0].set_title("Raw Data: Max Temperature Distribution", fontsize=14)
axes[0].set_ylabel("Pixel Intensity (Raw)", fontsize=12)
axes[0].grid(True, axis='y', alpha=0.3)

# Plot B: Processed Data
sns.violinplot(
    data=df_proc,
    x="Class",
    y="Max Temp",
    hue="Class",
    palette=my_palette,
    legend=False,
    ax=axes[1],
    cut=0
)
axes[1].set_title("Processed Data: Max Temperature Distribution", fontsize=14)
axes[1].set_ylabel("Pixel Intensity (Diff Magnitude)", fontsize=12)
axes[1].grid(True, axis='y', alpha=0.3)

plt.suptitle("Feature Analysis – Maximum Temperature", fontsize=16, y=1.05)
plt.tight_layout()
plt.show()
```

6. Feature Analysis - Maximal Blob Size

```
def extract_ground_truth_areas(dataset, dataset_name):
    """
    Extracts the area of the largest GROUND TRUTH bounding box for each
frame.
    """
    data = []
    print(f"Extracting Ground Truth areas from {dataset_name}...")

    for i in range(len(dataset)):
        # Unpack the 3 items: image, label_tensor, target
        _, label_tensor, target = dataset[i]
        labels = label_tensor.numpy()

        # Determine Class
        if labels[1] == 1:
            category = "Fire"
        elif labels[0] == 1:
            category = "Human"
        else:
            category = "Empty"

        # Extract Ground Truth Box Areas
        boxes = target['boxes']

        if boxes.shape[0] > 0:
            # Calculate Area: (x2 - x1) * (y2 - y1)
            widths = boxes[:, 2] - boxes[:, 0]
            heights = boxes[:, 3] - boxes[:, 1]
            areas = widths * heights

            # Take the largest box in the frame
            max_area = torch.max(areas).item()
        else:
            # No boxes (Empty frame)
            max_area = 0

        data.append({
            "Max GT Area": max_area,
            "Class": category
        })

    return pd.DataFrame(data)

# --- 1. Extract Data ---
# We only need to do this once since Raw and Processed share the same
labels
```

```
df_gt = extract_ground_truth_areas(ds_raw, "Ground Truth Labels")

# --- 2. Plotting ---
plt.figure(figsize=(10, 6))

my_palette = {"Empty": "#95a5a6", "Human": "#3498db", "Fire": "#e74c3c"}

sns.violinplot(
    data=df_gt,
    x="Class",
    y="Max GT Area",
    hue="Class",
    palette=my_palette,
    legend=False,
    cut=0,  # Cut at 0 to show that Empty frames are exactly 0
    inner="quartile"
)

plt.title("Feature Analysis: Blob Maximal Size", fontsize=16)
plt.ylabel("Bounding Box Area (Pixels)", fontsize=12)
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

7. Feature Analysis - Width-Height Ratio

```
def extract_box_dimensions_merged(dataset, dataset_name):
    """
    Extracts Width and Height of bounding boxes.
    SPECIAL LOGIC: For 'Touch' frames with multiple boxes, merges them
    into a single 'Union' bounding box (encompassing all actors).
    """
    data = []
    print(f"Extracting box dimensions from {dataset_name}...")

    for i in range(len(dataset)):
        # Unpack: image, labels, target
        _, label_tensor, target = dataset[i]
        labels = label_tensor.numpy() # [Human, Fire, Touch]

        # Determine Class
        if labels[2] == 1:
            category = "Touch"
        elif labels[1] == 1:
            category = "Fire"
        elif labels[0] == 1:
            category = "Human"
        else:
```

```
            category = "Other"

        # Get Boxes (Tensor)
        boxes = target['boxes']

        # --- MERGING LOGIC ---
        # If class is Touch AND there are multiple boxes, treat them as one large event
        if category == "Touch" and boxes.shape[0] > 1:
            # Find the extreme coordinates across ALL boxes in this frame
            min_x1 = torch.min(boxes[:, 0]).item()
            min_y1 = torch.min(boxes[:, 1]).item()
            max_x2 = torch.max(boxes[:, 2]).item()
            max_y2 = torch.max(boxes[:, 3]).item()

            width = max_x2 - min_x1
            height = max_y2 - min_y1

            data.append({
                "Width": width,
                "Height": height,
                "Class": category,
                "Aspect Ratio": height / width if width > 0 else 0
            })

        # --- STANDARD LOGIC ---
        # For non-Touch frames OR Touch frames with only 1 box (or 0)
        else:
            if boxes.shape[0] > 0:
                for box in boxes:
                    x1, y1, x2, y2 = box.numpy()

                    width = x2 - x1
                    height = y2 - y1

                    data.append({
                        "Width": width,
                        "Height": height,
                        "Class": category,
                        "Aspect Ratio": height / width if width > 0 else 0
                    })

    return pd.DataFrame(data)

# --- 1. Extract Data ---
df_dims = extract_box_dimensions_merged(ds_raw, "Ground Truth Labels")
```

```
68.
69.# --- 2. Plotting ---
70.plt.figure(figsize=(10, 8))
71.
72.# Define colors
73.my_palette = {"Human": "#3498db", "Fire": "#e74c3c", "Touch":
   "#2ecc71"}
74.
75.# Scatter Plot
76.sns.scatterplot(
77.    data=df_dims,
78.    x="Width",
79.    y="Height",
80.    hue="Class",
81.    style="Class",
82.    palette=my_palette,
83.    s=80,
84.    alpha=0.7
85.)
86.
87.# Reference Lines
88.max_dim = max(df_dims["Width"].max(), df_dims["Height"].max())
89.plt.plot([0, max_dim], [0, max_dim], 'k--', alpha=0.3, label="Square
   (1:1)")
90.plt.plot([0, max_dim/2], [0, max_dim], 'k:', alpha=0.3, label="Tall
   (2:1)")
91.
92.plt.title("Geometric Feature Analysis: Box Width vs. Height\n(Merged
   Bounding Boxes for Touch)", fontsize=16)
93.plt.xlabel("Box Width (Pixels)", fontsize=12)
94.plt.ylabel("Box Height (Pixels)", fontsize=12)
95.plt.legend(title="Class", loc='upper left')
96.plt.grid(True, alpha=0.3)
97.plt.axis('equal')
98.plt.tight_layout()
99.plt.show()
```

8. Feature Analysis - Mean and Standard Deviation

```
def extract_roi_stats(dataset, dataset_name):
    """
    Extracts Mean and Std Dev of the ROI (Region of Interest).
    - If boxes exist: Stats calculated on the UNION of all box pixels.
    - If no boxes: Stats calculated on the ENTIRE image.
    """
    data = []
    print(f"Extracting ROI stats from {dataset_name}...")

    # Safe iteration
    num_samples = len(dataset.samples)

    for i in tqdm(range(num_samples)):
        # Unpack
        img_tensor, label_tensor, target = dataset[i]

        img_np = img_tensor.squeeze().numpy()
        labels = label_tensor.numpy()
        boxes = target['boxes']

        # 1. Create the ROI Mask
        if boxes.shape[0] > 0:
            # Initialize empty mask (False)
            mask = np.zeros_like(img_np, dtype=bool)

            for box in boxes:
                x1, y1, x2, y2 = box.numpy().astype(int)

                # Clip to image dimensions to prevent crashes
                x1, y1 = max(0, x1), max(0, y1)
                x2, y2 = min(img_np.shape[1], x2), min(img_np.shape[0],
  y2)

                # Set ROI region to True
                mask[y1:y2, x1:x2] = True

            # Select only the pixels inside the mask
            roi_pixels = img_np[mask]

            # Edge case safety: if box has 0 area
            if roi_pixels.size == 0:
                roi_pixels = img_np.flatten()
        else:
            # No boxes -> Use entire image
            roi_pixels = img_np.flatten()

        # 2. Determine Class
```

```
        if labels[2] == 1:
            category = "Touch"
        elif labels[1] == 1:
            category = "Fire"
        elif labels[0] == 1:
            category = "Human"
        else:
            category = "Empty"

        # 3. Calculate Stats on the ROI pixels only
        data.append({
            "ROI Mean Intensity": np.mean(roi_pixels),
            "ROI Std Deviation": np.std(roi_pixels),
            "Class": category,
            "Dataset": dataset_name
        })

    return pd.DataFrame(data)

# --- 1. Extract Data ---
df_roi_raw = extract_roi_stats(ds_raw, "Raw Data")
df_roi_proc = extract_roi_stats(ds_proc, "Processed Data")

# --- 2. Plotting ---
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# Consistent Palette
my_palette = {"Empty": "#95a5a6", "Human": "#3498db", "Fire":
   "#e74c3c", "Touch": "#2ecc71"}

# Plot A: Raw Data
sns.scatterplot(
    data=df_roi_raw,
    x="ROI Mean Intensity",
    y="ROI Std Deviation",
    hue="Class",
    style="Class",
    palette=my_palette,
    s=60,
    alpha=0.7,
    ax=axes[0]
)
axes[0].set_title("Raw Data: ROI Statistics", fontsize=14)
axes[0].set_xlabel("Mean Intensity (Inside Box)")
axes[0].set_ylabel("Std Deviation (Inside Box)")
axes[0].grid(True, alpha=0.3)

# Plot B: Processed Data
```

```
sns.scatterplot(
    data=df_roi_proc,
    x="ROI Mean Intensity",
    y="ROI Std Deviation",
    hue="Class",
    style="Class",
          palette=my_palette,
          s=60,
          alpha=0.7,
          ax=axes[1]
      )
      axes[1].set_title("Processed Data: ROI Statistics", fontsize=14)
      axes[1].set_xlabel("Mean Intensity (Inside Box)")
      axes[1].set_ylabel("Std Deviation (Inside Box)")
      axes[1].grid(True, alpha=0.3)

      plt.suptitle("Object-Specific Statistics (Calculated on ROI
   Union)", fontsize=16)
      plt.tight_layout()
      plt.show()
```

9. Feature Analysis - Histogram of Gradients

```
# --- Configuration ---
# Adjust these based on your thermal image resolution
# For blurry thermal blobs, larger cells (16x16) often work better than standard 8x8
HOG_PARAMS = {
    'orientations': 9,
    'pixels_per_cell': (8, 8),
    'cells_per_block': (2, 2),
    'visualize': True,
    'block_norm': 'L2-Hys'
}

def analyze_hog_features(dataset, max_samples=500):
    """
    Extracts HoG features from the dataset and visualizes class separability.
    Focusing on Class 2 ('touch') vs Class 0 ('human' but no touch).
    """

    hog_vectors = []
    labels = []

    # Accumulators for "Average HoG Image" visualization
    sum_hog_image_contact = None
    count_contact = 0

    sum_hog_image_no_contact = None
    count_no_contact = 0

    print(f"Extracting HoG features from up to {max_samples} samples...")

    # Iterate through dataset (randomly shuffle indices to get a representative mix)
    indices = np.random.permutation(len(dataset))[:max_samples]

    for idx in tqdm(indices):
        # 1. Get Image and Label
        # processed_tensor shape: (1, H, W)
        img_tensor, class_labels, _ = dataset[idx]

        # Convert to numpy 2D array (H, W) for Skimage
        img = img_tensor.squeeze(0).numpy().astype(np.uint8)

        # Determine Class: Let's assume index 2 is 'touch'
        is_contact = (class_labels[2] == 1)
        is_human = (class_labels[0] == 1)
```

```
44.
45.         # Filter: We only care about frames with humans (Human vs
    Contact)
46.         # If you want to test Human vs Background, remove this check.
47.         if not is_human:
48.             continue
49.
50.         # 2. Compute HoG
51.         # vec: 1D array (the feature vector)
52.         # vis: 2D array (the visualization image)
53.         vec, vis = hog(img, **HOG_PARAMS)
54.
55.         hog_vectors.append(vec)
56.         labels.append(1 if is_contact else 0)
57.
58.         # 3. Accumulate for Average Visualization
59.         if is_contact:
60.             if sum_hog_image_contact is None:
61.                 sum_hog_image_contact = np.zeros_like(vis, dtype=float)
62.             sum_hog_image_contact += vis
63.             count_contact += 1
64.         else:
65.             if sum_hog_image_no_contact is None:
66.                 sum_hog_image_no_contact = np.zeros_like(vis,
    dtype=float)
67.             sum_hog_image_no_contact += vis
68.             count_no_contact += 1
69.
70.     # --- Analysis Part 1: Average HoG Images ---
71.     avg_contact = sum_hog_image_contact / count_contact if
    count_contact > 0 else np.zeros_like(vis)
72.     avg_no_contact = sum_hog_image_no_contact / count_no_contact if
    count_no_contact > 0 else np.zeros_like(vis)
73.
74.     plt.figure(figsize=(12, 5))
75.
76.     plt.subplot(1, 2, 1)
77.     plt.title(f"Average HoG: No Contact (N={count_no_contact})")
78.     plt.imshow(avg_no_contact, cmap='inferno')
79.     plt.axis('off')
80.
81.     plt.subplot(1, 2, 2)
82.     plt.title(f"Average HoG: Contact (N={count_contact})")
83.     plt.imshow(avg_contact, cmap='inferno')
84.     plt.axis('off')
85.
86.     plt.tight_layout()
87.     plt.show()
```

```
88.
89.     # --- Analysis Part 2: PCA Scatter Plot ---
90.     print("Computing PCA...")
91.     X = np.array(hog_vectors)
92.     y = np.array(labels)
93.
94.     # Project down to 2 Dimensions
95.     pca = PCA(n_components=2)
96.     X_pca = pca.fit_transform(X)
97.
98.     plt.figure(figsize=(10, 8))
99.     scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y,
   cmap='coolwarm', alpha=0.6, edgecolors='k')
100.
101.         plt.title(f"PCA of HoG Features (Explained Var:
   {np.sum(pca.explained_variance_ratio_):.2f})")
102.         plt.xlabel("Principal Component 1")
103.         plt.ylabel("Principal Component 2")
104.
105.         # Legend
106.         handles, _ = scatter.legend_elements()
107.         plt.legend(handles, ['No Contact', 'Contact'], loc="best")
108.         plt.grid(True, alpha=0.3)
109.         plt.show()
110.
111.     # --- Run it ---
112.     # Initialize your dataset first
113.     analyze_hog_features(ds_proc)
```

10. Feature Analysis - Skewness and Kurtosis

```
def analyze_skew_kurtosis(dataset, max_samples=None):
    """
    Computes Skewness and Kurtosis for the entire dataset and plots
    the 4 classes (Empty, Fire, Human, Touch) in different colors.
    """

    # Storage for features and colors
    features = []
    colors = []
    labels_text = []

    # Define Color Map
    # Order of priority: Touch > Fire > Human > Empty
    COLOR_MAP = {
        'Touch': 'green',
        'Fire':  'red',
        'Human': 'blue',
        'Empty': 'gray'
    }

    print("Extracting Histogram Statistics (Skew/Kurt)...")

    # Determine number of samples
    num_samples = len(dataset)
    if max_samples and max_samples < num_samples:
        indices = np.random.permutation(num_samples)[:max_samples]
    else:
        indices = range(num_samples)

    for idx in tqdm(indices):
        # 1. Get Image and Labels
        img_tensor, class_labels, _ = dataset[idx]
        img = img_tensor.squeeze(0).numpy().flatten() # Flatten to 1D
  array for stats

        # Labels are [Human, Fire, Touch]
        is_human = class_labels[0] == 1
        is_fire  = class_labels[1] == 1
        is_touch = class_labels[2] == 1

        # 2. Determine Class (Hierarchy matters!)
        if is_touch:
            cls = 'Touch'
        elif is_fire:
            cls = 'Fire'
        elif is_human:
            cls = 'Human'
```

```
        else:
            cls = 'Empty'

        # 3. Compute Statistics
        # Skewness: Asymmetry of the pixel distribution
        s = skew(img)
        # Kurtosis: "Tailedness" of the distribution
        k = kurtosis(img)

        if k<10:
            features.append([s, k])
            colors.append(COLOR_MAP[cls])
            labels_text.append(cls)

    # Convert to Numpy for easy slicing
    features = np.array(features)
    labels_text = np.array(labels_text)

    # --- Visualization ---
    plt.figure(figsize=(10, 8))

    # Plot each class separately to handle the legend correctly
    for cls, color in COLOR_MAP.items():
        # Filter points belonging to this class
        mask = (labels_text == cls)
        if np.sum(mask) > 0:
            plt.scatter(
                features[mask, 0],
                features[mask, 1],
                c=color,
                label=cls,
                alpha=0.6,
                edgecolors='w',
                linewidth=0.5
            )

    plt.title("Pixel Intensity Statistics: Skewness vs. Kurtosis")
    plt.xlabel("Skewness (Asymmetry)")
    plt.ylabel("Kurtosis (Peakedness)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

# --- Run Analysis ---
# Assuming 'dataset' is your ProcessedProjectDataset instance
# We use all samples to see the full distribution
analyze_skew_kurtosis(ds_proc, max_samples=2000)
```

8.2.2 Algorithm Evaluations

11. Algorithm Evaluation – Person Detection – Adaptive gaussian thresholding

```
1. def adaptive_gaussian_logic(processed_frame, block_size=7, C=7,
   min_area=8):
2.     # 1. Minimal Crop: Only remove the outermost 1-pixel border
3.     # This keeps the sensing area large while still removing edge
   noise.
4.     roi = processed_frame[1:-1, 1:-1]
5.
6.     # 2. Relaxed Temperature Gate
7.     if (roi.max() - roi.min()) < 0.8:
8.         return 0
9.
10.    # 3. Normalization
11.    f_min, f_max = roi.min(), roi.max()
12.    gray = np.uint8(255 * (roi - f_min) / (f_max - f_min + 1e-6))
13.
14.    # 4. Adaptive Thresholding (Inverted)
15.    # Lower C (7) makes it easier to connect fragmented human parts
16.    binary = cv2.adaptiveThreshold(
17.        gray, 255,
18.        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
19.        cv2.THRESH_BINARY_INV,
20.        block_size, C
21.    )
22.
23.    # 5. Geometric Filtering
24.    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL,
   cv2.CHAIN_APPROX_SIMPLE)
25.
26.    for cnt in contours:
27.        area = cv2.contourArea(cnt)
28.        if area >= min_area:
29.            # Relaxed Solidity: 0.2 is enough to catch
   moving/fragmented humans
30.            hull = cv2.convexHull(cnt)
31.            hull_area = cv2.contourArea(hull)
32.            solidity = float(area)/hull_area if hull_area > 0 else 0
33.
34.            if solidity > 0.2:
35.                return 1
36.
37.    return 0
```

12. Integration script:

```
# 1. Folders to compare
human_folders = [
    '1_man_jump_in_place_4_sec',
    '1_man_out_in_out_7_sec',
    '1_man_out_in_out_no_coat_7_sec',
    '1_sec_static_1_man',
    '2_men_far_near_far',
    '5_sec_1_man_movement'
]

# Ensure datasets are initialized
ds_raw = RawProjectDataset(ROOT, LABELS)
ds_proc = ProcessedProjectDataset(ROOT, LABELS)

comparison_summary = []

def calculate_detailed_metrics(y_true, y_pred):
    """Calculates statistical metrics and raw frame counts."""
    # Count correctly classified frames (TP + TN)
    correct_frames = np.sum(y_pred == y_true)
    total_frames = len(y_true)

    tp = np.sum((y_pred == 1) & (y_true == 1))
    fp = np.sum((y_pred == 1) & (y_true == 0))
    fn = np.sum((y_pred == 0) & (y_true == 1))

    acc = (correct_frames / total_frames) * 100 if total_frames > 0 else 0
    prec = (tp / (tp + fp)) * 100 if (tp + fp) > 0 else 0
    rec = (tp / (tp + fn)) * 100 if (tp + fn) > 0 else 0
    f1 = 2 * (prec * rec) / (prec + rec) if (prec + rec) > 0 else 0

    return acc, prec, rec, f1, correct_frames, total_frames

print("Starting Parallel Comparison (Raw vs Processed)...")

for folder in human_folders:
    # Filter indices for this folder
    indices = [i for i, s in enumerate(ds_proc.samples) if s['exp_name'] == folder]
    if not indices: continue

    raw_preds, proc_preds, gts = [], [], []

    for idx in tqdm(indices, desc=f"Processing {folder}"):
        raw_tensor, labels, _ = ds_raw[idx]
        proc_tensor, _, _ = ds_proc[idx]
```

```
46.
47.         raw_np = raw_tensor.squeeze().numpy()
48.         proc_np = proc_tensor.squeeze().numpy()
49.
50.         # Run identical logic on both pipelines
51.         p_res = adaptive_gaussian_logic(proc_np, block_size=7, C=10,
    min_area=17)
52.         r_res = adaptive_gaussian_logic(raw_np, block_size=7, C=10,
    min_area=17)
53.
54.         proc_preds.append(p_res)
55.         raw_preds.append(r_res)
56.         gts.append(int(labels[0]))
57.
58.     # Calculate metrics and frame counts
59.     y_true = np.array(gts)
60.     r_metrics = calculate_detailed_metrics(y_true,
    np.array(raw_preds))
61.     p_metrics = calculate_detailed_metrics(y_true,
    np.array(proc_preds))
62.
63.     comparison_summary.append({
64.         'Folder': folder,
65.         'Raw': r_metrics,
66.         'Proc': p_metrics
67.     })
68.
69.# 2. Updated Final Summary Table with "Correct/Total" column
70.print("\n" + "="*125)
71.print(f"{'Folder Name':<30} | {'Mode':<6} | {'Acc':<6} | {'Prec':<6}
    | {'Rec':<6} | {'F1':<6} | {'Correct/Total'}")
72.print("-" * 125)
73.for res in comparison_summary:
74.     f = res['Folder']
75.     r = res['Raw']
76.     p = res['Proc']
77.     # Format: Accuracy | Precision | Recall | F1 | Correct/Total
78.     print(f"{f:<30} | Raw    | {r[0]:>5.1f}% | {r[1]:>5.1f}% |
    {r[2]:>5.1f}% | {r[3]:>5.1f}% | {int(r[4])}/{int(r[5])}")
79.     print(f"{'':<30} | Proc   | {p[0]:>5.1f}% | {p[1]:>5.1f}% |
    {p[2]:>5.1f}% | {p[3]:>5.1f}% | {int(p[4])}/{int(p[5])}")
80.     print("-" * 125)
```

13. confusion matrix script:

```
# 1. Target experiments within 'man_presence_easy'
human_folders = [
    '1_man_jump_in_place_4_sec',
    '1_man_out_in_out_7_sec',
    '1_man_out_in_out_no_coat_7_sec',
    '1_sec_static_1_man',
    '2_men_far_near_far',
    '5_sec_1_man_movement'
]

# Accumulators for aggregate metrics
agg_raw_preds = []
agg_proc_preds = []
agg_gts = []

print(f"Aggregating results across {len(human_folders)} experiments...")

for folder in human_folders:
    indices = [i for i, s in enumerate(ds_proc.samples) if s['exp_name'] == folder]

    for idx in indices:
        # Load raw and processed frames
        raw_tensor, labels, _ = ds_raw[idx]
        proc_tensor, _, _ = ds_proc[idx]

        raw_np = raw_tensor.squeeze().numpy()
        proc_np = proc_tensor.squeeze().numpy()

        # Run detection (Using the tuned C=10, min_area=17)
        r_pred = adaptive_gaussian_logic(raw_np, block_size=7, C=10, min_area=17)
        p_pred = adaptive_gaussian_logic(proc_np, block_size=7, C=10, min_area=17)

        agg_raw_preds.append(r_pred)
        agg_proc_preds.append(p_pred)
        agg_gts.append(int(labels[0]))

# 2. Manual Confusion Matrix Calculation
def calculate_cm(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    tp = np.sum((y_pred == 1) & (y_true == 1))
    tn = np.sum((y_pred == 0) & (y_true == 0))
    fp = np.sum((y_pred == 1) & (y_true == 0))
    fn = np.sum((y_pred == 0) & (y_true == 1))
```

```
    return np.array([[tn, fp], [fn, tp]])

cm_raw = calculate_cm(agg_gts, agg_raw_preds)
cm_proc = calculate_cm(agg_gts, agg_proc_preds)

# 3. Plotting the Comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
labels = ['Empty', 'Human']

# Plot Raw
sns.heatmap(cm_raw, annot=True, fmt='d', cmap='Reds', ax=axes[0],
            xticklabels=labels, yticklabels=labels)
axes[0].set_title('Confusion Matrix: Raw Data\n(Baseline Detection)')
axes[0].set_xlabel('Predicted Label')
axes[0].set_ylabel('Actual Label')

# Plot Processed
sns.heatmap(cm_proc, annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=labels, yticklabels=labels)
axes[1].set_title('Confusion Matrix: Processed Data\n(Optimized Pipeline)')
axes[1].set_xlabel('Predicted Label')
axes[1].set_ylabel('Actual Label')

plt.suptitle("Comparative Performance Analysis: Man Presence Easy", fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

14. Algorithm Evaluation – Fire Detection – Otsu's Method

```
import os
import cv2
import numpy as np
import pandas as pd
import torch
from torch.utils.data import Dataset
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score
from tqdm import tqdm

# --- Helper: Save Images with Unicode/Hebrew Paths ---
def save_image_safe(path, img):
    is_success, buffer = cv2.imencode(".png", img)
    if is_success:
        with open(path, "wb") as f:
            f.write(buffer)

```

```
# ========================================
# 1. Data Ingestion Classes
# ========================================
class RawProjectDataset(Dataset):
    def __init__(self, root_dir, excel_file):
        self.root_dir = root_dir
        self.samples = []
        self.data_cache = {}
        self.exp_path_map = {}

        # Map directories
        for root, dirs, files in os.walk(root_dir):
            for d in dirs: self.exp_path_map[d] = os.path.join(root, d)

        try:
            xls = pd.read_excel(excel_file, sheet_name=None)
        except Exception as e:
            print(f"Error loading Excel: {e}")
            return

        # Load Data Cache
        for exp_name, df in xls.items():
            if exp_name not in self.exp_path_map: continue
            if 'touch' in df.columns: df.loc[df['touch'] == 1, 'human'] = 1

            for ch in df['channel'].unique():
                cache_key = f"{exp_name}_ch{ch}"
                npz_path = os.path.join(self.exp_path_map[exp_name], f"ch{ch}_raw_data.npz")
                if cache_key not in self.data_cache and os.path.exists(npz_path):
                    try:
                        loaded = np.load(npz_path)
                        self.data_cache[cache_key] = loaded[list(loaded.keys())[0]].astype(np.float32)
                    except: pass

            # Create Index
            for _, row in df.iterrows():
                ch, f_idx = int(row['channel']), int(row['frame'])
                cache_key = f"{exp_name}_ch{ch}"
                if cache_key in self.data_cache and f_idx < len(self.data_cache[cache_key]):
                    self.samples.append({
                        'cache_key': cache_key, 'frame_idx': f_idx,
                        'exp_name': exp_name, 'channel': ch
                    })
```

```
60.
61.     def load_yolo_boxes(self, sample, H, W):
62.         exp_name, ch, f_idx = sample['exp_name'], sample['channel'],
    sample['frame_idx']
63.         label_path = os.path.join(self.exp_path_map[exp_name],
    f"ch{ch}_frames", f"frame_{f_idx:05d}.txt")
64.         boxes = []
65.         if os.path.exists(label_path):
66.             with open(label_path, 'r') as f:
67.                 for line in f.readlines():
68.                     parts = list(map(float, line.strip().split()))
69.                     if len(parts) >= 5:
70.                         xc, yc, w, h = parts[1], parts[2], parts[3],
    parts[4]
71.                         x1, y1 = (xc - w/2)*W, (yc - h/2)*H
72.                         x2, y2 = (xc + w/2)*W, (yc + h/2)*H
73.                         boxes.append([max(0, x1), max(0, y1), min(W,
    x2), min(H, y2)])
74.         return {"boxes": torch.tensor(boxes, dtype=torch.float32) if
    boxes else torch.empty((0, 4))}
75.
76.     def __len__(self): return len(self.samples)
77.     def __getitem__(self, idx):
78.         s = self.samples[idx]
79.         raw = self.data_cache[s['cache_key']][s['frame_idx']]
80.         return torch.tensor(raw).float().unsqueeze(0),
    self.load_yolo_boxes(s, raw.shape[0], raw.shape[1])
81.
82. class ProcessedProjectDataset(RawProjectDataset):
83.     def __init__(self, root_dir, excel_file,
    bg_exp_name='no_man_3_sec'):
84.         super().__init__(root_dir, excel_file)
85.         self.bg_ref = {}
86.         print("Computing Background Models...")
87.         for key, data in self.data_cache.items():
88.             if key.startswith(bg_exp_name):
89.                 try:
90.                     self.bg_ref[int(key.split('_ch')[-1])] =
    cv2.GaussianBlur(np.mean(data, axis=0), (3,3), 0)
91.                 except: pass
92.
93.     def __getitem__(self, idx):
94.         raw_t, target = super().__getitem__(idx)
95.         raw = raw_t.squeeze(0).numpy()
96.         ch = int(self.samples[idx]['cache_key'].split('_ch')[-1])
97.
98.         # Background Subtraction
99.         blurred = cv2.GaussianBlur(raw, (3,3), 0)
```

```
            proc = np.abs(blurred - self.bg_ref[ch]) if ch in
   self.bg_ref else blurred
            return torch.tensor(proc).float().unsqueeze(0), target

    # ========================================
    # 2. Evaluation & Algorithm Logic
    # ========================================
    def evaluate_and_visualize(dataset,
   output_folder="debug_output"):
        if not os.path.exists(output_folder):
            os.makedirs(output_folder)

        print(f"Saving visualization to: {output_folder}")

        frame_ious = []
        frame_gt_fire = []
        frame_pred_fire = []

        # --- OPTIMIZED HYPERPARAMETERS ---
        # Derived from Grid Search optimization
        NOISE_FLOOR = 1.5           # Variance threshold to reject
   sensor noise
        KERNEL = np.ones((3,3), np.uint8)
        DILATION_ITERS = 2          # Expands blob to match physical
   object size
        # --------------------------------

        for i in tqdm(range(len(dataset))):
            img_t, target = dataset[i]
            img = img_t.squeeze().numpy()

            # 1. Visualization Setup (Upscale for readability)
            norm_display = cv2.normalize(img, None, 0, 255,
   cv2.NORM_MINMAX).astype(np.uint8)
            display_img = cv2.cvtColor(norm_display,
   cv2.COLOR_GRAY2BGR)
            H, W = img.shape
            SCALE = 20
            display_large = cv2.resize(display_img, (W * SCALE, H *
   SCALE), interpolation=cv2.INTER_NEAREST)

            # 2. Ground Truth Processing
            gt_mask = np.zeros_like(img, dtype=np.uint8)
            boxes = target['boxes'].numpy()
            has_fire_gt = len(boxes) > 0

            if has_fire_gt:
                for box in boxes:
```

```
                    x1, y1, x2, y2 = box.astype(int)
                    gt_mask[y1:y2, x1:x2] = 1
                    cv2.rectangle(display_large, (x1*SCALE,
   y1*SCALE), (x2*SCALE, y2*SCALE), (0, 255, 0), 2)

            # --- 3. PROCESSING PIPELINE (THE CORE ALGORITHM) ---
            data_min, data_max = img.min(), img.max()
            diff = data_max - data_min

            # A. Variance Gating (Noise Floor)
            if diff > NOISE_FLOOR:
                # B. Dynamic Range Normalization
                norm = ((img - data_min) / diff *
   255).astype(np.uint8)

                # C. Otsu Thresholding
                _, otsu_mask = cv2.threshold(norm, 0, 1,
   cv2.THRESH_BINARY + cv2.THRESH_OTSU)

                # D. Morphological Shaping
                # Step 1: Erode (Filter high-freq noise)
                otsu_mask = cv2.erode(otsu_mask, KERNEL,
   iterations=1)
                # Step 2: Dilate (Restore shape & expand to bounding
   box)
                otsu_mask = cv2.dilate(otsu_mask, KERNEL,
   iterations=DILATION_ITERS)

            else:
                # Frame rejected as background noise
                otsu_mask = np.zeros_like(img, dtype=np.uint8)
            # ------------------------------------------------

            # 4. Metrics Calculation
            has_fire_pred = np.any(otsu_mask == 1)

            # Draw Prediction (Red)
            contours, _ = cv2.findContours(otsu_mask,
   cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
            scaled_contours = [cnt * SCALE for cnt in contours]
            cv2.drawContours(display_large, scaled_contours, -1, (0,
   0, 255), 2)

            intersection = np.logical_and(gt_mask, otsu_mask).sum()
            union = np.logical_or(gt_mask, otsu_mask).sum()
            iou = 1.0 if (union == 0 and not has_fire_pred) else
   (intersection / union if union > 0 else 0.0)

```

```
180.             frame_ious.append(iou)
181.             frame_gt_fire.append(has_fire_gt)
182.             frame_pred_fire.append(has_fire_pred)
183.
184.             # 5. Save Debug Image (Sampled)
185.             if i % 20 == 0 or (has_fire_gt and iou < 0.3):
186.                 cv2.putText(display_large, f"IoU: {iou:.2f}", (10,
    30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
187.                 save_path = os.path.join(output_folder,
    f"frame_{i:04d}.png")
188.                 save_image_safe(save_path, display_large)
189.
190.         # 6. Aggregated Results
191.         return {
192.             "Mean IoU": np.mean(frame_ious),
193.             "Frame Accuracy": accuracy_score(frame_gt_fire,
    frame_pred_fire),
194.             "Frame Recall": recall_score(frame_gt_fire,
    frame_pred_fire, zero_division=0),
195.             "Frame Precision": precision_score(frame_gt_fire,
    frame_pred_fire, zero_division=0),
196.             "Frame F1-Score": f1_score(frame_gt_fire,
    frame_pred_fire, zero_division=0)
197.         }
198.
199.     # ========================================
200.     # 3. Execution Entry Point
201.     # ========================================
202.     if __name__ == "__main__":
203.         # Project Paths
204.         ROOT = r"C:\Users\roylb\Desktop\Afeka\שנה ד\סמסטר א\prjct
    gmr\Final Project\Dataset\dataset_raw"
205.         LABELS = r"C:\Users\roylb\Desktop\Afeka\שנה ד\סמסטר א\prjct
    gmr\Final Project\Dataset\dataset_raw\labels.xlsx"
206.
207.         # Set output directory to script location
208.         script_location = os.path.dirname(os.path.abspath(__file__))
209.         debug_folder_path = os.path.join(script_location,
    "debug_output")
210.
211.         print(f"Output Directory: {debug_folder_path}")
212.
213.         if os.path.exists(ROOT):
214.             ds_processed = ProcessedProjectDataset(ROOT, LABELS)
215.             results = evaluate_and_visualize(ds_processed,
    output_folder=debug_folder_path)
216.
217.             print("\n" + "="*40)
```

```
            print(" FINAL ALGORITHM PERFORMANCE")
            print("="*40)
            print(f"Mean IoU:        {results['Mean IoU']:.4f}")
            print(f"Frame F1-Score:  {results['Frame F1-
   Score']:.4f}")
            print(f"Frame Recall:    {results['Frame Recall']:.4f}")
            print(f"Frame Precision: {results['Frame
   Precision']:.4f}")
            print(f"Frame Accuracy:  {results['Frame
   Accuracy']:.4f}")
            print("="*40)
        else:
            print("Error: Dataset path not found.")
```

8.2.3 System Initializtion and Data Recording Script

```
import time
import os
import threading
import datetime
import board
import busio
import numpy as np
import cv2
from tkinter import *
from tkinter import messagebox
from PIL import Image, ImageTk

# Hardware Imports
import adafruit_tca9548a
import adafruit_mlx90640

# --- Configuration ---
REFRESH_RATE = adafruit_mlx90640.RefreshRate.REFRESH_16_HZ
FPS_TARGET = 16   # Target FPS for the output video
I2C_FREQ = 800000 # High speed I2C
CHANNELS = [0, 2] # Active Sensor Channels
OUTPUT_DIR = "dataset_raw"

class ThermalRecorderApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Stereo Thermal Data Recorder")
        self.root.geometry("1000x700")

        # --- State Variables ---
        self.running = True
        self.is_recording = False
        self.recording_start_time = 0
        self.recording_buffer = []  # RAM buffer for frames

        self.latest_frames = {}
        self.lock = threading.Lock()

        if not os.path.exists(OUTPUT_DIR):
            os.makedirs(OUTPUT_DIR)

        # --- GUI Layout ---
        # 1. Top Control Bar
        control_frame = Frame(root, pady=10, padx=10, bg="#dddddd")
        control_frame.pack(side=TOP, fill=X)

        # Label Entry
```

```
48.         Label(control_frame, text="Activity Label:", bg="#dddddd",
    font=("Arial", 12)).pack(side=LEFT)
49.         self.label_var = StringVar(value="activity_walk")
50.         self.label_entry = Entry(control_frame,
    textvariable=self.label_var, width=20, font=("Arial", 12))
51.         self.label_entry.pack(side=LEFT, padx=10)
52.
53.         # Record Toggle Button
54.         self.btn_record = Button(control_frame, text="START
    RECORDING", command=self.toggle_recording,
55.                                  bg="green", fg="white",
    font=("Arial", 12, "bold"), width=20)
56.         self.btn_record.pack(side=LEFT, padx=20)
57.
58.         # Status & Timer
59.         self.status_label = Label(control_frame, text="Ready",
    bg="#dddddd", font=("Arial", 12))
60.         self.status_label.pack(side=LEFT, padx=20)
61.
62.         self.timer_label = Label(control_frame, text="00:00",
    bg="#dddddd", font=("Courier", 14, "bold"))
63.         self.timer_label.pack(side=LEFT, padx=10)
64.
65.         # 2. Video Display Area
66.         self.canvas_frame = Frame(root, bg="black")
67.         self.canvas_frame.pack(expand=True, fill=BOTH)
68.
69.         self.display_label = Label(self.canvas_frame, bg="black")
70.         self.display_label.pack(expand=True)
71.
72.         self.root.bind('<q>', lambda e: self.on_close())
73.         self.root.bind('<space>', lambda e: self.toggle_recording())
    # Spacebar shortcut
74.
75.         # --- Initialization ---
76.         self.init_hardware()
77.
78.         # Start Sensor Thread
79.         self.thread = threading.Thread(target=self.sensor_loop,
    daemon=True)
80.         self.thread.start()
81.
82.         # Start GUI Update Loop
83.         self.update_gui()
84.
85.     def init_hardware(self):
86.         try:
87.             print("Initializing I2C...")
```

```
            i2c = busio.I2C(board.SCL, board.SDA, frequency=I2C_FREQ)
            mux = adafruit_tca9548a.TCA9548A(i2c)

            self.sensors = {}
            for ch in CHANNELS:
                try:
                    sensor = adafruit_mlx90640.MLX90640(mux[ch])
                    sensor.refresh_rate = REFRESH_RATE
                    self.sensors[ch] = sensor
                    print(f"Sensor Ch{ch} initialized.")
                except Exception as e:
                    print(f"Failed to init Ch{ch}: {e}")
                        self.sensors[ch] = None
            except Exception as e:
                print(f"Critical Hardware Error: {e}")
                self.status_label.config(text="HW Error!", fg="red")

        def toggle_recording(self):
            """Switches between Recording and Saving states."""
            if not self.is_recording:
                # START RECORDING
                label = self.label_var.get().strip()
                if not label:
                    messagebox.showwarning("Missing Label", "Please
   enter an Activity Label before recording.")
                    return

                self.recording_buffer = [] # Clear buffer
                self.recording_start_time = time.time()
                self.is_recording = True

                # Update UI
                self.btn_record.config(text="STOP RECORDING",
   bg="red")
                self.status_label.config(text=f"Recording: {label}",
   fg="red")
                self.label_entry.config(state='disabled') # Lock
   label during record

            else:
                # STOP RECORDING
                self.is_recording = False
                self.btn_record.config(text="Saving...", bg="gray",
   state='disabled')
                self.label_entry.config(state='normal')

                # Launch Save Thread
```

```
                save_thread =
    threading.Thread(target=self.save_buffer_to_disk)
                save_thread.start()

        def sensor_loop(self):
            """Continuously reads frames and appends to buffer if
    recording."""
            frame_buffer = [0] * 768

            while self.running:
                temp_raw_storage = {}
                temp_vis_storage = {}
                temp_display_imgs = []

                for ch in CHANNELS:
                    sensor = self.sensors.get(ch)
                    if sensor:
                        try:
                            sensor.getFrame(frame_buffer)
                            data_array =
    np.array(frame_buffer).reshape((24, 32))

                            # Store raw data
                            temp_raw_storage[ch] = data_array.copy()

                            # --- Visualization ---
                            t_min = np.min(data_array)
                            t_max = np.max(data_array)
                            # Normalize to 0-255
                            norm = (data_array - t_min) / (t_max -
    t_min + 0.001) * 255
                            norm = norm.astype(np.uint8)

                            # Upscale for UI/Video (320x240)
                            upscaled = cv2.resize(norm, (320, 240),
    interpolation=cv2.INTER_CUBIC)
                            colormap = cv2.applyColorMap(upscaled,
    cv2.COLORMAP_INFERNO)

                            # Add Overlay
                            cv2.putText(colormap, f"Ch{ch} |
    {t_min:.1f}C - {t_max:.1f}C", (10, 20),
                                        cv2.FONT_HERSHEY_SIMPLEX,
    0.5, (255, 255, 255), 1)

                            temp_vis_storage[ch] = colormap.copy() #
    Keep BGR for OpenCV saving

```

```
                            # Convert to RGB for Tkinter
                            rgb_img = cv2.cvtColor(colormap,
    cv2.COLOR_BGR2RGB)
                            temp_display_imgs.append(rgb_img)

                        except Exception as e:
                            # print(f"Read error Ch{ch}: {e}") #
    Suppress spam
                            pass
                    else:
                        # Placeholder if sensor missing
                        blank = np.zeros((240, 320, 3),
    dtype=np.uint8)
                        temp_display_imgs.append(blank)

                # Update shared state for GUI
                with self.lock:
                    if temp_display_imgs:
                        self.latest_frames =
    np.hstack(temp_display_imgs)

                    # --- RECORDING BUFFER ---
                    if self.is_recording:
                        packet = {
                            "timestamp": time.time(),
                            "raw": temp_raw_storage, # {ch: 24x32
    array}
                            "vis": temp_vis_storage  # {ch: 320x240
    BGR image}
                        }
                        self.recording_buffer.append(packet)

        def update_gui(self):
            """Updates the video feed and the timer."""
            # Update Video
            with self.lock:
                if isinstance(self.latest_frames, np.ndarray):
                    img_pil = Image.fromarray(self.latest_frames)
                    img_tk = ImageTk.PhotoImage(image=img_pil)
                    self.display_label.config(image=img_tk)
                    self.display_label.image = img_tk

            # Update Timer
            if self.is_recording:
                elapsed = int(time.time() -
    self.recording_start_time)
                mins, secs = divmod(elapsed, 60)
```

```
209.                 self.timer_label.config(text=f"{mins:02}:{secs:02}",
    fg="red")
210.             else:
211.                 self.timer_label.config(text="00:00", fg="black")
212.
213.             if self.running:
214.                 self.root.after(30, self.update_gui)
215.
216.         def save_buffer_to_disk(self):
217.             """Process buffer: Create folders, Save NPZ, MP4, and
    individual PNGs."""
218.             if not self.recording_buffer:
219.                 self._finish_saving()
220.                 return
221.
222.             # 1. Setup Directory Structure
223.             # Structure: output / label_name / timestamp / [data]
224.             label_name = self.label_var.get().strip()
225.             timestamp_str =
    datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
226.
227.             session_path = os.path.join(OUTPUT_DIR, label_name,
    timestamp_str)
228.             os.makedirs(session_path, exist_ok=True)
229.
230.             self.status_label.config(text=f"Saving
    {len(self.recording_buffer)} frames...", fg="blue")
231.             print(f"--- Saving Session to: {session_path} ---")
232.
233.             # 2. Pivot Data (Organize by channel)
234.             channel_data = {ch: {"raw": [], "vis": []} for ch in
    CHANNELS}
235.
236.             for packet in self.recording_buffer:
237.                 for ch in CHANNELS:
238.                     if ch in packet["raw"]:
239.                         channel_data[ch]["raw"].append(packet["raw"][
    ch])
240.                         channel_data[ch]["vis"].append(packet["vis"][
    ch])
241.
242.             # 3. Write Files per Channel
243.             for ch, data in channel_data.items():
244.                 if not data["raw"]: continue
245.
246.                 print(f"Processing Channel {ch}...")
247.
248.                 # --- A. Save Raw Data (Stacked NPZ) ---
```

```
249.                # Instead of 1000 tiny files, we save one 3D array
    (Frames, 24, 32)
250.                # This is much faster and cleaner for analysis.
251.                raw_stack = np.array(data["raw"])
252.                npz_path = os.path.join(session_path,
    f"ch{ch}_raw_data.npz")
253.                np.savez_compressed(npz_path, frames=raw_stack)
254.
255.                # --- B. Save Video (MP4) ---
256.                mp4_path = os.path.join(session_path,
    f"ch{ch}_video.mp4")
257.                height, width, _ = data["vis"][0].shape
258.                fourcc = cv2.VideoWriter_fourcc(*'mp4v')
259.                out_video = cv2.VideoWriter(mp4_path, fourcc,
    FPS_TARGET, (width, height))
260.
261.                # --- C. Save Individual PNGs (Folder) ---
262.                png_folder = os.path.join(session_path,
    f"ch{ch}_frames")
263.                os.makedirs(png_folder, exist_ok=True)
264.
265.                for i, frame in enumerate(data["vis"]):
266.                    # Write video frame
267.                    out_video.write(frame)
268.
269.                    # Write PNG frame
270.                    png_name = os.path.join(png_folder,
    f"frame_{i:05d}.png")
271.                    cv2.imwrite(png_name, frame)
272.
273.                out_video.release()
274.
275.            print("Save Complete.")
276.            self._finish_saving()
277.
278.        def _finish_saving(self):
279.            """Reset UI after saving is done (called from save
    thread)."""
280.            def _reset_ui():
281.                self.btn_record.config(text="START RECORDING",
    bg="green", state='normal')
282.                self.status_label.config(text="Ready", fg="green")
283.                self.recording_buffer = [] # Free RAM
284.
285.            # Schedule UI update on main thread
286.            self.root.after(0, _reset_ui)
287.
288.        def on_close(self):
```

```
            self.running = False
            self.root.destroy()

    if __name__ == "__main__":
        root = Tk()
        app = ThermalRecorderApp(root)
        root.protocol("WM_DELETE_WINDOW", app.on_close)
        root.mainloop()
```

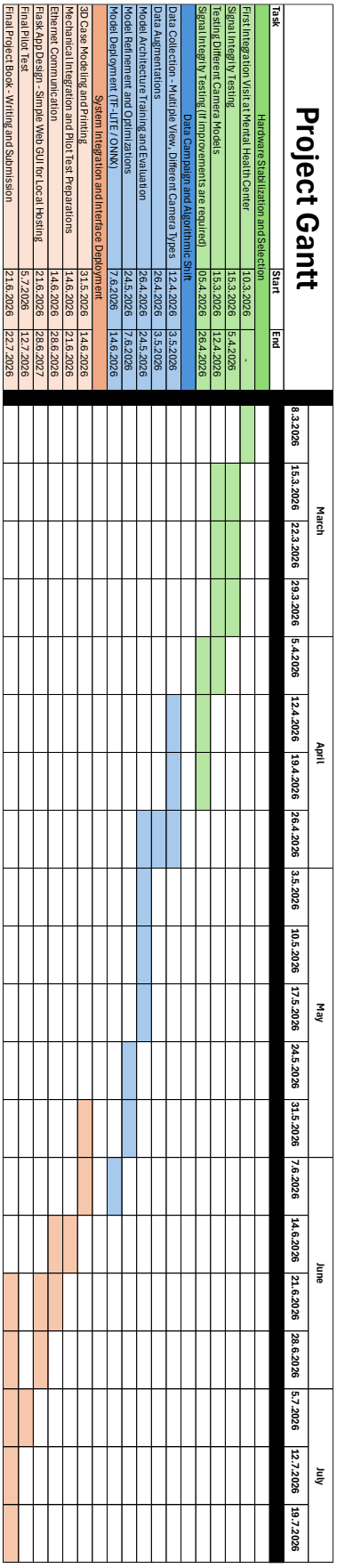
8.3 Scientific Abstract for Research Paper

Staff in psychiatric facilities and closed wards face a significantly higher risk of violence compared to other medical environments. Current security measures are inaccurate (high false positives), rely on human supervision (limited staff availability), and are limited by law regarding patient privacy. To address this, this project proposes the development and evaluation of an intelligent, privacy-preserving alert system utilizing thermal sensors, eliminating the need for intrusive monitoring and reducing work overload for the medical staff.

The primary objective is to design a functional prototype capable of real-time anomaly detection through thermal pattern analysis. Following consultation with clinical professionals, the system targets four critical scenarios: violent acts, sexual behavior, fire ignition, and unauthorized entry into restricted areas. Architecture prioritizes edge computing to ensure immediate alert generation without external data transmission, thereby enhancing security and privacy.

This project tests several advanced thermal image processing techniques and algorithms, with a few sensor types and attempts to find the optimal pairing for each problem. Up until now, classical computer vision algorithms have proved to be somewhat ineffective, failing to detect anomalies correctly. In the future, we will attempt to optimize current algorithms and implement machine and deep learning algorithms, hoping to achieve better results while still managing efficient computation on an edge system.

8.4 Work Plan (Gantt)



8.5 Table of Changes

This chapter describes the changes made to the project since the submission of the Statement of Work document.

Change Made	Relevant Section	Date of Approval	Change Approved By
PCB requirement was removed from the project since it was deemed to be unnecessary, given that breakout board sensors worked seamlessly.	Introduction – Goals, Objectives and Metrics	21/1/2026	Or Zilberberg – Project supervisor Avi Rabinovich – Project manager
Metrics section revised – definitions tightened, clarified and refined.	Introduction – Goals, Objectives and Metrics	7/2/2026	Or Zilberberg – Project supervisor